

FEST™

Fast & Easy & Safe Tool

A USER'S GUIDE

- Copyright © GES
- 이 Manual 의 저작권은 (주)GES 에 있습니다.
- 이 Manual 은 (주)GES 가 공급하는 FEST™를 이용하여 장비 제어 SW 를 개발하고자 하는 User 를 Support 하는 목적으로 제작되었습니다.

- FEST™ is a trademark of **GES**(**G**lobal **E**ngineering **S**olutions) Corporation, KOREA.
- Windows™ NT™, 2000, XP and 7 are trademarks of Microsoft Corporation, USA.

FEST - A USER'S GUIDE

GES (Global engineering solutions) Limited

- Revision History -

2010.01.01 : First Release

Contents

Part1 : Basic

Chapter 1. FEST 소개	6
Chapter 2. 개발 환경 준비	8
Chapter 3. 기초 개념	13
Chapter 4. SW Startup	17
Chapter 5. IO	21
Chapter 6. Driver	27
Chapter 7. Sequence	32
Chapter 8. Interlock	39
Chapter 9. Alarm	42
Chapter 10. GUI	46
Chapter 11. Screen Navigation	61

Part2 : Advance

Chapter 12. RS232 Driver	67
Chapter 13. TCP/IP Driver	75

Part3 : Utility

Chapter 14. FEST Control Window	84
Chapter 15. Alarm Manager	91
Chapter 16. Chart Manager	98
Chapter 17. Job Log Manager	101
Chapter 18. ETC Log Manager	104
Chapter 19. Online Log Viewer	107

Part4 : ETC

Chapter 20. System 예약 IO	112
Chapter 21. 제한사항	114
Chapter 22. Scheduler Interface DLL	115

Basic

Chapter 1: FEST 소개

Chapter 2: 개발환경 준비

Chapter 3: 기본 배경 지식

Chapter 4: FEST Startup

Chapter 5: IO

Chapter 6: Driver

Chapter 7: Sequence

Chapter 8: Interlock

Chapter 9: Alarm

Chapter 10: GUI

Chapter 11: Screen Navigation

Chapter 1

FEST 소개

What's In this chapter...

- ♦ FEST is...
- ♦ FEST 특징
- ♦ FEST 구조
- ♦ 기타

■ FEST is...

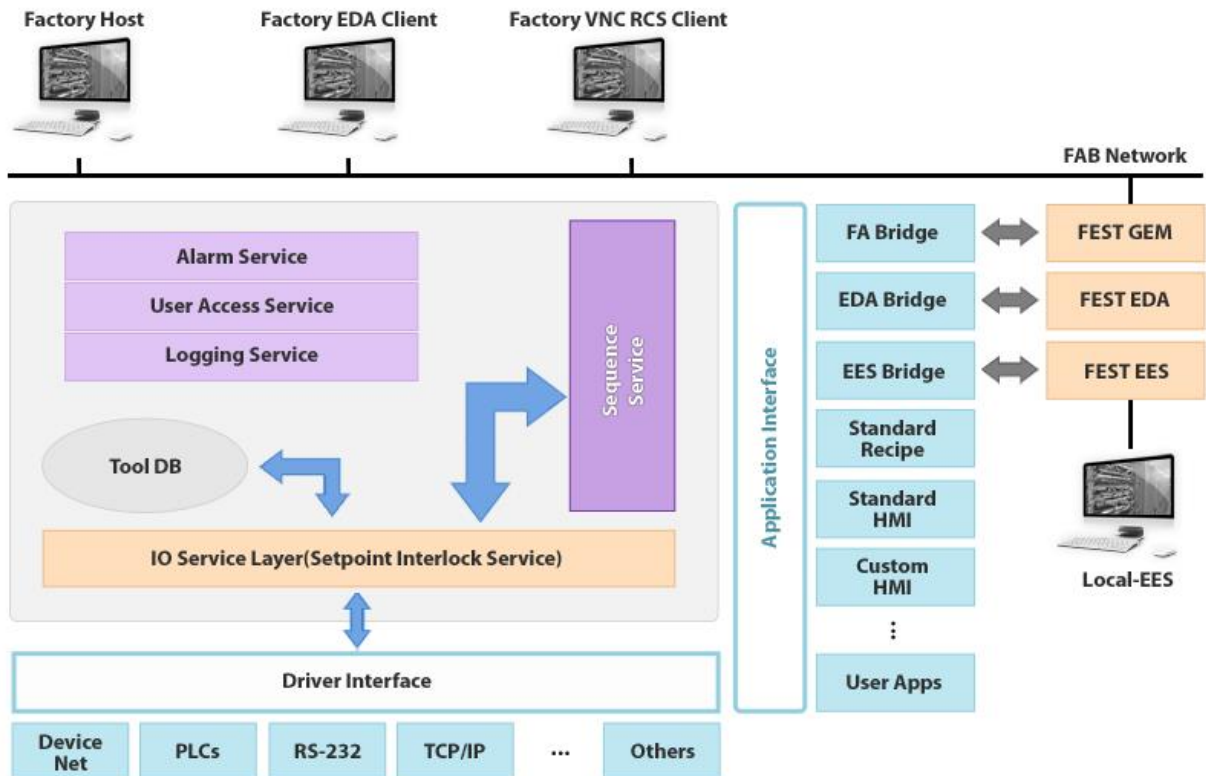
- 반도체/FPD/태양광/LED 등 여러 산업분야에서 필요로 하는 다양한 종류의 장비들에 대한 제어 및 자동화 솔루션을 제공하는 Control Framework
- 표준화 Sequence 및 Component 의 제공으로 Application 개발시간을 단축시켜 End User 의 요구사항을 빠르게 만족시킬 수 있다.
- 장비의 구성과 용도에 따라 아래의 제품으로 구성

- ❖ FEST CTC
- ❖ FEST TMC
- ❖ FEST PMC
- ❖ FEST GEM

■ FEST 특징

- C/C++/.Net Interface 와 Windows 2000/XP/7 지원
- C++ 바탕의 OOP 프로그래밍으로 유용성, 확장성이 우수(C lass 기반)
- SW Shutdown 없이 Sequence/GUI 수정 후 Upload 가능
- Script 형식의 Setpoint-Interlock/GUI/Recipe-Editor 제공
- GOI Table 을 이용한 편리한 GUI 구성과 재사용 용이
- Alarm/IO/Sequece/Setpoint 등록 개수 무제한
- 레벨에 따른 Screen 접근 설정이 용이한 User Access Control 제공
- 빠르고 쉽고 안정적인 개발을 위한 다양한 API/Utility 제공
- 디버깅을 위한 System/Operation 등 다양한 Log 지원

Control Platform



참조. FES Platform 구조 도식화

■ 구조

- CTC, TMC, PMCs 는 Ethernet 를 통한 안정적인 분산제어
- 장비의 복잡도에 따라 SW/HW 를 유연하게 구성 가능(통합/분리)
- HW Independent Driver 를 통한 유연한 SW 구현 용이
- Object-Oriented SW 구조에 따른 재사용 용이
- GUI(Graphic User Interface)가 SEMATECH 규정에 따름
- Windows 계열 운영체제를 지원하는 IO 모듈의 사용 가능
(Direct IO(AIO/DIO), Serial, DeviceNet, VME, PLC, etc...)
- Standard Interface Function 을 통한 skill up 용이

■ 기타

- FES™ is a trademark of **GES**(**G**lobal **E**ngineering **S**olutions) Corporation, KOREA.
- Windows™ NT™, 2000, XP and 7 are trademarks of Microsoft Corporation, USA.

Chapter 2

개발환경 준비

What's In this chapter...

- ◆ System 권장사항
- ◆ 개발환경
- ◆ FEST Engine 설치방법
- ◆ T Drive
- ◆ FEST Platform Directory 구조

■ System 권장사항

- Customize PC Hardware – User select
 - Intel Based PC Compatible
 - Pentium-3 300Mhz or Higher
 - 4GB RAM or Higher
 - 100 GB Hard Disk or Higher(Log 포함)
 - VGA 512M Color 1280x1024 Resolution or Higher
 - up to 65536 Color Mode(**Never use TRUE Color Mode**)
 - 15" SVGA Color Monitor or Higher(Support 1280x1024)
 - TCP/IP Network Communication
 - Internal CD-ROM Drive(at CTC)
- I/O device Module Support – User select(Direct I/O, PLC, VME, Device Net.....)

■ 개발환경

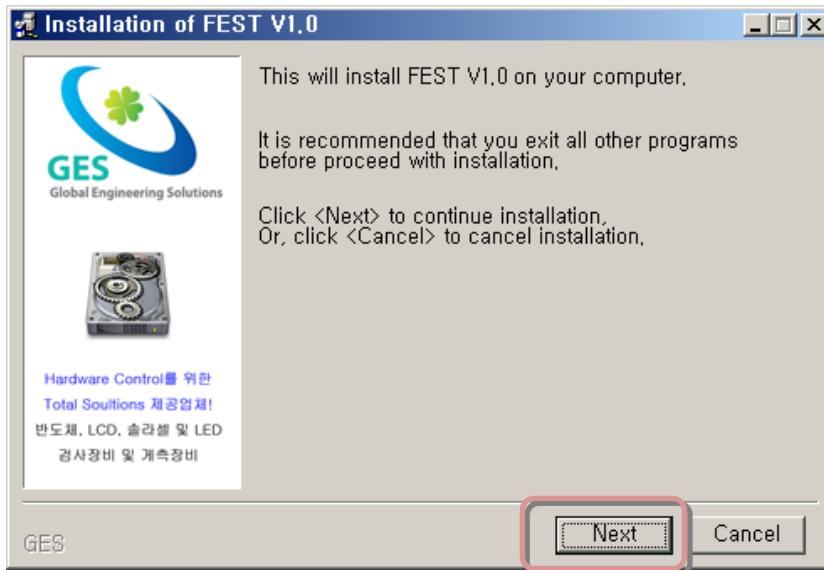
- Operating System – Windows™ NT™, 2000, XP and 7
- Development Tool

Microsoft Visual C++ 6.0
Microsoft Visual Studio 2005
Microsoft Visual Studio 2008
Microsoft Visual Studio 2010
Microsoft Visual Studio 2012

■ FEST Engine 설치방법

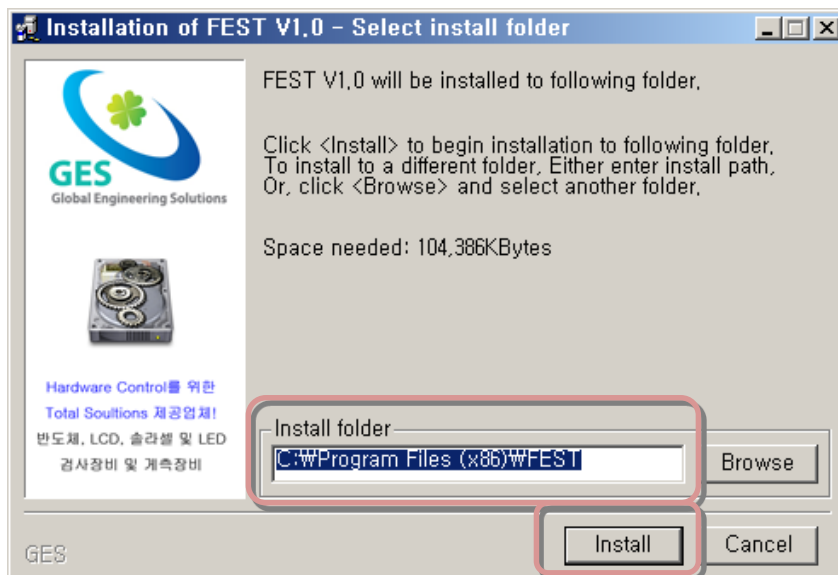
- 기본적으로 FEST SW Platform 위에서 SW 개발이 진행되기 때문에 실행 File, Library 등을 포함하고 있는 FEST Engine 을 먼저 PC 에 설치한다.
- Install Directory 에 FestSetup.exe File 을 수행시킨다.

1) 안내문



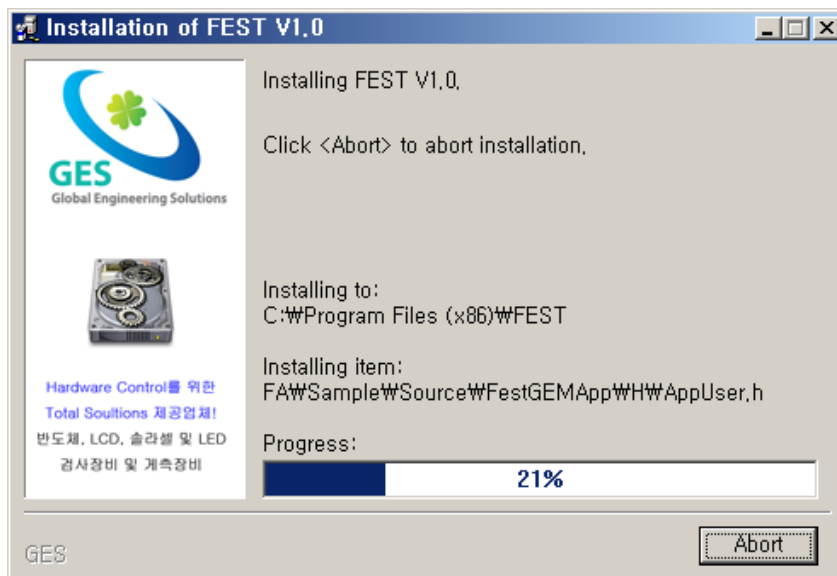
- Next 를 선택

2) Select Setup Directory



- 원하는 설치 Directory 를 선택
- Install 버튼을 click 하면 설치가 시작

3) File 복사중인 화면(Screen of copying files)



- Engine File 복사중

4) 완료(Install Finished)



- FEST Engine 설치 완료
- Ok 를 click 하여 Message box Close

■ T Drive

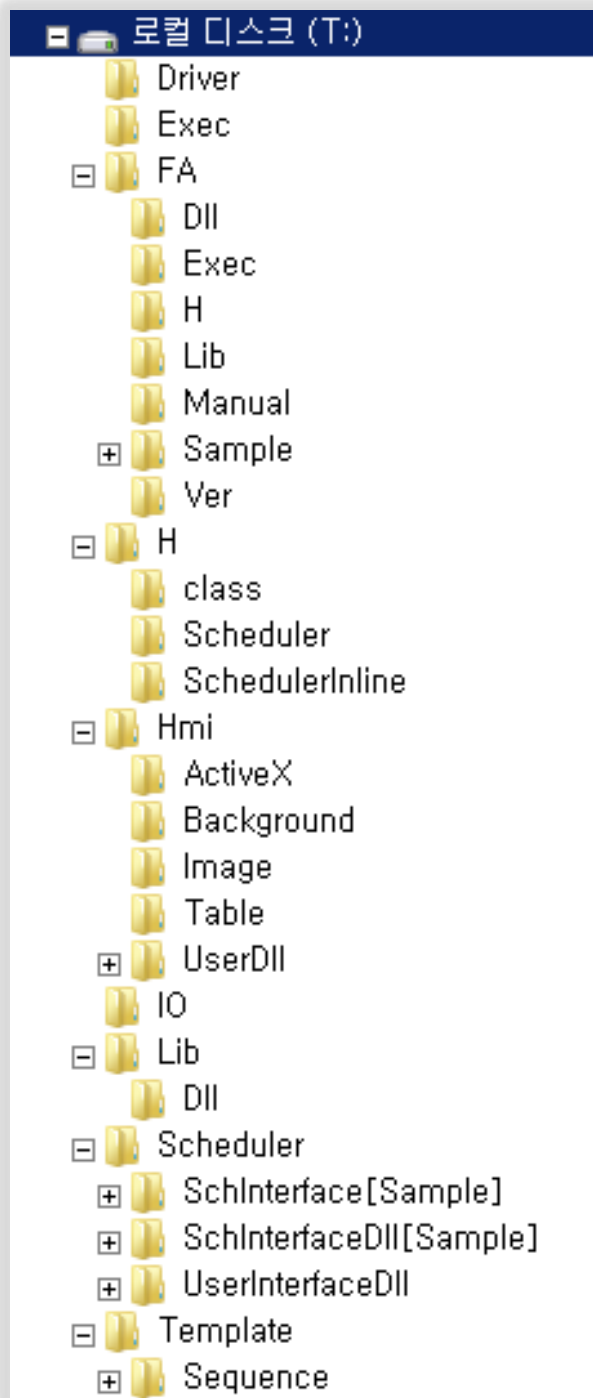


- Setup 이 완료되면 **아래와 같이** 시작 프로그램에 Make TDrive 라는 Batch file 이 등록된다. FEST Engine 을 설치한 Directory 가 가상 T Drive 로 등록된다. System 이 Booting 될 때 자동으로 수행되므로 삭제하지 않도록 주의한다.

- 위와 같이 T Drive 로 생성되는 FEST Engine 의 실제 경로는 User 가 지정한 Directory 에 따라 다르지만, 일반적인 경우 아래와 같다

C:\Program Files\FEST (= Drive T:\)

FEST Platform Directory 구조



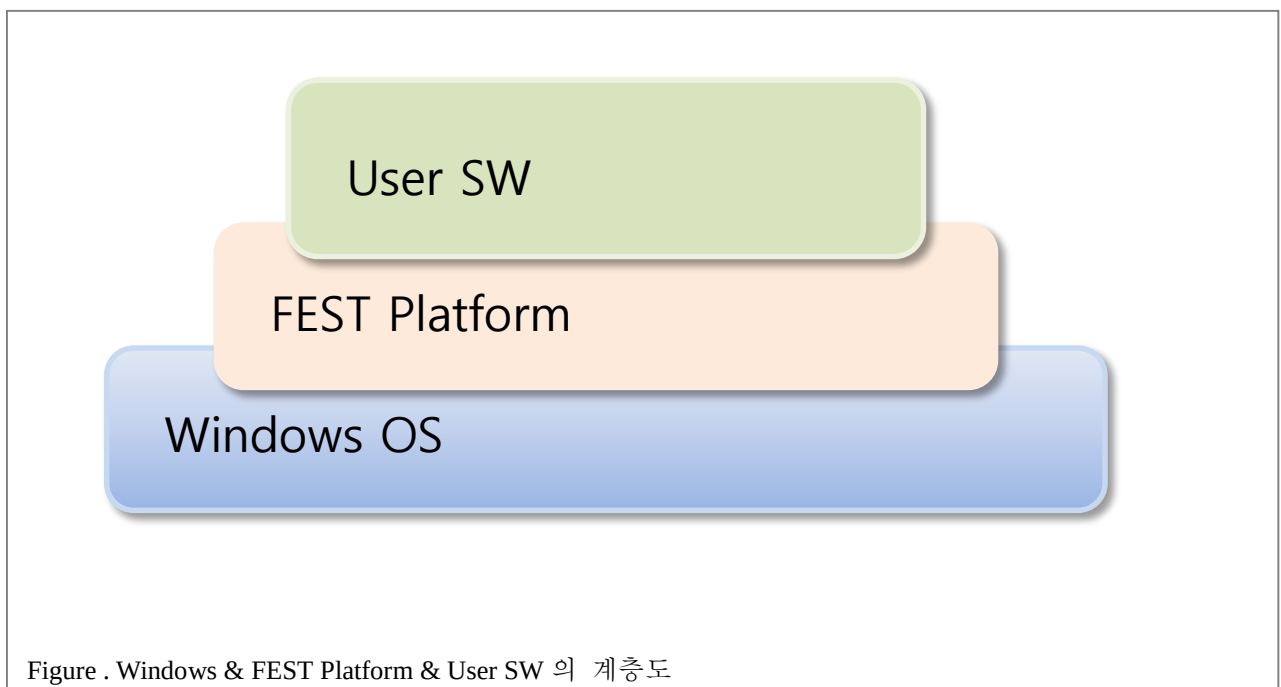
Chapter 3

기본 개념

What's In this chapter...

- ♦ FEST SW Layer
- ♦ Module Controller Architecture
- ♦ FEST™(CTC) - Module Controller Interface Architecture
- ♦ SW Design Flow

■ FEST SW Layer



Note

- FEST Platform 은 Windows OS 상에서 구동된다.
- User SW 는 FEST Platform 에서 제공하는 Resource 를 활용하여 개발되어지는 장비제어 SW 를 칭한다. 즉, 이 Manual 에서는 User SW 개발 방법을 설명한다.

■ Module Controller Architecture

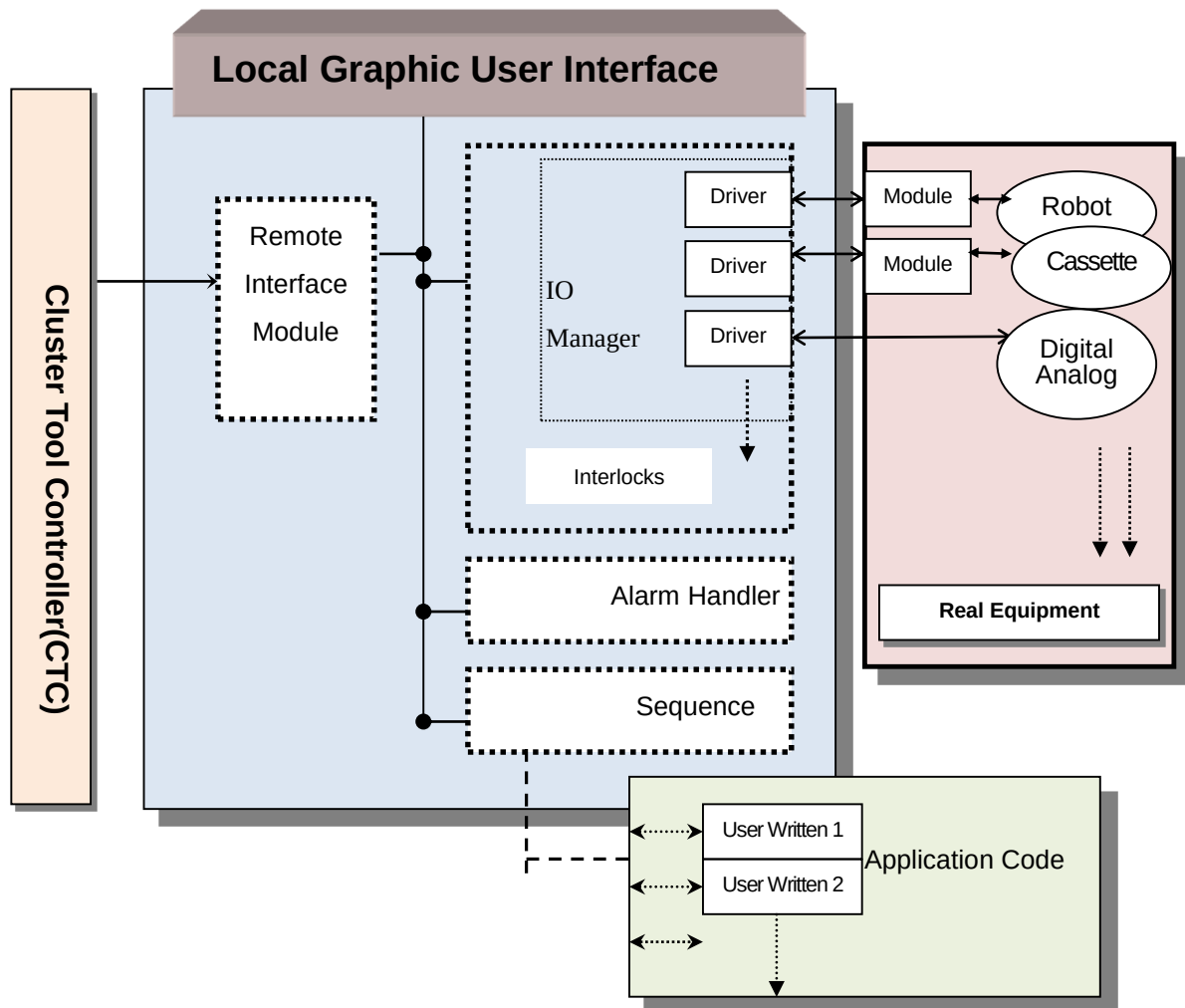


Figure. 각 Module 별로 구성되는 Control SW 의 구조도이다.

Note

- 한 개 단위의 Module Control SW 의 구조도이다.
Ex) PMC, TMC...

■ FEST™(CTC) - Module Controller Interface Architecture

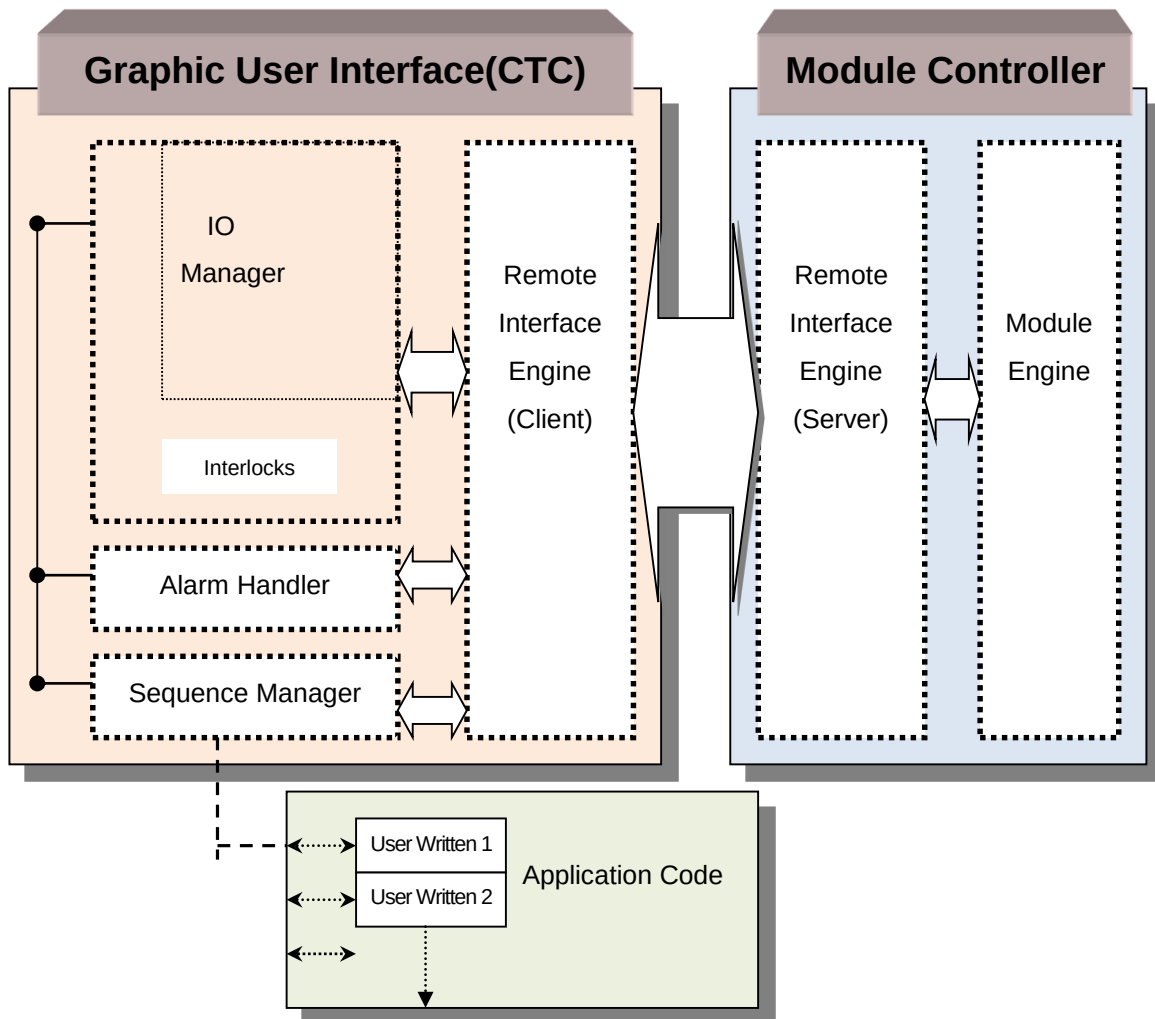


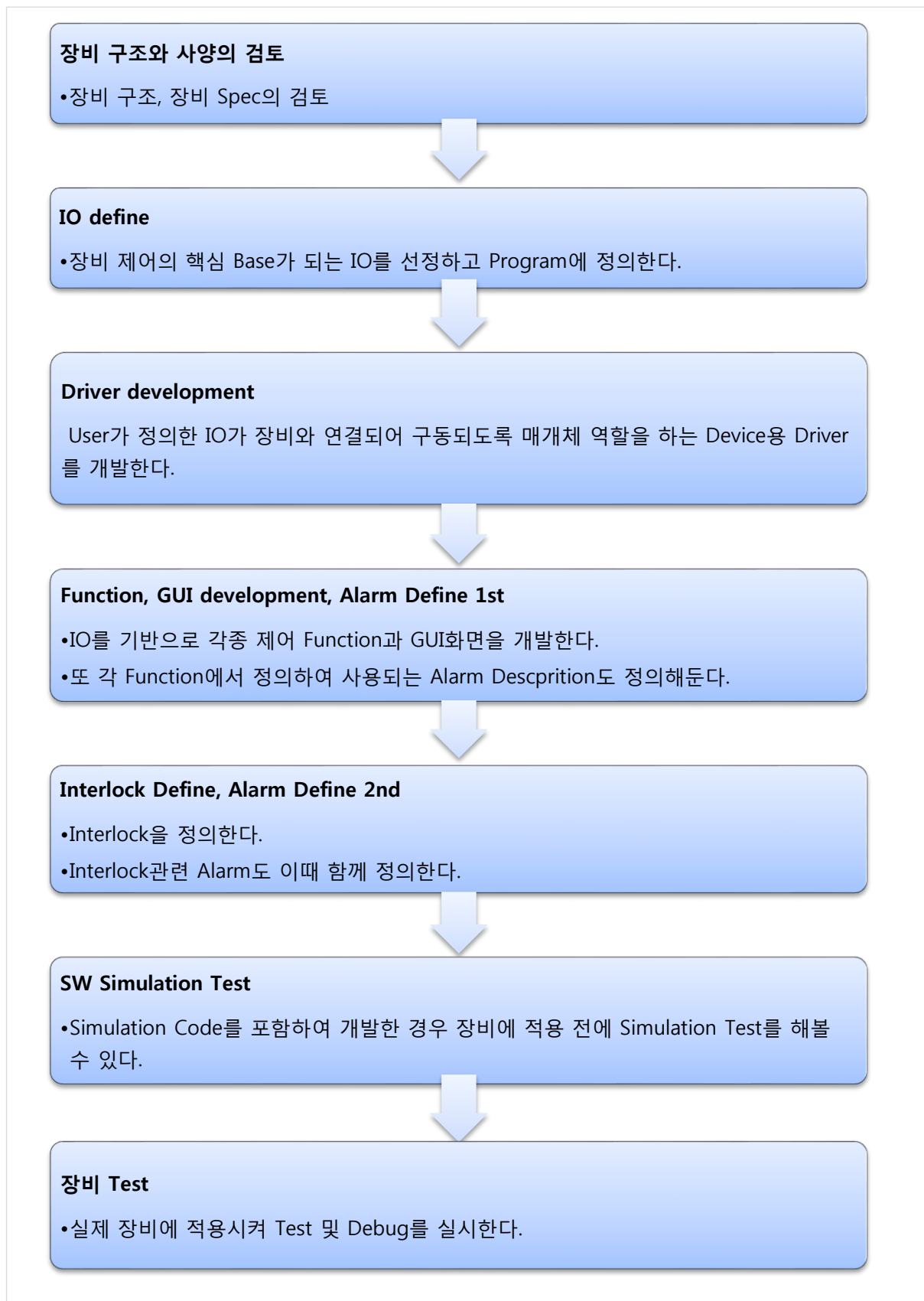
Figure. 각 Module Control SW 와 상위에서 각 Module 을 제어하는 CTC SW 와의 구조도이다.

Note

- 이전 page 의 한 개 단위의 Module Control SW (ex: PMC, TMC)를 중앙에서 관리해주는 CTC SW 와의 연결 구조도이다.

■ SW Design Flow

- FEST Platform 상에서 장비 제어 SW 개발을 할 경우 아래와 같은 흐름으로 진행하게 된다.



Chapter 4

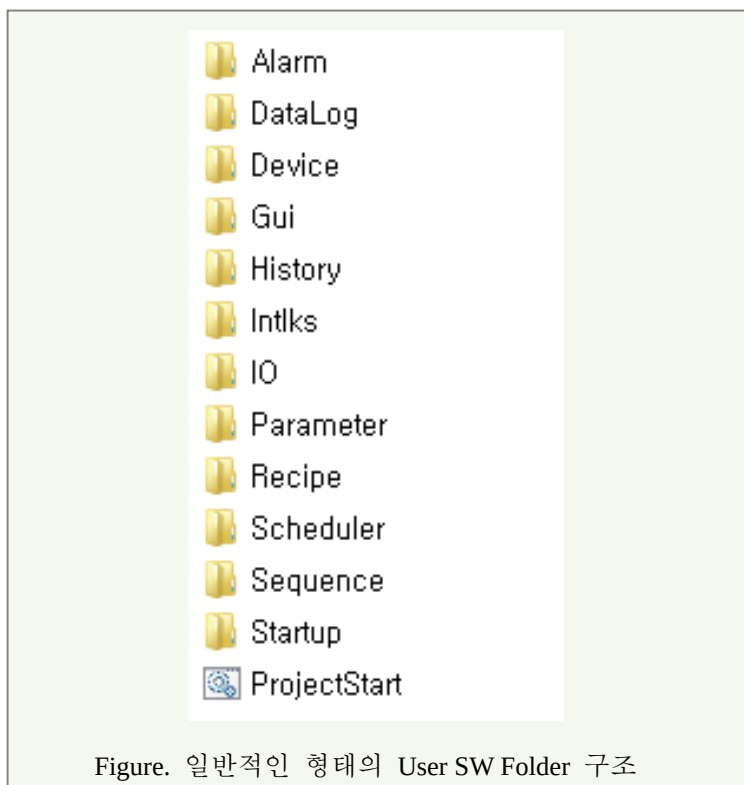
SW Start Up

What's In this chapter...

- ♦ User SW Folder 구조
- ♦ ProjectStart.bat
- ♦ Startup.cfg

User SW Folder 구조

- 하기 **Foler** 구조는 일반적인 형태의 User Develop SW 의 구조를 보여주고 있다.
- 이후 이 Manual 에서 Folder 및 File 설명시에는 아래 Folder 구조에 의거하여 설명하도록 한다.



Note

위의 Folder 구조는 통상적으로 개발되어지는 User SW 의 Folder 구조로써 GES 가 제안하고 있는 구조로써 User 의 필요에 의해 조정할 수 있다

ProjectStart.bat

- Figure4-1.에서 소개된 User SW Folder 구조에서 Program Startup 를 위한 File 인 ProjectStart.bat 파일의 내부구조에 대하여 설명한다.

```
@echo off
```

```
Startup\WaitTDrive
```

(SW 구동을 위해 FEST Engine 이 위치한 T Drive 가 존재하는지 체크한다)

```
set path=T:\Lib\dll;T:\FA\dll;%path%
```

(SW 에서 사용하는 FEST Engine Resource 용 Path 를 지정한다.)

```
T:\exec\CimMain.exe Startup\Startup.cfg
```

(FEST Main Engine 을 실행하며 Startup 환경 File 인 Startup\Startup.cfg File 을 참조하도록 지정한다)

```
Pause
```

Note

User SW 구동을 위한 FEST Main 실행 파일이 T:\Exec\CimMain.exe 가 되며 CimMain.exe 가 실행될 때에는 항상 환경 File 인 Startup.cfg 를 읽어들이 환경 File 에 정의된 정보들을 Memory 에 적재한다.

- ProjectStart.bat 을 실행하면 아래와 같은 Message Box 가 표시된다.

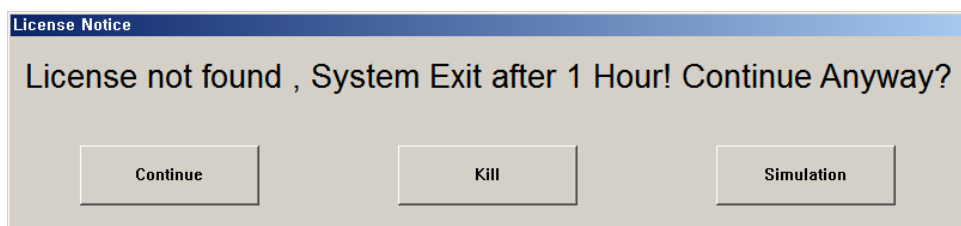


Figure. Project Start 시 Message Box

Note

Continue – 장비 제어 Mode 로 실행되어 실제 Device Driver 와 연결되어 실행된다.
Simulation – Simulation Mode 로 실행되어 모든 IO 가 가상 IO 로 동작된다.
(License 가 없는 경우 최대 1 시간 후 자동 Shutdown)

Startup.cfg

- Start Up File 은 FEST 를 시작할 때 시작 Directory 의 Startup.cfg File 로 시작되어 진다. Program 을 시작할 때 기본 구성을 위한 초기 File 로서 아래 Table 의 내용들을 정의해 주어야 한다.

No	1 st Tag	2 st Tag	Comment
1	SYSTEM	CTC_for_Demo	System Name(Must Unique name)
2	IO	IO/IO.list	IO 의 정보가 정의된 File (A file which defined the information of the IO)
3	INTERLOCK	INTLKS/Interlock.int	Setpoint Interlock 이 정의된 File (A file which defined the Setpoint Interlock)
4	SIMULATION	REAL or TRUE	REAL/TRUE 중 선택 (TRUE 시 Simulation)
5	SEQUENCE	Sequence/Sequence.dat	SEQUENCE 이 정의된 File (A file which defined the SEQUENCE)
6	ALARM	ALARM/Alarm.cfg	ALARM 에 관련된 정보가 정의된 File (A file which defined ALARM information)
7	HISTORY	History	각종 History Log 가 저장될 directory 를 정의한다
8	USER	GUI/LayOut/User.cfg	Account 등록시 Password 를 정의한 File
9	REMOTE_PORT	9001	Remote Module 에서 Connection 하기위한 Server Remote Port 를 선언한다.
10	HMI_ORIGIN	[1680,0]	[1680,0] : Screen 의 시작 Point 를 정의한다 정의하지 않으면 좌측상단 [0,0]이 default 이다

SYSTEM Definition

- 같은 PC 내에서 다른 장비의 Control 을 하는 경우를 위해 유일한 이름을 정의해 주어야 한다.
- 반드시 정의해 주어야 한다

(The system must be defined an unique name enable to control several equipments within same controller.)

ex) **SYSTEM** **CTC_for_Demo**

IO Definition

- IO 를 선언한 File 경로를 정의한다

ex) **IO** **IO/IO.list**

INTERLOCK Definition

- Interlock 를 선언한 File 경로를 정의한다

ex) **INTERLOCK** **INTLKS/Interlock.int**

SIMULATION Definition

- Simulation 사용 여부를 정의한다

ex) **SIMULATION** **TRUE** ← Simulation 으로 모든 Driver 가 Unload 됨
SIMULATION **REAL** ← Not Simulation

SEQUENCE Definition

- Sequence 를 선언한 File 경로를 정의한다

ex) **SEQUENCE** **Sequence/Sequence.dat**

ALARM Definition

- Alarm 를 선언한 File 경로를 정의한다

ex) **ALARM** **Alarm/Alarm.cfg**

HISTORY Definition

- Starting or Running 중에 발생하는 정보를 저장할 directory 를 정의한다

ex) **HISTORY** **History**

USER Definition

- User(Account) 등록 Password 를 선언한 File 경로를 정의한다

ex) **USER** **GUI/LayOut/User.cfg**

[User.cfg 내용] <= “GES” 로 Password 설정예제

#DEFAULT_PASSWORD **"GES"**

REMORT_PORT Definition

- Remote Port 를 정의한다

ex) **REMOTE_PORT** **9001**

HMI_ORIGIN Definition

- Screen 의 시작 Point 를 정의한다(X=1680, Y=0)
- 정의하지 않으면 Default [0,0] 이다.

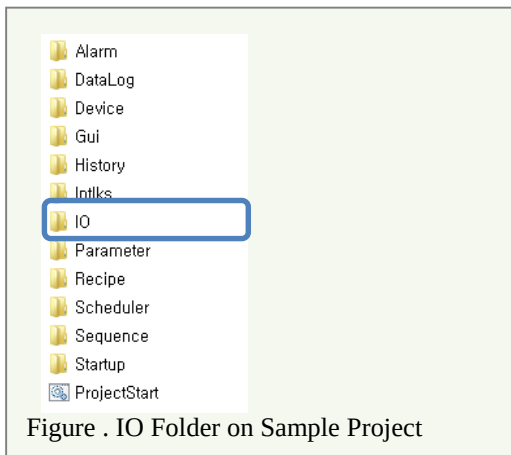
ex) **HMI_ORIGIN** **[1680,0]**

Chapter 5

IO

What's In this chapter...

- ♦ IO File
- ♦ IO File 의 연결
- ♦ IO Handling Driver 의 정의
- ♦ Enumeration 의 정의
- ♦ Digital / Analog / String IO 의 정의
- ♦ Lookup Table
- ♦ IO Driver 정의 예시
- ♦ IO Driver Load Option



IO File

- Program 에서 사용할 IO 관련 내용을 정의하는 File 이다.
- IO 는 Driver 를 통해 동작하고 갱신되며 SW 상에서 GUI, Sequence, Interlock 등에서 참조 및 사용되어진다. 즉 GUI, Sequence, Interlock 등을 작성하기 위해서는 모든 근간이 되는 IO 부터 형식에 맞추어 정의해두어야 한다.
- IO 는 DIGITAL , ANALOG , STRING 의 3 가지 Type 으로 구성되어 진다.

Type	내 용
DIGITAL	Enumeration 으로 표기하기를 원하는 IO, 소수점이 필요 없이 0 에서 99999 사이의 Range 에서 표시가 되어지는 IO 에 사용한다.
ANALOG	소수점이 필요한 IO, DIGITAL 에서의 Range 보다 더 크거나 작은 Range 가 필요한 IO, LookUp Table 형식으로 IO 의 표시가 필요한 IO, Data 의 Conversion 이 필요한 IO 등에 사용한다.
STRING	IO 의 내용을 String 형태로 표시하기 원할 때 사용하며 Output 의 경우는 String 수가 제한이 없으나 Input 의 경우는 99 자 까지만 표시되어 진다.

- IO 의 정의시의 Tag 의 의미는 다음과 같다.

No	Tag 1 st - 2 st		내 용	Comment
1	#INCLUDE		File Include	
2	#DECLARE DRIVER		IO Handling Driver 정의	
3	#DECLARE ENUM		Enumeration 정의	
4	#DECLARE LOOKUP		LOOKUP Table 정의	
5	#DECLARE IO			
		DI	DIGITAL Input IO 정의	
		DO	DIGITAL Output IO 정의	
		AI	ANALOG Input IO 정의	
		AO	ANALOG Output IO 정의	
		SI	STRING Input IO 정의	
		SO	STRING Output IO 정의	

■ IO File 의 연결

- IO File 을 아래와 같이 정의하여 해당 File 에 포함된 IO 들을 Loading 시 Memory 에 적재되도록 한다.

ex)

```
#INCLUDE IO\Driver\Robot\Robot.cfg
#include t:/IO/System.cfg
#include IO/Enumuration.con
```

- #INCLUDE 는 File 을 연결하여 사용하겠다는 tag 이고 t:/IO/System.cfg 는 연결되어질 File 이름이다.

- 연결되어지는 File 에서도 IO 의 관련내용을 정의하는 방식은 똑같이 사용되어 진다.

■ IO Handling Driver 의 정의

- Digital/Analog/String 으로 정의 된 IO 들은 각각 실제의 Input 또는 Output Point 를 가질 수 있다. 각각의 실제 Input or Output Point 를 정의하는 것이 Driver 라고 한다.

ex)

#DECLARE DRIVER

DRIVERNAME LOAD DLL IODriver.dll 1 2 3 4 5 6 7 8 9 10

- 위의 예를 기준으로 설명하는 Tag 의 의미는 다음과 같다.

Tag	표시(ex)	내 용	기타
1	#DECLARE	Fixed	Fixed
2	DRIVER	Fixed	Fixed
3	DRIVERNAME	Digital/Analog/String IO 에서 Interface 를 위해 사용할 Driver 이름	Max 8 Char
4	LOAD	LOAD/UNLOAD 중 하나를 선택(Select LOAD or UNLOAD) - LOAD: 5 번의 IO Handling 용 Driver(DLL File)를 Loading 한다. - UNLOAD:5 번의 IO Handling 용 Driver(DLL File)를 Loading 하지 않고 3 번의 DriverName 을 사용하는 IO 들을 모두 가 상 IO Channel 로 사용하게 한다.	
5	DLL	Driver dll 선언시	
	REMOTE	Remote Driver 선언시	
6	IODriver.dll	5 번이 DLL 선언시(dll path 정의)	
	%SYSTEM%	5 번이 REMOTE 선언시	
7	1	6 번이 LOADING 시 초기화를 위한 1 st Parameter(Com Port 번호)	
8	2	6 번이 LOADING 시 초기화를 위한 2 st Parameter(BaudRate)	
9	3	6 번이 LOADING 시 초기화를 위한 3 st Parameter(Data Bit)	
10	4	6 번이 LOADING 시 초기화를 위한 4 st Parameter(Stop Bit)	
11	5	6 번이 LOADING 시 초기화를 위한 5 st Parameter(Parity)	
12	6	6 번이 LOADING 시 초기화를 위한 6 st Parameter	
13	7	6 번이 LOADING 시 초기화를 위한 7 st Parameter	
14	8	6 번이 LOADING 시 초기화를 위한 8 st Parameter	
15	9	6 번이 LOADING 시 초기화를 위한 9 st Parameter	
16	10	6 번이 LOADING 시 초기화를 위한 10 st Parameter	

■ Enumeration 의 정의

- Enumeration 은 DIGITAL IO 에 한정되어 사용되며 DIGITAL IO 값의 숫자를 문자로 치환하여 보여 주고자 할 때 사용한다.

ex)

#DECLARE ENUM
CloseOpen Close Open

- 위의 예에서 “#DECLARE ENUM” 이후의 내용은 모두 Enum 으로 간주된다.
“CloseOpen Close Open” 이 부분이 실제 Enumeration 을 정의한 것이다.

- Enumeration 을 정의시

첫번째 Tag - DIGITAL IO 에서 사용하기 위한 Enumeration 의 이름

두번째부터 - 공백을 단위로 0 부터 값을 할당하여 User 에게는 String 으로 표시된다.

ex)

CloseOpen(Enum) Close(0) Open(1)

Digital / Analog / String IO 정의 방법

```
ValveStatus DI DRIVERNAME CloseOpen - d 1 2 3 4 Poll
ValveName DO DRIVERNAME CloseOpen - d 1 2 3 4 Poll
Press_Read AI DRIVERNAME - Torr .3f 1 2 3 4 Poll 0 1000 0 16383
Press_Set AO DRIVERNAME - Torr .3f 1 2 3 4 Poll 0 1000 0 16383
Response SI DRIVERNAME - - s 1 2 3 4 Poll
Command SO DRIVERNAME - - s 1 2 3 4 Poll
```

- 위의 예처럼 정의할 때 DI or DO Type 으로 선언된 것은 모두 Digital IO,
AI or AO Type 으로 선언된 것은 모두 Analog IO,
SI or SO Type 으로 선언된 것은 모두 String IO 이다.

- 위의 예시에 따른 Tag 의 내용은 다음과 같다.

Tag	Part	표시	내용
1	Name	ValveStatus ValveName	장비를 위한 IO 이름 (중복불가)
2	Type	DI/DO AI/AO SI/SO	장비에 사용할 IO 의 Type 을 정의한다
3	Driver	DRIVERNAME	- IO Interface 를 위해 사용할 Driver 이름 - DRIVER 선언하는 곳에 정의되어 있어야 한다 - “-“로 선언하였을 경우 Output Value 의 FeedBack 값이 된다, 즉, Virtual IO 이다
4	Enum	CloseOpen	- Enumeration Name - “-“로 표시했을 경우는 숫자로 표시
5	Unit	Torr	IO Unit 을 정의한다. Digital IO 와 같이 unit 이 없을 경우는 “-“로 정의
6	Format	d	Digital 의 Enumeration 으로 사용
		.3f	Analog 의 소수점 이하 자리수 표현
		s	String 의 character 로 정의
7	ID1	1	- IO Handling DRIVER 의 Input/Output Part 와 통신을 위한 1 st Parameter - “-“로 선언하였을 경우 Parameter 값은 -1
8	ID2	2	- IO Handling DRIVER 의 Input/Output Part 와 통신을 위한 2 st Parameter - “-“로 선언하였을 경우 Parameter 값은 -1
9	ID3	3	- IO Handling DRIVER 의 Input/Output Part 와 통신을 위한 3 st Parameter - “-“로 선언하였을 경우 Parameter 값은 -1
10	ID4	4	- IO Handling DRIVER 의 Input/Output Part 와 통신을 위한 4 st Parameter - “-“로 선언하였을 경우 Parameter 값은 -1

11	PollType	Poll	Polling
12	LMin	0	화면 display Min 값
13	LMax	10	화면 display Max 값
14	PMin	0	Driver 에서 리턴되는 Min 값
15	PMax	1000	Driver 에서 리턴되는 Max 값
16	Init	10	초기 로딩시에 설정하기 위한 초기값

LookUp Table

- Lookup Table 은 ANALOG IO 에 한정되어 사용되는 것으로 ANALOG IO 의 Read 한 값이 Linear Scale 이 아닌 Lookup table 을 기준으로 변환 처리하기 위해 정의하는 참조 Table 이다.

ex) **#DECLARE LOOKUP**

KVC450 // Lookup table name

0 **0.5**

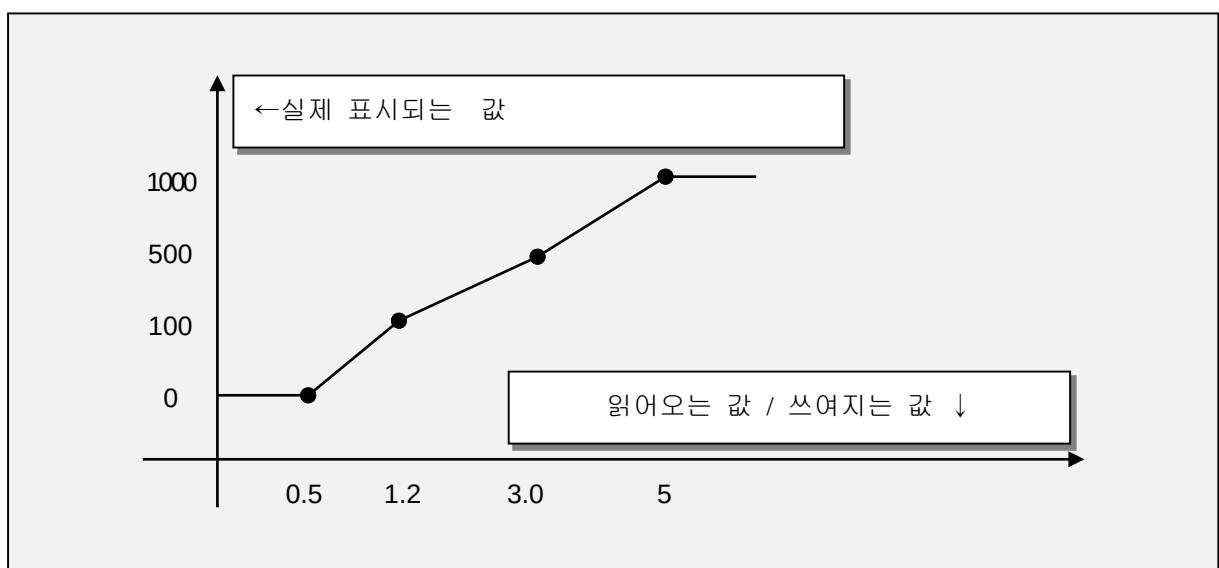
100 **1.2**

500 **3.0**

1000 **5.0**

// Display Real

- 위의 예에서 “#DECLARE LOOKUP” 이후의 내용은 모두 Lookup Table 에 관련된 내용이다.
“KVC450 -----이후” 이 부분이 실제 Lookup Table 을 정의한 것이다.
- 실제 읽어온 값이 Lookup Table 의 사이에 있는 경우는 그 사이의 Linear Scale 로 계산되어 표시된다. 위의 예를 Graph 형태로 보면 아래와 같다.



■ IO Driver 정의 예시

- io.list (sample)

```
// IO Handling Driver Definition
#DECLARE          DRIVER
MKS651      LOAD  DLL   IO\Driver\MKS651D\MKS651.dll    1  9600  8  1  0  0  0  0  0  500
TEMPCTRL    LOAD  DLL   IO\Driver\TempCtrl\TempCtrl.dll  2  9600  8  1  0  0  0  0  0  500
WAGO        LOAD  DLL   IO\Driver\Wago\Ethernet_Wago.dll

//
// Include Files
#include      IO\Driver\MKS651D\MKS651.cfg
#include      IO\Driver\RS232_TempCtrl\RS232_TempCtrl.cfg
#include      IO\Driver\Ethernet_Wago\Ethernet_Wago.cfg
```

- MKS651.cfg (sample)

```
// Enumerations
#DECLARE          ENUM
      M651Comm      Offline Online
      M651OffOn     Off      On
      M651LocRem    Local Remote

// Digital IO Definition
#DECLARE          IO#
//-----
// IO Name      Type  Driver      Enum  Unit  Format ID1-2--3--4  PollType  LMin LMax PMin PMax Init
//-----
APC_Remote_Sts  DI  MKS651  M651LocRem  -  d 1  -  -  -  Poll
APC_Valve_Ctl   DO  MKS651  M651Vlv    -  d 5  -  -  -  Poll
```

■ IO Driver Load Option

- Startup File 에서 SIMULATION 라는 Tag 를 이용하면 초기 Loading 시 IO Driver 의 Load Mode 를 지정할 수 있다.
(SIMULATION REAL 또는 SIMULATION TRUE)

- SIMULATION Tag 를 정의하지 않으면 Default 로 REAL 이 선택되어 Loading 된다.

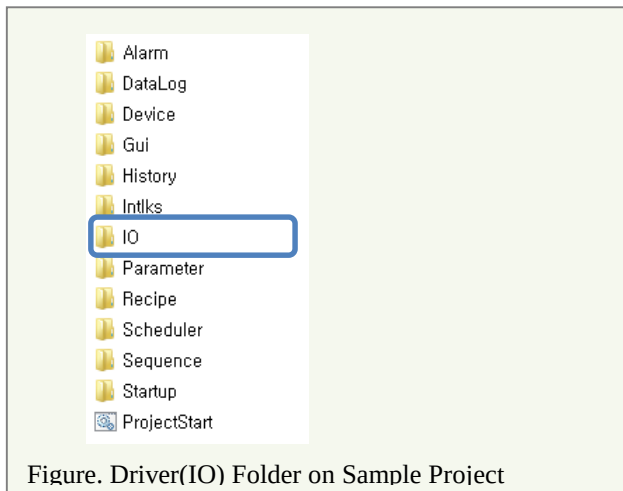
Option	Local Device Driver	Remote Driver	Interlock Engine
REAL (Default)	Loading	Loading	Running
TRUE	Unloading	UnLoading	Stop

Chapter 6

Driver

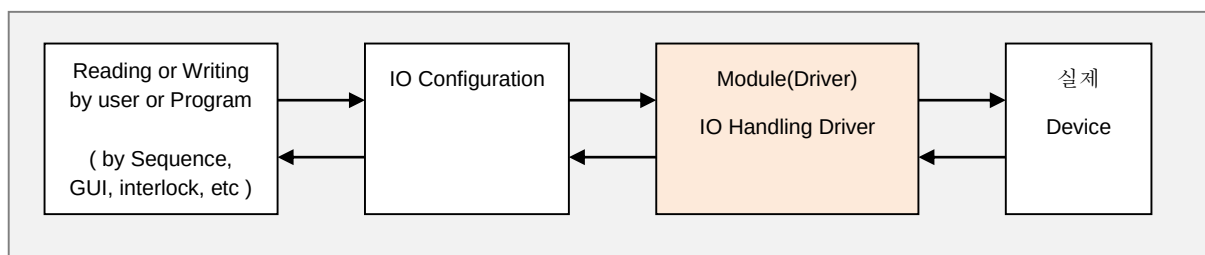
What's In this chapter...

- ◆ Driver 의 정의
- ◆ Driver Interface Prototype
- ◆ Driver Interface 관련 File



Driver 의 정의

- 정의되는 IO 들은 외부 Device 와 연결 되어질 필요가 있다. 여기서는 사용자가 정의한 IO 가 실제로 외부 Module 과 어떻게 Interface 되는지를 다룬다.
- IO Handling Driver 를 위해 Driver Interface Tool kit 을 제공한다. Tool Kit 은 Windows 상에서 DLL(Dynamic Linked Library)형태의 Module(Driver)를 만들어 주는 kit 이다.
- IO 를 제어하는 Library나 API 등을 이용해 Driver Interface Tool kit 에 실제 IO 를 제어하는 Library나 API 등을 연결하여 Digital/Analog/String 으로 정의한 IO 가 실제 IO 를 제어할 수 있도록 해주는 작업을 의미한다.
- Diagram



Note

System IO 도 내부 선언되는 Driver 의 통로를 통해 동작된다.

User 가 정의한 IO 도 User 가 추가 작성한 Driver 의 통로를 통해 동작된다.

이와 같이, Driver 는 IO 를 동작시키는 매개체가 된다.

Driver Interface Prototype

- 사용자에게 의하여 정의된 Digital/Analog/String IO 는 Interface ID 를 정의하고 해당 ID 가 인식번호로 역할을 하여 DRIVER 와 Module(Driver)이 연결하도록 되어 있다. Driver Interface Tool kit 에는 크게 8 개의 Interface API 가 제공된다. 각각의 API 는 아래와 같이 분류된다.

• Interface API

Type	Prototype
Initialize	BOOL OnInitDevice(int ID1 , int ID2 , int ID3 , int ID4 , int ID5 , int ID6 , int ID7 , int ID8 , int ID9 , int ID10, TCHAR *szParm , void **ppDrvData)
DeInitialize	BOOL OnKillDevice(void* pDrvData , int ID1 , int ID2 , int ID3 , int ID4 , int ID5 , int ID6 , int ID7 , int ID8 , int ID9 , int ID10, TCHAR *szParm)
Digital Input	int OnReadDigital(void* pDrvData , void* , int ID1, int ID2, int ID3, int ID4, int* Result)
Digital Output	void OnWriteDigital(void* pDrvData , void* , int ID1, int ID2, int ID3, int ID4, int SetValue , int* Result)
Analog Input	double OnReadAnalog(void* pDrvData , void* , int ID1, int ID2, int ID3, int ID4, int* Result)
Analog Output	void OnWriteAnalog(void* pDrvData , void* , int ID1, int ID2, int ID3, int ID4, double SetValue , int* Result)
String Input	void OnReadString(void* pDrvData , void* , int ID1, int ID2, int ID3, int ID4, TCHAR* pszRtn , int* Result)
String Output	void OnWriteString(void* pDrvData , void* , int ID1, int ID2, int ID3, int ID4, TCHAR* SetValue , int* Result)

• Initialize

Type	내 용
API	BOOL OnInitDevice(int ID1 , int ID2 , int ID3 , int ID4 , int ID5 , int ID6 , int ID7 , int ID8 , int ID9 , int ID10, TCHAR *szParm , void **ppDrvData)
역할	- Driver 가 Load 로 되어 있을 때 해당하는 DLL File 을 찾아 Memory 에 Loading 한 후 OnInitDevice 를 수행한다. - DLL 이 Memory 에 Loading 될 때 오직 한번만 수행하는 API 이며, 주로 Driver Initialize 에 사용된다.
Parameters	ppDrvData = Chapter 5-2 의 Tag 3 (String).(Name is a string, as a tag 3 of chapter 5-2) szParm = Chapter 5-2 의 Tag 5 (String).(File is a string , as a tag 5 of chapter 5-2) ID1 = Chapter 5-2 의 Tag 6 (Integer).(ID1 is an integer, as a tag 6 of chapter 5-2) ID2 = Chapter 5-2 의 Tag 7 (Integer).(ID2 is an integer, as a tag 7 of chapter 5-2) ID3 = Chapter 5-2 의 Tag 8 (Integer).(ID3 is an integer, as a tag 8 of chapter 5-2) ID4 = Chapter 5-2 의 Tag 9 (Integer).(ID4 is an integer, as a tag 9 of chapter 5-2) ID5 = Chapter 5-2 의 Tag 10 (Integer).(ID5 is an integer, as a tag 10 of chapter 5-2) ID6 = Chapter 5-2 의 Tag 11 (Integer).(ID6 is an integer, as a tag 11 of chapter 5-2) ID7 = Chapter 5-2 의 Tag 12 (Integer).(ID7 is an integer, as a tag 12 of chapter 5-2) ID8 = Chapter 5-2 의 Tag 13 (Integer).(ID8 is an integer, as a tag 13 of chapter 5-2) ID9 = Chapter 5-2 의 Tag 14 (Integer).(ID9 is an integer, as a tag 14 of chapter 5-2) ID10 = Chapter 5-2 의 Tag 15 (Integer).(ID10 is an integer, as a tag 15 of chapter 5-2)
Return	성공적으로 수행 했을 때는 TRUE, 그렇지 않을 때는 FALSE 를 Return 하며, FALSE 로 return 되었을 때는 Driver Initialize 실패로 더 이상 Main Program 이 진행되지 않고 종료된다

• DeInitialize

Type	내 용
API	BOOL OnKillDevice(void* pDrvData , int ID1 , int ID2 , int ID3 , int ID4 , int ID5 , int ID6 , int ID7 , int ID8 , int ID9 , int ID10, TCHAR *szParm)
역할	- 정의한 Module 이 UnLoad 로 될때 OnKillDevice 를 수행하고 Memory 로부터 Unloading 하게 된다. - DLL 이 Memory 로 부터 Unloading 하기 전 오직 한번만 수행하는 API 이며, 주로 Driver DeInitialize 에 사용된다.
Parameters	Initialize Part 와 동일
Return	성공적으로 수행 했을 때는 TRUE, 그렇지 않을 때는 FALSE 를 Return 한다.(

• Digital Input Part

Type	내 용
API	int OnReadDigital(void* pDrvData , void* , int ID1, int ID2, int ID3, int ID4, int* Result)
역할	정의한 Digital IO 의 Input(DI)쪽에 관련된 API 로써 Digital IO 의 Input Data 를 Program 이나 화면에서 읽으려고 할 때마다 이 API 가 불러지게 된다.
Parameters	ID1 = Chapter 5-4 의 Tag 7 (Integer) ID2 = Chapter 5-4 의 Tag 8 (Integer) ID3 = Chapter 5-4 의 Tag 9 (Integer) ID4 = Chapter 5-4 의 Tag 10 (Integer)
Result	Parameter "Int *Result" 는 IO Reading 의 성공여부를 나타내는 Parameter 로, 성공적으로 Reading 했을 때는 TRUE, 그렇지 않을 때는 FALSE 를 Setting 하면 된다.
Return	요구에 따른 Digital Input 의 상태 값을 Return 한다. Parameter "Int *Result" 가 FALSE 일 때는 의미가 없으며, Digital Input 의 내용이 Update 되지 않고 이전 상태를 유지하게 된다.

• Digital Output Part

Type	내 용
API	void OnWriteDigital(void* pDrvData , void* , int ID1, int ID2, int ID3, int ID4, int SetValue , int* Result)
역할	정의한 Digital IO 의 Output(DO) 쪽에 관련된 API 로써 Digital IO 의 Output 에 Data 를 쓰려고 할 때마다 이 API 가 불러지게 된다
Parameters	ID1 = Chapter 5-4 의 Tag 7 (Integer) ID2 = Chapter 5-4 의 Tag 8 (Integer) ID3 = Chapter 5-4 의 Tag 9 (Integer) ID4 = Chapter 5-4 의 Tag 10 (Integer) Data = 쓰려고 하는 Data (Integer)
Result	Parameter "Int *Result" 는 IO Writing 의 성공여부를 나타내는 Parameter 로, 성공적으로 Reading 했을 때는 TRUE, 그렇지 않을 때는 FALSE 를 Setting 하면 된다.
Return	없음

• Analog Input Part

Type	내 용
API	double OnReadAnalog(void* pDrvData , void* , int ID1, int ID2, int ID3, int ID4, int* Result)
역할	정의한 Analog IO 의 Input (AI)쪽에 관련된 API 로써 Analog IO 의 Input Data 를 Program 이나 화면에서 읽으려고 할 때마다 이 API 가 불러지게 된다.
Parameters	ID1 = Chapter 5-4 의 Tag 7 (Integer) ID2 = Chapter 5-4 의 Tag 8 (Integer) ID3 = Chapter 5-4 의 Tag 9 (Integer) ID4 = Chapter 5-4 의 Tag 10 (Integer)
Result	Parameter "Int *Result" 는 IO Reading 의 성공여부를 나타내는 Parameter 로, 성공적으로 Reading 했을 때는 TRUE, 그렇지 않을 때는 FALSE 를 Setting 하면 된다.
Return	요구에 따른 Analog Input 의 상태 값을 Return 한다. Parameter "Int *Result" 가 FALSE 일 때는 의미가 없으며, Analog Input 의 내용이 Update 되지 않고 이전 상태를 유지하게 된다

• Analog Output Part

Type	내 용
API	void OnWriteAnalog(void* pDrvData , void* , int ID1, int ID2, int ID3, int ID4, double SetValue , int* Result)
역할	정의한 Analog IO 의 Output (AO)쪽에 관련된 API 로써 Analog IO 의 Output 에 Data 를 쓰려고 할 때마다 이 API 가 불러지게 된다.
Parameters	ID1 = Chapter 5-4 의 Tag 7 (Integer) ID2 = Chapter 5-4 의 Tag 8 (Integer) ID3 = Chapter 5-4 의 Tag 9 (Integer) ID4 = Chapter 5-4 의 Tag 10 (Integer) Data = 쓰려고 하는 Data (Double).
Result	Parameter "Int *Result" 는 IO Writing 의 성공여부를 나타내는 Parameter 로, 성공적으로 Reading 했을 때는 TRUE, 그렇지 않을 때는 FALSE 를 Setting 하면 된다.
Return	없음

• String Input Part

Type	내 용
API	void OnReadString(void* pDrvData , void* , int ID1, int ID2, int ID3, int ID4, TCHAR* pszRtn , int* Result)
역할	정의한 String IO 의 Input (SI)쪽에 관련된 API 로써 String IO 의 Input Data 를 Program 이나 화면에서 읽으려고 할 때마다 이 API 가 불러지게 된다.
Parameters	ID1 = Chapter 5-4 의 Tag 7 (Integer) ID2 = Chapter 5-4 의 Tag 8 (Integer) ID3 = Chapter 5-4 의 Tag 9 (Integer) ID4 = Chapter 5-4 의 Tag 10 (Integer)
Result	Parameter "Int *Result" 는 IO Reading 의 성공여부를 나타내는 Parameter 로, 성공적으로 Reading 했을 때는 TRUE, 그렇지 않을 때는 FALSE 를 Setting 하면 된다.
Data	요구에 따른 String Input 의 상태 값을 Parameter "Int *Data"에 쓰면 된다. Parameter "Int *Result" 가 FALSE 일 때는 의미가 없으며, Digital Input 의 내용이 Update 되지 않고 이전 상태를 유지하게 된다

• String Output Part

Type	내 용
API	void OnWriteString(void* pDrvData , void* , int ID1, int ID2, int ID3, int ID4, TCHAR* SetValue , int* Result)
역할	정의한 String IO 의 Output (SO)쪽에 관련된 API 로써 String IO 의 Output 에 Data 를 쓰려고 할 때마다 이 API 가 불러지게 된다.
Parameters	ID1 = Chapter 5-4 의 Tag 7 (Integer) ID2 = Chapter 5-4 의 Tag 8 (Integer) ID3 = Chapter 5-4 의 Tag 9 (Integer) ID4 = Chapter 5-4 의 Tag 10 (Integer) Data = 쓰려고 하는 Data (String).
Result	Parameter "Int *Result" 는 IO Writing 의 성공여부를 나타내는 Parameter 로, 성공적으로 Reading 했을 때는 TRUE, 그렇지 않을 때는 FALSE 를 Setting 하면 된다.
Data	없음

■ Driver Interface 관련 File

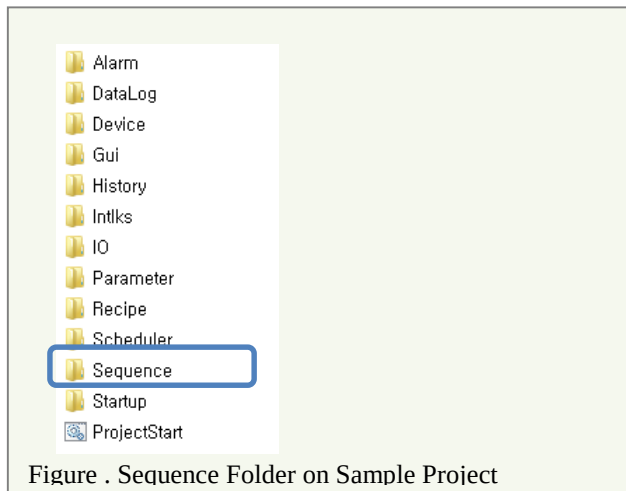
- Library file : TDrvIF.lib, TSeqIF.lib
TComSerial.lib (RS232 Protocol)
TP2PSock.lib (WinSock Protocol)
- Header file : TDrvInterface.h, TSeqIF.h
TComSerial.h (RS232 Protocol)
TP2PSocket.h (WinSock Protocol)

Chapter 7

Sequence

Whats' In this chapter...

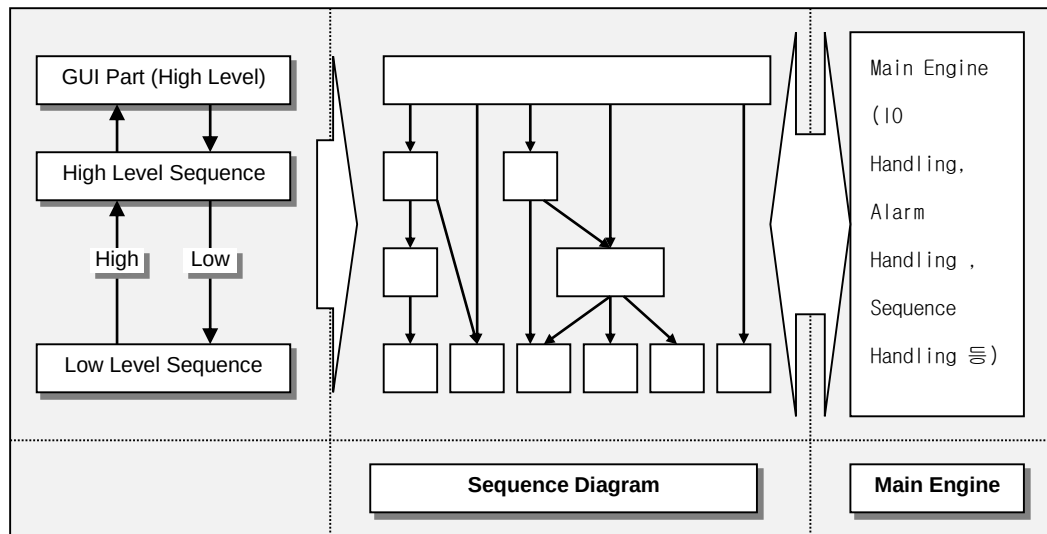
- ◆ **Sequence** 의 정의
- ◆ **Sequence** 등록 방법
- ◆ **Sequence Coding**



■ Sequence 의 정의

- Sequence 은 IO Handling 와 Alarm Management 및 사용자가 제작한 Programming Code 를 나타낸다.
- Sequence 은 C++ Programming Code 에 기반을 하고 있지만 사용자들의 다양한 방식의 Programming 형태를 사용하기 보다는 정해진 틀을 제공하고 그 틀 안에서 사용자가 원하는 형태를 적용하도록 제안하고 있다.
- Sequence 은 Main Program 과 연결되어지는 단일 실행 Code 로 사용자는 별개로 Sequence 들을 유지,보수,관리 할 수 있다.
- Sequence 들 사이에는 서로 정보를 전달하며 수행 시킬 수 있도록 되어 있다. Sequence 들은 다른 Sequence 로부터 정보를 받아 수행되어지는 Server Part 와 다른 Sequence 에 정보를 전달하여 그 Sequence 을 수행시키며 상태를 알아 낸다든지 Main Engine 으로부터 IO 의 상태들을 읽어오거나 쓴다든지 하는 Client Part 를 가지고 있다.

- Sequence 은 독립적인 실행 File 형태로 만들어 지며 Main Engine 과 Process 통신으로 동작되어 진다. 또한 Sequence 마다 Level 이 있으며 S/W Start 시 높은 Level 의 Sequence 가 먼저 Loading 되어 진다.



- Sequence 은 일종의 실행 File(exe) 형태로 되어 실행이 가능하다. 하지만 단독으로 동작하지는 않고 Main Engine 에 연결되어서만 동작이 가능하다.
- Sequence 은 제작 목적에 따라 GUI 용 Sequence 와 Control(장비제어)용 Sequence, 그리고 기타 Utility Sequence 등으로 분류할 수 있다. 주로 사용자들은 Control(장비제어)용 Sequence 들을 제작하게 될 것이다.
- Sequence 은 크게 아래와 같이 구성되어 있다. Program Coding 과 상관없이 이미 Coding 되어 있는 Sequence 을 Main Engine 에 등록하는 Sequence Registry Part 와 Sequence 을 제작하는데 관련된 Programming 에 대한 내용을 다루는 Execution Coding Part 로 나누어 진다.

■ Sequence 등록 방법

- Sequence 은 고유의 이름을 부여하여 사용하게 되는 데 이때 고유의 이름마다 어떤 실행 File 이 동작하여야 하는지를 정의해 주어야 한다.
- 정의를 위해서는 먼저 Startup File 의 SEQUENCE Tag 에 어떤 File 에 정의해 놓을지를 먼저 설정한다.

SEQUENCE

Sequence/Sequence.dat

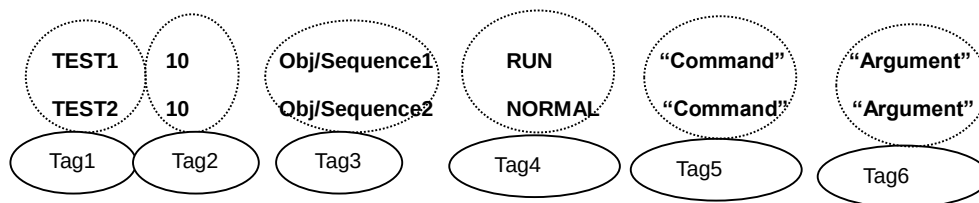
■ Sequence Name Registry

- Sequence Name Registry Part 는 각 Sequence 에 고유의 이름을 부여하여 Engine 이 이 Sequence 들을 서로 운용하기 위해 사용되며 Sequence 마다 수행되어질 실행 File 들을 정의해 놓는 곳이다.

- SEQUENCE FileName

에서 Filename 의 File 안에 아래와 같은 형식으로 등록 할 수 있다.

ex) #DECLARE SEQUENCE



- Tag1 : Sequence Name
Sequence 에 사용될 고유의 이름이다(중복되면 안됨).
- Tag2 : Sequence Priority
Sequence 에 부여된 Priority 로, 큰 의미는 없으며 단지 높은 Priority 의 Sequence 가 먼저 Loading 된다..
- Tag3 : Sequence Execution File
Sequence 에 대해 실제로 동작할 실행 File 을 나타낸다.
(Tag3 is an execution file which shall be actually practiced by the sequence)
기본적으로 수행된 Sequence 은 공통의 Console 을 사용하게 된다.

- Tag4 : Sequence Option

[NORMAL]

등록된 각 Sequence 들은 Main 프로그램을 수행하게 되면 실제 동작은 이루어 지지 않고 Memory 에 Loading 되어 Initial Part 만 수행되고 Wait 상태에 들어서서 Run Message 가 올 때까지 기다리게 된다(TEST2).

[RUN]

Main 프로그램을 수행하게 되면서 각 Sequence 을 Memory 에 Loading 한 후 Wait 상태로 두지 않고 바로 동작 시키고자 할 때가 있는데 이때는 Tag 에 **RUN Option** 을 사용하면 된다(TEST1)

- Tag5 : Command Option

Tag4 가 RUN 으로 정의되어진 경우에 유효하며, 각 Sequence 가 Memory 에 Loading 완료된 후 Tag4 의 RUN 에 의해 동작 될 때 전달되는 Command 이다.

- Tag6 : Argument Option

각 Sequence 가 Memory 에 Loading 될 때 함께 전달되는 Data 를 ARGUMENT 라고 하고, Sequence 의 void OnLoadComplete(const TCHAR* szArgument) 부분으로 전달된다.

- Sequence Name 은 유일해야 하지만 그것에 관련된 실행 File 들은 같은 것으로 정의될 수 있다.

■ Sequence Coding

- Sequence 에 대해 실제 연결되어 있는 실행 File 을 제작할 때의 내용을 나타낸다.
- Sequence 은 C++ Language 로 작성되어 진다.
- Sequence 은 크게 Connection Part, Basic Operation Part, Interface API Part 로 구성되어 있으며, 기본적인 Scenario 로 이루어져 있고 그 위에서 사용자가 프로그래밍을 할 수 있다.

■ Connection Part

- Sequence Name Registry Part 에서 등록하였고 library 로 실행 File 을 제작하였을 경우는 Sequence 와 Main Engine 과의 Connection 에 대한 일은 자동으로 수행하게 되어 있다. 따라서 사용자가 프로그래밍으로 특별히 하여야 할 일은 없다.

■ Basic Operation Part

- Sequence 은 Main Engine 과의 Connection 부분을 제외하면 아래와 같은 정해진 틀 안에서 동작하게 된다.
- Sequence 이 Loading 되면 OnLoadComplete() 가 수행된다. 이 Procedure 는 IO 를 정의한 정보와 무관하게 동작한다. 결과도 없는 Procedure 이다. 이 Procedure 의 목적은 주로 Program 변수 초기화 라든지 IO 에 관한 정보를 읽기 전에 먼저 IO Table 의 내용을 조작 한다든지 기타 사용자의 필요에 따라 사용되어지기 위한 Procedure 이다.
- OnLoadComplete() 가 수행된 후 IO Table 에 사용자가 정해놓은 IO 의 내용을 Main Engine 으로부터 확인하고 사용가능 여부를 통보 받는다. 이때 잘못된 IO Table 이나 없는 내용이 있을 경우 이 Sequence 은 Loading 에 실패하게 된다.
- IO 정보의 확인 후 OnInitComplete() 가 수행되게 되며 이 Procedure 의 결과가 FALSE 일 때는 이 Sequence 은 Loading 에 실패하게 된다. OnInitComplete()에서는 IO 의 정보를 통한 IO 의 상태를 읽거나 쓸 수 있게 된다. 이 Procedure 의 목적은 OnLoadComplete()보다 더 큰 범위로 초기화가 필요할 경우 사용되어지기 위해 존재한다.
- 여기까지가 Sequence 의 Loading 에 관련된 부분으로 이 부분까지 완료되면 Running 에 관련된 부분으로 넘어가게 된다.
- Running Part 에서는 Server 역할을 하여 다른 Sequence 로부터 Command 를 기다리게 된다. Command 는 크게 Run Command 와 Event Command 로 나뉘어 지는데 Run Command 가 도착하였을 때는 SeqMain() Procedure 가 수행되며 Event Command 가 도착하였을 때는 OnEventRecv() Procedure 가 수행되게 된다. 둘 사이의 차이는 Run Command 의 경우는 SeqMain() Procedure 수행 후 결과와 결과 Data 가 있다는 것이고 Event Command 의 경우는 OnEventRecv() Procedure 가 수행되는 것이 전부이다.

OnLoadComplete(const TCHAR* szArgument)

- 사용자 정의 Code Part.
- 결과 없음.
- FEST Library Manual 참조

IO Information Check

- BEGIN_OBJECT_ENUMERATION 와 END_OBJECT_ENUMERATION 사이에 정의한 IO 정보 Check.
- 결과 FALSE -> Function Loading Fail.

OnInitComplete(const TCHAR* szArgument)

- 사용자 정의 Code Part.
- 결과 FALSE -> Function Loading Fail.

SeqMain(const char* szCommand, const void* pParam, int nNumOfByte)

- 사용자 정의 Code Part.
- 도착한 Run Command 가 szCommand 로 온다.

OnEventRecv(const char* szCommand, const void* pParam, int NumOfByte)

- 사용자 정의 Code Part.
- 도착한 Event Command 가 szCommand 로 온다.

■ Interface API Part

- Sequence 에서 Main Engine 의 상태를 읽거나 쓸 때 사용되는 API Part 로 IO, Sequence, Alarm 및 Scenario 등에 관련된 여러 형태의 API 들을 제공한다.

■ Execution Code Sample

- main.cpp

```
#include <stdio.h>
#include <windows.h>
#include <process.h>

//-----

#include <TMain.h>
#include <TUtilStrr.h>
#include <TTimer.h>

//-----

BEGIN_OBJECT_ENUMERATION
    CDIO SIMULATION_CONTROL("System.SimulMode");
    DIO(PRM_DOOR_CONTROL      )
    DIO(Door_Status           )
    DIO(Door_Ctl              )
END_OBJECT_ENUMERATION

//-----
//-----

SEQ_STATUS SeqMain(const char* szCommand, const void* pParam, int nNumOfBytesRecved) {
    int CommStatus;
    SEQ_STATUS Result;
    if( StrCmpNoCase( (char*)szCommand, "OPEN"    ) )    Result = Control( OPEN    );
    ... ..
}

//-----
//-----

void OnLoadComplete( const TCHAR* szArgument ) {
}

BOOL OnInitComplete( const TCHAR* szArgument ) {
    return TRUE;
}

void OnUnLoad( int nCode ) {
}

void OnEventRecv( const char* szCommand, const void* pParam , int nNumOfBytesRecved ) {
}

//-----
```

Chapter 8

Interlock

What's In this chapter...

- ♦ Interlock 개요
- ♦ Interlock 정의 방법

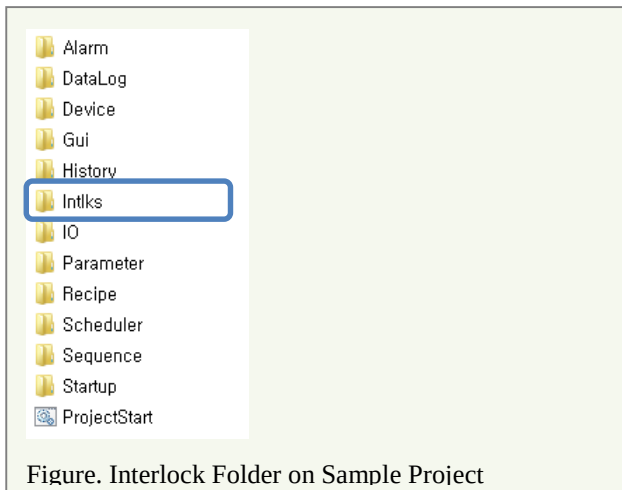


Figure. Interlock Folder on Sample Project

Interlock 개요

- 특정 IO에 대하여 Writing을 하려고 할 때 관계가 있는 다른 IO의 상황을 조건 검사 후 Writing 가능한 지 확인 후 Writing을 할 수 있도록 하는 것이 Setpoint Interlock이다.
즉, Safety check 기능이라고 칭할 수 있다.
- Setpoint Interlock을 정의할 때는 Text File에 Interpreter 방식의 자체 표현방식을 사용한다.
(// 으로 시작하는 줄은 Comment)
- Script 표현방식은 다음과 같다.

```
try ( 변환 시키려는 IO ) {  
    if ( 검사 조건 및 다른 IO의 상황 ) alarm("AlarmName");  
}
```

* comment

변환 시키려는 IO에 대해 사용자 또는 Program에서 제어 요청시

- 검사 조건 및 다른 IO의 상황일 때 변환 시키려는 IO에 대한 요청을 거부하고 alarm("Alarm 이름")이 발생하게 되며,
- 현재 상황이 검사 조건 및 다른 IO의 상황이 아닐 때는, 변환 시키려는 IO에 대한 요청을 수행한다.

*** if 조건은 반드시 1 줄 안에 표시해야만 한다.

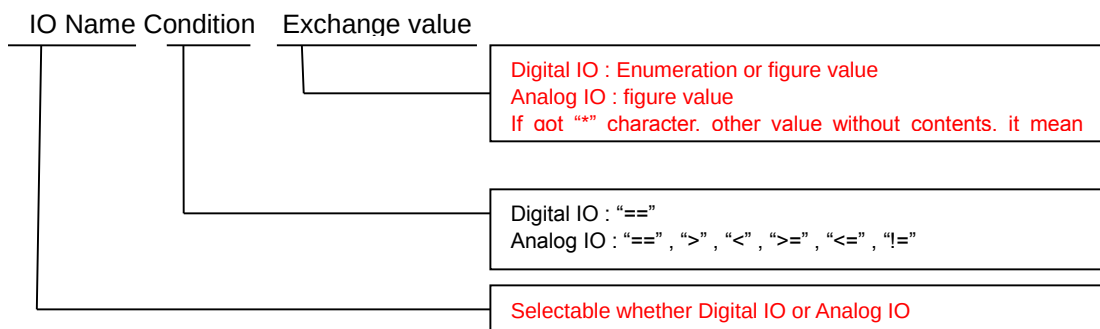
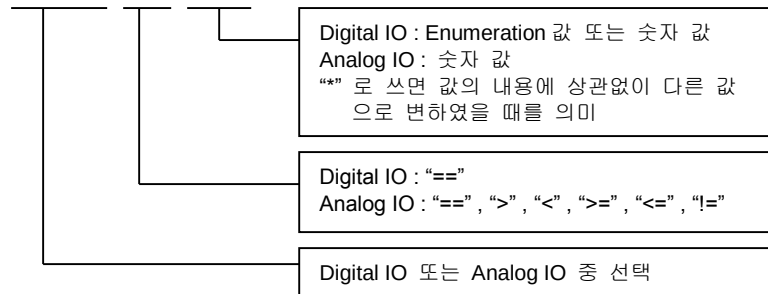
Note

User가 정의한 Setpoint Interlock 정의는 Project Start 시에 Engine에서 읽어들이 Memory에 적재하고, 정의된 IO가 동작하는 매순간마다 Interlock Engine에서 check 판단을 한다.

■ Interlock 정의방법

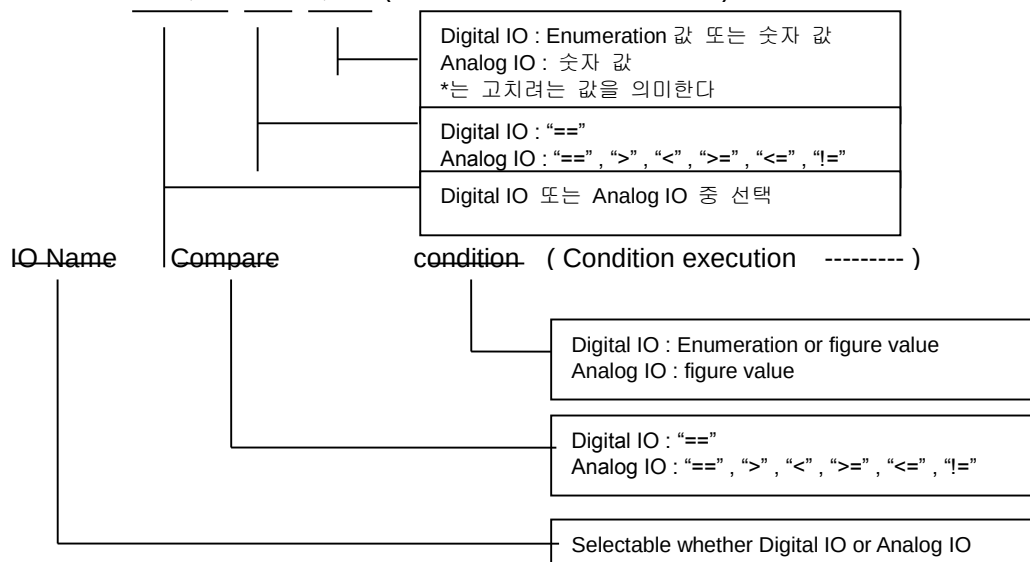
- **변환 시키려는 IO(enable to change IO)**

Syntax : **IO 이름** **조건** **변환값**



- **검사 조건 및 다른 IO 의 상황(check condition or other IO status)**

Syntax : **IO 이름** **조건** **비교** (조건연산 조건문 -----)



- **alarm(“AlarmName”);**

- Alarm 이 발생하게 될 상황이 되었을 때 **AlarmName** 에 해당하는 Alarm 을 발생한다.
- Alarm 사용을 하지 않으려면 “alarm(“**Alarm 이름**”);” 부분을 선언하지 않으면 된다.

• Setpoint Interlock 의 예).(For example of Setpoint Interlock)

No	예 제
1	try(VAT_Valve == *) { if(Cover == Up) alarm("ALMVatCannotAction"); }
2	try(PM_Fast_Pump == Open) { if(PM_Pressure > 700 (PM_AtmSensor == ATM)) alarm("ALMFastPumpCannotOpen"); }
3	try(TM_Fast_Pump == Open) { if(TM_Pressure > 700) alarm("ALMFastPumpCannotOpen"); }
4	try(PM_Chuck == Down) { if(PM_Chuck_Sns != Downed Motor_Position > 1000) alarm("ALMChuckCannotDown"); }
5	try(SlotCtl == Close) { if(Motor_Position > 1000) alarm("ALMSlotCannotClose"); }
6	try(TMPM_SlotCtl == Open) { if((TM_Pressure > 700 && PM_Pressure < 500) (TM_Pressure < 200 && PM_Pressure > 999)) alarm("ALMTMPMSlotCannotClose"); }

Note

User 가 정의한 Setpoint Interlock 정의는 Project Start 시에 Engine 에서 읽어들이 Memory 에 적재하고, 정의된 IO 가 동작하는 매순간마다 Interlock Engine 에서 check 판단을 한다.

Chapter 9

Alarm

What's In this chapter...

- ♦ Alarm Manage
- ♦ Alarm Management File
- ♦ Alarm History File

Alarm Manage

- 어떤 상황에 대하여 사용자에게 통보해 주고 그 상황에 대한 대처가 필요할 때 이런 일들을 담당해 주는 일련의 일들을 Alarm Management 라고 한다.

- Alarm Management 에서 관리되는 5 가지 항목은 다음과 같다.

Item	설 명	관계되는 항목
Alarm Post	Alarm 을 생성하고 User 에게 알린다.	Interlock, Sequence
Alarm Clear	Alarm 을 지운다.	Sequence, Alarm Manager
Alarm Result	Alarm 의 상태를 알아낸다	Sequence
Alarm Recovery	Alarm 발생시 사용자의 처리에 대응한다.	
Alarm Handling Engine	Alarm 처리의 전반적인 Manage	

- Alarm 은 Alarm Name 단위 기준으로 관리되며, Alarm Post , Alarm Clear, Alarm Status 를 User 의 판단에 의하여 후속 처리를 할 수 있도록 설계되어 있다.

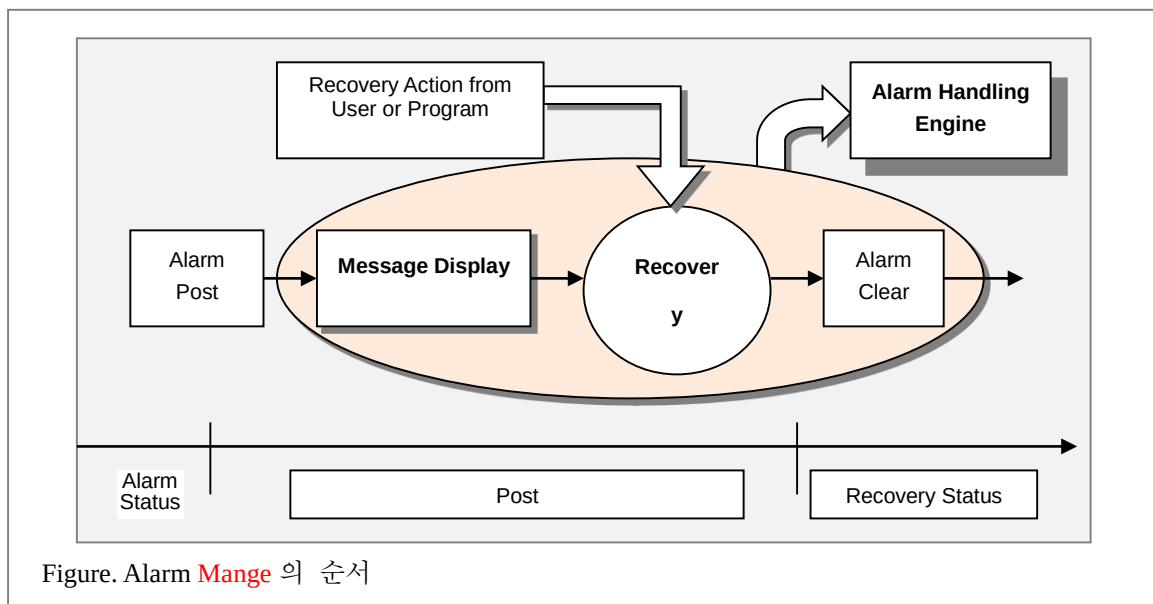


Figure. Alarm Manage 의 순서

- Alarm Post 이후 Alarm 을 관리하는 것은 Alarm Handling Engine 이다. Alarm Handling Engine 은 Alarm Post 시 해당 Alarm 의 Message 와 Recovery Action 등을 GUI 화면에 보여주고 User 가 Recovery 를 했을 때 사전에 정의된 Recover Action 을 처리하고 Recovery 한 정보를 관리하는 역할을 한다.

Alarm Management File

- 사전에 User 가 Alarm 목록을 정의하고 이것을 User SW 에서 참조하여 사용하게 된다.
이 때 실제 해당 Alarm 이 발생하는 상황이 되어 Alarm Post 시
 - 어떤 Message 를 GUI 화면에 보여주는지,
 - 어떤 Recover Action 을 User 에게 제시하고 Recovery Action 을 통보 받을 것인지 정의 해두는 File 을 의미한다.
- Startup File(Startup.cfg)의 “ALARM” Tag 에서 정의한 File 에 Text 형식으로 아래와 같이 실제로 Alarm 의 내용을 정의한 File 을 삽입형식으로 정의한다. 실제 Alarm 의 내용은 삽입형식으로 지정해준 File 안에 정의되어 있다.

```
#INCLUDE      Alarm/Pin.alm
#INCLUDE      Alarm/Door.alm
#INCLUDE      Alarm/Temp.alm
```

- Alarm 의 내용을 정의한 File 에 Comment 를 하고자 할 때엔 “//”를 삽입한다.

- 표현방식

```
Alarm(이름) 번호(No) "Alarm Message" {
    Recovery_Result_1    " Recovery_Result_Message 1"
    Recovery_Result_2    " Recovery_Result_Message 2"
    Recovery_Result_3    " Recovery_Result_Message 3"
    Recovery_Result_4    " Recovery_Result_Message 4"
    Recovery_Result_5    " Recovery_Result_Message 5"
    DESCRIPTION          "File Name"
}
```

• 각 Tag 의 의미

Item	설 명
Alarm(이름)	Alarm Post / Clear / Status 를 이용할 때 사용되어진다. 중복되지 않은 이름 과 번호로 1-9999 사이에서 정의해야 한다.
Alarm 이름(Name)	은 Alarm 내용에 붙여지는 String 형식의 이름으로 중복되어서는 안 된다. 또한 Alarm 번호도 중복 되어서는 안된다.
"Alarm Message"	Alarm 이 Post 되었을 때 나타나는 Message 이다.
Recovery_Result_*	"Recovery_Result_Message *" 부분은 사용자가 Alarm 에 대한 Recovery 를 원할 때 사용 되어지며 1 줄 이상 최대 5 줄 까지 정의할 수 있다. Recovery 를 선택하면 Recovery_Result_Message_* 들이 나열되고 사용자가 그 중에 하나를 선택하였을 때 Alarm 의 상태는 Recovery_Result_* 로 바뀌게 되며 이것을 프로그램(Sequence)에서 사용하게 된다. 정의 할 수 있는 Recovery_Result 의 종류는 7 개로 아래와 같으며 이중 한 개를 정의해 주어야 한다. * CLEAR / RETRY / IGNORE / ERROR / FAIL / ABORT / POSTED
DESCRIPTION	Alarm 에 대한 자세한 내용을 수록한 Text File 이다. 정의하지 않을 경우 default 로 [Alarm 번호].txt 로 사용된다.

• Alarm Management File 의 예시)

```

    AlmPinCtrlTimeout    200    "Pin Contorl Timeout"{
        RETRY  "Retry Action"
        IGNORE "Ignore Action"
        ABORT  "Abort Action"
        DESCRIPTION      "Alarm/AlarmText/alarmPinErr.txt"
    }

```

Note

Recovery Action 은 Alarm 의 후속 처리를 감안하여 적절한 Recovery Action 을 정의하는 것이 좋다. 예를 들어,

Setpoint interlock 은 더 이상 해당 IO 를 속행할 수 없으므로 CLEAR 만으로 정의하거나 Sequence 에서 발행하는 Alarm 은 RETRY / ABORT / IGNORE 등을 적절히 선택하여 정의한다. 특히 IGNORE 는 상황을 무시하고 속행하는 의미이므로 사용시 주의가 필요하며 특별한 이유가 없는 한 IGNORE 는 정의하지 않는 것이 좋다.

Alarm History File

- Alarm 에 관련된 Post , Clear , Recovery 등의 일이 일어날 때 그것에 대한 정보를 저장할 File 이름을 정의한다.

- Alarm History File 의 내용은 Text File 형식에 아래와 같이 저장된다.

- Startup.cfg 에 HISTORY tag 에 Log 가 저장될 위치를 선언한다.

ex) HISTORY History

Alarm-년월.log 라는 파일로 저장되어 진다(Alarm-201001.log)

[01/05-14:45:07.046]:[POSTED]	[600]	"AlmLampOnLifeTimeErr"	"Lamp usage time has been exceed, Please check LAMP Status[1 Lamp]"
[01/05-14:45:07.140]:[POSTED]	[604]	"AlmWagoCommErr"	"Wago DeviceNet IO Module is Offline. Check!!!"
[01/05-23:49:52.218]:[POSTED]	[600]	"AlmLampOnLifeTimeErr"	"Lamp usage time has been exceed, Please check LAMP Status[1 Lamp]"
[01/05-23:49:52.234]:[POSTED]	[604]	"AlmWagoCommErr"	"Wago DeviceNet IO Module is Offline. Check!!!"
[01/08-20:53:21.812]:[POSTED]	[504]	"AlmHeaterPowerSisOff"	"Cannot Heater power on, Because now the heater power is blocked"
[01/08-20:53:29.328]:[CLEAR]	[504]	"AlmHeaterPowerSisOff"	"Cannot Heater power on, Because now the heater power is blocked"

Note

Alarm Manager, Alarm History Viewer 등의 구체적 설명은 Part3.Utility- Chapter32. Alarm Manager 를 참조

Chapter 10

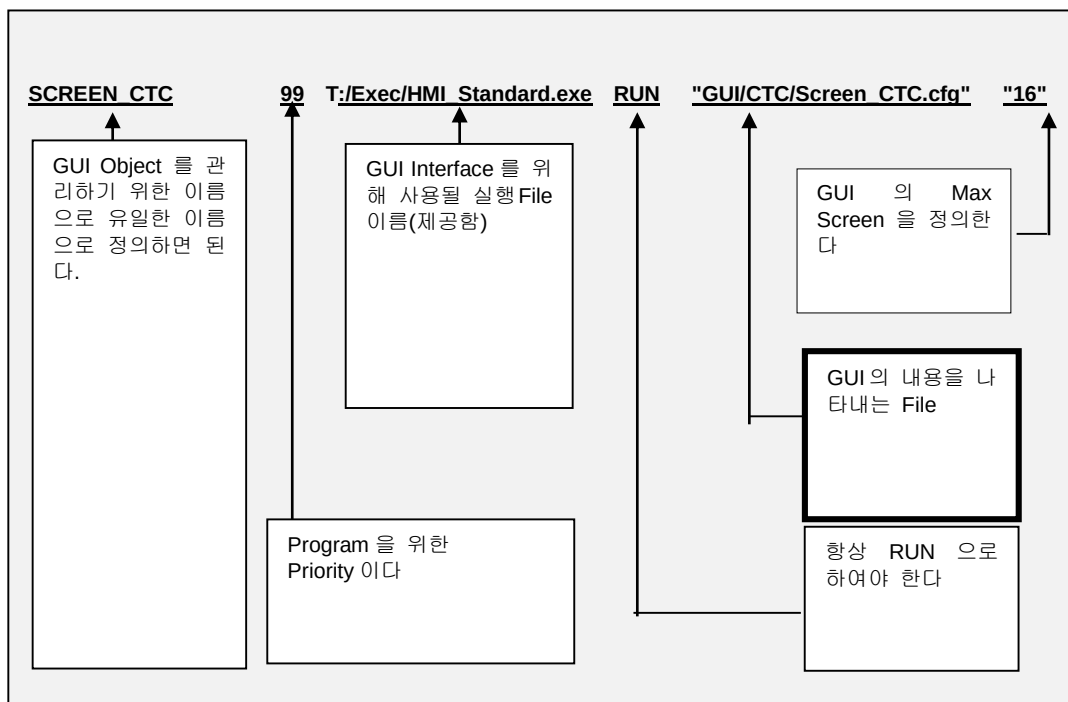
GUI

In this chapter...

- ◆ GUI 의 정의
- ◆ Configuration Part
- ◆ Object Interface Part
- ◆ Paramter Interface Part

GUI 의 정의

- 사용하는 IO 와 Sequence 을 가지고 GUI 를 적용하는 Tool 로서 Text Based 에서의 정의로 GUI 의 구성을 가능케 해준다.
- Sequence 에서 이 Tool 을 사용하기 위한 방법은 아래와 같이 선언하면 된다.



- Note**
- **"GUI/Screen_CTC.cfg"** 의 File(이하 GUI Definition File(GDF))에 GUI 를 표현한 내용이 담겨져 있으며 다음에서 GDF 를 표현하는 방법들에 대해서 다루게 된다.

- GDF 는 크게 아래와 같이 3 개 Part 로 구분되어 진다.
 - Configuration Part
 - Object Interface Part (IO 및 Sequence 을 위한 Object Interface)
 - Parameter Interface Part (Parameter 의 저장 및 적용)

또한 Page 를 구분하여 여러 장의 내용을 표현할 수 있으며 Popup Screen 을 지원하고 Dialog 창을 File 로부터 지원 받는다. 각각의 자세한 내용은 아래에 설명되어 있다

■ Configuration Part

- Configuration Part 는 GUI 의 표현을 위한 환경을 설정하는 부분으로 크게 IMAGE_BACKGROUND, IMAGE_LIBRARY, COLOR_TABLE, FONT_TABLE, GOI_TABLE, POPUP_SCREEN 등으로 구분되어 진다.

■ Image Background

- Image Background 는 현재 보여질 Page 에 나타내어질 Bitmap 형태의 Background 화면 및 Popup 화면에 보여질 Bitmap 형태의 Background 화면의 모음을 나타내는 File 을 의미하며 DLL 형태로 되어 있어야 한다.
- 선언은 GDF File 에서 **#DECLARE IMAGE_BACKGROUND** 의 선언을 사용하여 아래와 같이 나타내며 GUI\PMC\Background\ImageBackground.dll 이 Background 로 사용할 Bitmap 의 모음이다.

ex) **#DECLAREIMAGE_BACKGROUND**
BackImage "GUI\PMC\Background\ImageBackground.dll"

- Image Background 는 Visual C++(6.0 이상) 에서 아래의 DLL Interface File(C File)과 Bitmap Resource File(rc File)을 가지고 Compile 하여 생성되어 지는 DLL File 로 Bitmap Resource 에 정의한 Bitmap 의 이름에 따라 각각의 Page 및 Popup Screen 의 Background 를 다르게 사용할 수 있게 된다.

DLL Interface File

```
#include <windows.h>

BOOL WINAPI DllMain(HINSTANCE hinstDLL, DWORD fdwReason, LPVOID lpvReserved) {
    return( TRUE );
}
```

Bitmap Resource File 을 Text 형태로 읽어 보면

```
////////////////////////////////////
//
// Bitmap
//
BACKGROUND_0111 BITMAP BACKGROUND_System_Main.bmp
BACKGROUND_0112 BITMAP BACKGROUND_System_Transfer.bmp
BACKGROUND_POPUP_0001 BITMAP BACKGROUND_Popup_LampLife.bmp
BACKGROUND_POPUP_0002 BITMAP BACKGROUND_Popup_LotControl.bmp
```

Background 의 Index 로 사용될 이름으로 Background 일 경우는 BACKGROUND_???? 형태로 이름을 정의해야 하고 ???에는 Page 값이 들어간다.

Popup 의 경우는 BACKGROUND_POPUP_???? 형태로 이름을 정의해야 하고 ???에는 Popup Page 를 위한 Pointer 값 이 들어간다

Bitmap 형태의 내용
이 있는 실제 File 을
정의
(define actual file
name which included
real bitmap)

Image Library

- Image Library는 IO 상태를 이용하여 Animation을 사용할 때, 각각의 IO 상태(값)에 따른 Bitmap 형태의 화면을 모아놓은 File 을 의미하며 DLL 형태로 되어 있어야 하며 Visual C++을 통해 간단히 작성할 수 있다.
- 선언은 GDF File 에서 **#DECLARE IMAGE_LIBRARY** 의 선언을 사용하여 아래와 같이 나타내며 GUI\PMC\ImageLibrary\ImageLibrary.dll 이 Image Library 로 사용할 Bitmap 의 모음이다.

ex) **#DECLARE IMAGE_LIBRARY**
ImgLibrary "GUI\PMC\ImageLibrary\ImageLibrary.dll"

- Image Library 는 Visual C++(6.0 이상)에서 아래의 DLL Interface File(C File)과 Bitmap Resource File(rc File)을 가지고 Compile 하여 생성되어 지는 DLL File 로 Bitmap Resource 에 정의한 Bitmap 의 이름에 따라 Image Pointer 와 각각의 상태(값)에 따라 보여질 Bitmap 화면 등을 나타낸다.

DLL Interface File

```
#include <windows.h>

BOOL WINAPI DllMain( HINSTANCE hinstDLL , DWORD fdwReason , LPVOID lpvReserved ) {
    return( TRUE );
}
```

Bitmap Resource File 을 Text 형태로 읽어 보면(The next box is an example of read Bitmap Resource file as text format)

```

////////////////////////////////////
//
// Bitmap
//

IMG_0101_0000000000 BITMAP IMAGE/GATE_NONE.bmp
IMG_0101_0000000001 BTMAP IMAGE/GATE_OPEN.bmp
IMG_0101_0000000002 BITMAP IMAGE/GATE_CLOSE.bmp
IMG_0101_0000000003 BITMAP IMAGE/GATE_ERROR.bmp

```

IMG_xxxx_xxxxxxxxxx 형태로 이름을 정의해야 하고 앞의 xxxx(4 자)는 Image 의 Pointer 를 의미하며 뒤의 xxxxxxxxxxx(10 자)는 상태 값을 의미한다. 어떤 IO 가 101 번의 Image Pointer 를 사용하고자 할 때, 그 IO 의 상태가 0 일 때는 IMAGE/GATE_NONE.bmp 의 내용, 1 일 때는 IMAGE/GATE_OPEN.bmp 의 내용, 2 일 때는 IMAGE/GATE_CLOSE.bmp 의 내용, 3 일 때는 IMAGE/GATE_ERROR.bmp 의 내용이 화면에 나타나게 된다.

(It must define like as B_xxxx_xxxxxxxxxx format, a pointer value of image is able to set into the front four question marks, an IO value must be written at rear ten question marks to call image screen, as above box, if an IO used 0101 number for image pointer to call an image screen and an IO status was zero, it call Value_0.bmap file from IMAGE directory. and was one, call Value_1.bmap file. And was three, call Value_3.bmap file).

Bitmap 형태의 내용이 있는 실제 File 을 정의
(define actual file name which included real bitmap)

- Resource 표현에는 의미상으로 다른 1 가지와 추가적인 3 가지의 총 4 가지 예외적인 표현 방법이 있다.

의미상 다른 경우(For difference in meaning) – Multiple IO(Object Interface Part 참

조)로 Image Pointer 의 내용을 표현하려는 경우 IMG_xxxx_xxxxxxxxxx 에서 뒤의 xxxxxxxxxxxx 이 하나의 IO 상태가 아닌 각각의 IO 상태를 의미하게 된다. 다시 말해서 xxxxxxxxxxxx 이 10 자이므로 1 자당 1 개의 IO 를 표현하여 Multiple IO 의 경우 최대 10 개까지 가능하지만 현재 8 까지 IO 를 표현 가능하다. 각 Character 의 IO 상태는 0 부터 F 까지 가능하다. Multiple IO 를 선언할 때는 IO 의 수가 2 개부터 8 개까지 다양하게 선언할 수 있고 선언된 IO 가 xxxxxxxxxxxx 의 Character 와 연결되는 순서를 보면 선언된 IO 의 마지막 IO 가 xxxxxxxxxxxx 의 마지막(뒤)의 Character 와 연결되어 진다. 정의가 되어지지 않은 IO 의 Character 의 경우는 0 으로 선언하면 된다.

추가 1(Blink) (For first option as Blink) - Image Pointer 의 내용을 표현하려는 경

우 Blink 속성으로 선언되어 지면 IMG_xxxx_xxxxxxxxxx 에서 뒤의 xxxxxxxxxxxx 의 상태가 되었을 때 xxxxxxxxxxxx 를 나타내고 일정 시간 간격으로
IMG_xxxx_xxxxxxxxxx_1(Image 가 있으면) → IMG_xxxx_xxxxxxxxxx_2(Image 가 있으면) → IMG_xxxx_xxxxxxxxxx_3(Image 가 있으면)
→ IMG_xxxx_xxxxxxxxxx_4(Image 가 있으면) 식으로 계속 돌아가며 화면에
Bitmap File 을 보여주게 되며, IMG_xxxx_xxxxxxxxxx_%에서 %에 쓰여질(Blink

될) 개수는 16 개(0~F) 이다.(GOI Table 의 Graphic Object 중 IMGSHIFT 의 fIMG_SHIFT_N()를 참고 할 것)

추가 2(OR+AND) (For second option as logic OR added AND function) - Image Pointer 의 내용을 표현하려는 경우 OR+AND 속성으로 선언되어 지면(GOI Table 의 Graphic Object 중 IMAGE 의 fIMG_NORMAL_T()를 참고 할것) IMG_xxxx_xxxxxxxxxx 에서 뒤의 xxxxxxxxxxxx 의 상태가 되었을 때 xxxxxxxxxxxx Image 를 OR Operation 으로 그리고 이어서 xxxxxxxxxxxx _X 라고 이름 붙인 Image 를 AND Operation 으로 그리게 된다. 이것은 Bitmap 을 이용한 투명성을 적용하기 위한 방법으로, 투명성을 적용하기 위한 곳에 대해 xxxxxxxxxxxx Image 에는 Black(Bit 0)으로 표현하고 xxxxxxxxxxxx _X Image 에는 White(Bit 1)으로 표현 하면 투명성이 적용된 구간은 바탕내용이 표현되게 된다.

추가 3(Blink+OR+AND) (For third option as Blink added both logic OR and AND function)– 이 경우는 Blink 를 사용하는데 Image Pointer 의 내용을 표현하려는 속성을 OR+AND 속성으로 하려고 할 때 Blink 를 표현하는 IMG_xxxx_xxxxxxxxxx_x 에서 Image 를 OR Operation 으로 그리고 이어서 xxxxxxxxxxxx _x_X 라고 이름 붙인 Image 를 AND Operation 으로 그리게 된다. 이것은 Bitmap 을 이용한 투명성을 적용하며 Blink 를 적용하기 위한 방법이다 (GOI Table 의 Graphic Object 중 IMGSHIFT 의 fIMG_SHIFT_T()를 참고 할 것)

■ Font Property Define

- Font Property Define 은 GUI 에서 사용하려는 Font 및 Style 을 Table 화 하여 준비하는 것으로 Table 화 하여 준비하여 놓은 Font 의 내용을 A-1-6 에서 Text Property Define 에서 사용한다.
- 선언은 GDF File 에서 **#DECLARE FONT_TABLE** 의 선언을 사용하여 아래와 같이 나타내며 T:/HMI/TABLE/Font.tbl 이 Text Property Define 에서 사용할 Font 의 모음이 되어 있는 File(Font 정의 File)이다.

```
예) #DECLARE FONT_TABLE
      #INCLUDE "T:/HMI/table/font.tbl"
```

- Font 정의 File 은 Text 형태로 되어 있으며 공백을 간격으로 정의되게 된다. 아래의 내용이 Font 정의 File 에 내용을 정의한 예이다.

Font 정의 File						
FNT_10	10	0	FALSE	FALSE	FALSE	"Arial"
FNT_12	12	0	FALSE	FALSE	FALSE	"Arial"
FNT_10_I	10	0	TRUE	FALSE	FALSE	"Arial"
FNT_12_I	12	0	TRUE	FALSE	FALSE	"Arial"
FNT_10_U	10	0	FALSE	TRUE	FALSE	"Arial"
FNT_12_U	12	0	FALSE	TRUE	FALSE	"Arial"
FNT_10_I_U	10	0	TRUE	TRUE	FALSE	"Arial"
FNT_12_I_U	12	0	TRUE	TRUE	FALSE	"Arial"

각 자리(Tag)별 내용을 살펴보면

No	이름	의미	범위
1	Pointer	TextProperty Define, 사용할 Font Pointer	중복 불가
2	Height	Character 의 높이(Height)	8-128 까지의 Integer 값 (Between 8 to 128)
3	Weight	Character 의 두께(Weight)	
4	Italic	이탤릭체로 할 지의 유무	TRUE FALSE
5	Underline	밑줄을 사용할 지의 유무	TRUE FALSE
6	StrikeOut	StrikOut 사용할 지의 유무	TRUE FALSE
7	Font	사용할 Font 를 정의	String 으로 정의 (Define to use font through string)

■ Palette Property Define

- Palette Property Define 은 GUI 에서 사용하려는 Color Palette 를 Table 화 하여 준비하는 것으로 Table 화 하여 준비하여 사용한다.

- 선언은 GDF File 에서 **#DECLARE COLOR_TABLE** 를 선언 하고 아래와 같이 나타내며 T:\HMI\table\color.tbl 이 Text Property Define 에서 사용할 Color Palette 의 모음이 되어 있는 File(Palette 정의 File)이다.

ex) **#DECLARE COLOR_TABLE**
#INCLUDE "T:\HMI\table\color.tbl"

- Palette 정의 File 은 Text 형태로 되어 있으며 공백을 간격으로 정의되게 된다. 아래의 내용이 Palette 정의 File 에 내용을 정의한 예이다.

Palette 정의 File		
#red	255,000,000	//ff0000
#crimson	220,020,060	//dc143c
#firebrick	178,034,034	//b22222
#maroon	128,000,000	//800000
#darkred	139,000,000	//8b0000
(255 개까지 계속 정의할 수 있음)		

한 줄마다 하나의 Palette 를 의미하며 각각 Text Property Define 에서 사용하기 위한 Color Palette Pointer 를 가지게 되는데 처음 정의된 Tag 가 Pointer 값 이다.

■ Text Property Define

- Text Property Define 은 GUI 에서 IO 상태를 String 으로 표현하고자 할 때 표현하고자 하는 String 의 형태가 어떤 식으로 나타낼지를 미리 준비하는 것으로 준비하여 놓은 Text Property 의 내용을 Object Interface Part 에서 사용한다.

- 선언은 GDF File 에서 **#DECLARE** **GOI_TABLE** 를 선언하여 사용하여 아래와 같이 나타내며 **#INCLUDE** "T:\HMI\table\GOI.tbl" 이 File 의 모음중에 TEXT 로 되어 있는 부분이 Text Property 를 정의한 것 이다.

ex) **#DECLARE** **GOI_TABLE**
#INCLUDE "T:\HMI\table\GOI.tbl"

- Text 정의 File 은 Text 형태로 되어 있으며, 아래의 내용이 Text 정의 File 에 내용을 정의한 예이다.

Text 정의 예

```

TEXT      TEXT_D_F10_I_U(StrLen) {
            #FONT                FNT_10_I_U
            #ALIGN                Center_Middle
            #COLOR_TEXT          #Black
            #COLOR_BACK          #Silver
            #COLOR_LT            #Black
            #COLOR_RB            #White
        }
    
```

Text 를 화면상에 표시할 때는 Font, TextAlign , TextColor , BackColor, LineUpColor , LineDownColor 의 6 가지로 나누어 정의할 수 있다. Text 는 사각형 영역 안에서 표현되어 지게 되는데 사각형 영역을 위해서 LineUpColor , LineDownColor , BackColor 를 이용하여 정의되고 쓰여지는 Text 를 위해서는 TextAlign , TextColor , Font 를 이용하여 정의된다.

각 라인별 내용을 살펴보면

No	이름	의미	범위
1	TEXT_D_F10_I_U	Object Interface Part 에서 사용할 Text Pointer (Is a text pointer to be able to use at Object Interface Part)	
2	#ALIGN	사각형 영역 안에서 Text 의 Align 위치 (Aligned text location within square)	CENTER (가운데) LEFT (왼쪽) RIGHT (오른쪽)
3	#COLOR_LT	Line Up Color	정의 되어 있는 시스템 Color or Palette 에서 정의된 Integer 값의 Pointer (Integer value pointer of System or palette color)
4	#COLOR_RB	Line Down Color	
5	#COLOR_TEXT	Text Color	
6	#COLOR_BACK	사각형의 Background Color (Background Color of Square)	

7	#FONT	Text 에 사용될 Font Type (Font type to be able to use Text)	Font 에서 정의된 Integer 값의 Pointer (Pointer of predefined integer value from font)

참조) 미리 정의되어 있는 시스템 Color 는 Color.tbl 참고 할것(약 140 개).

■ Page Setup

- Multi Page 를 위해 Page 를 구분하는 방법은 #SCREEN 0x0111(Page 번호 0x111) 과 같이 Page 번호를 정의하면 된다. (Page 번호는 중복 불가하다)

ex) #SCREEN 0x0111 BackImage 0 110 1170 804

0x0111 – Page 번호

BackImage – 사용할 BackGround Image

0 – Start X pos

110 – Start Y pos

1170 – Length X

804 – Length Y

- Page 는 Image Background 에서의 Background 와 연결되어 사용된다. 0x0111 Page 인 경우는 BACKGROUND_0111 과 같이 등록된 Page 번호와 같은 것이 할당된다..

■ Popup Screen

- Popup Screen 을 등록하기 위해서는 3 부분의 정의가 필요하다.

1. GDF file 에 Popup screen 정의

ex) #POPUP_SCREEN 1 BackImage "PopupTitle" "GUI/PMC/PopUp_Lamp_Life.cfg"

각 자리(Tag)별 내용을 살펴보면

No	Tag	의미	기타
1	#POPUP_SCREEN	고정값 (Fixed)	
2	1	Popup Window 를 위한 Background Pointer	
3	BackImage	Popup Background Image 등록 위치	
4	""	Popup Screen 의 Title	
5	""	Popup Screen 내에 나타날 내용이 있는 File	

2. GDF file 에 Popup Screen Button 정의.

ex) - - ImgLampLifeBtn("") "" 581 710 - - POPUP_LAMPLIFE() "" FALSE

No	Tag	의미	기타
1	-	고정값 (Fixed)	
2	-	고정값 (Fixed)	
3	ImgLampLifeBtn()	GOI.Table 에 등록된 IMGBTN	
4	""	Button Visable 조건	
5	581	Button 위치 X 좌표	
6	710	Button 위치 Y 좌표	
7	-	Button X size	없을때는 button 크기
8	-	Button Y size	없을때는 button 크기
9	POPUP_LAMPLIFE ()	GOI.Table 에 등록된 POPUPWND	
10	""	Button Click enable 조건	
11	FALSE	중첩되었을 경우 하위에 event 발생 조건	

3. Tag 8 - POPUP_LAMPLIFE ()

GOI.Table 에 등록 되어 있어야 한다.

```
ex) POPUPWND      POPUP_LAMPLIFE {
    #IMAGE_LIB      BackImage
    #POPUP_ID        1
    #TITLE           "Lamp Life Time"
    #PATH            "GUI/PMC/PopUp_Lamp_Life.cfg"
    #MODAL           TRUE
}
```

No	Tag	의미	기타
1	POPUP_LAMPLIFE	Popup Window 에 대한 Pointer	
2	#IMAGE_LIB	Popup Window 를 위한 Background 위치	
3	#POPUP_ID	Popup Background Image 번호	
4	#TITLE	Popup Screen 의 Title	
5	#PATH	Popup Screen 내에 나타날 내용이 있는 File 위치	
6	#MODAL	Modal Box 여부 설정	FALSE/TRUE

■ Popup Screen Close

- Popup Screen 의 Open 후 Close 를 위한 Button 을 정의하고자 할 때 사용된다.

ex) - - **ImgCloseBtn_1()** "" 980 2 - - **POPUP_CLOSE(1)** ""

각 자리(Tag)별 내용을 살펴보면

No	Tag	의미	기타
1	-	고정값 (Fixed)	
2	-	고정값 (Fixed)	
3	ImgCloseBtn_1()	GOI.Table 에 등록된 IMGBTN	
4	""	Button Visable 조건	
5	980	Button 위치 X 좌표	
6	2	Button 위치 Y 좌표	
7	-	Button X size	없을때는 button 크기
8	-	Button Y size	없을때는 button 크기
9	POPUP_CLOSE()	GOI.Table 에 등록된 POPUPCLOSE	
10	""	Button Click enable 조건	

POPUP 을 위해 정의된 GDF File 안에서 만 사용되며 Popup Screen 을 Close 하기 위한 Button 을 사용자가 원하는 형태로 만들기 원할 때 사용된다.

(It is only used within GDF file for POPUP function, use to made a button by user to be able to close Popup Screen).

■ FILE Include (Utility)

- File Include 는 다른 File 에 선언한 Interface Object 를 현재의 GDF File 에서 사용하는 것을 말한다.

ex) **#FILE "Overview.cfg" 275 190**

위의 ex)를 보면 “Overview.cfg” 의 GDF File 이 삽입되어 사용되어 진다. 뒤에 있는 275 와 190 은 “Overview.cfg” File 의 내용에만 적용되는 Reference X 값과 Reference Y 값이다. “Overview.cfg” File 안에 정의된 Object 의 표현 위치가 10,10 으로 정의되어 있다면 실제로 보여지는 위치는 10+275,10+190 인 285,200 의 위치에 보여지게 된다.

- File Include 는 Recursive 하게 Include 된 File 안에서도 사용할 수 있으나, Popup Screen 을 위해 연결한 File 안에서는 사용할 수 없다.

■ Name Replacement(Utility)

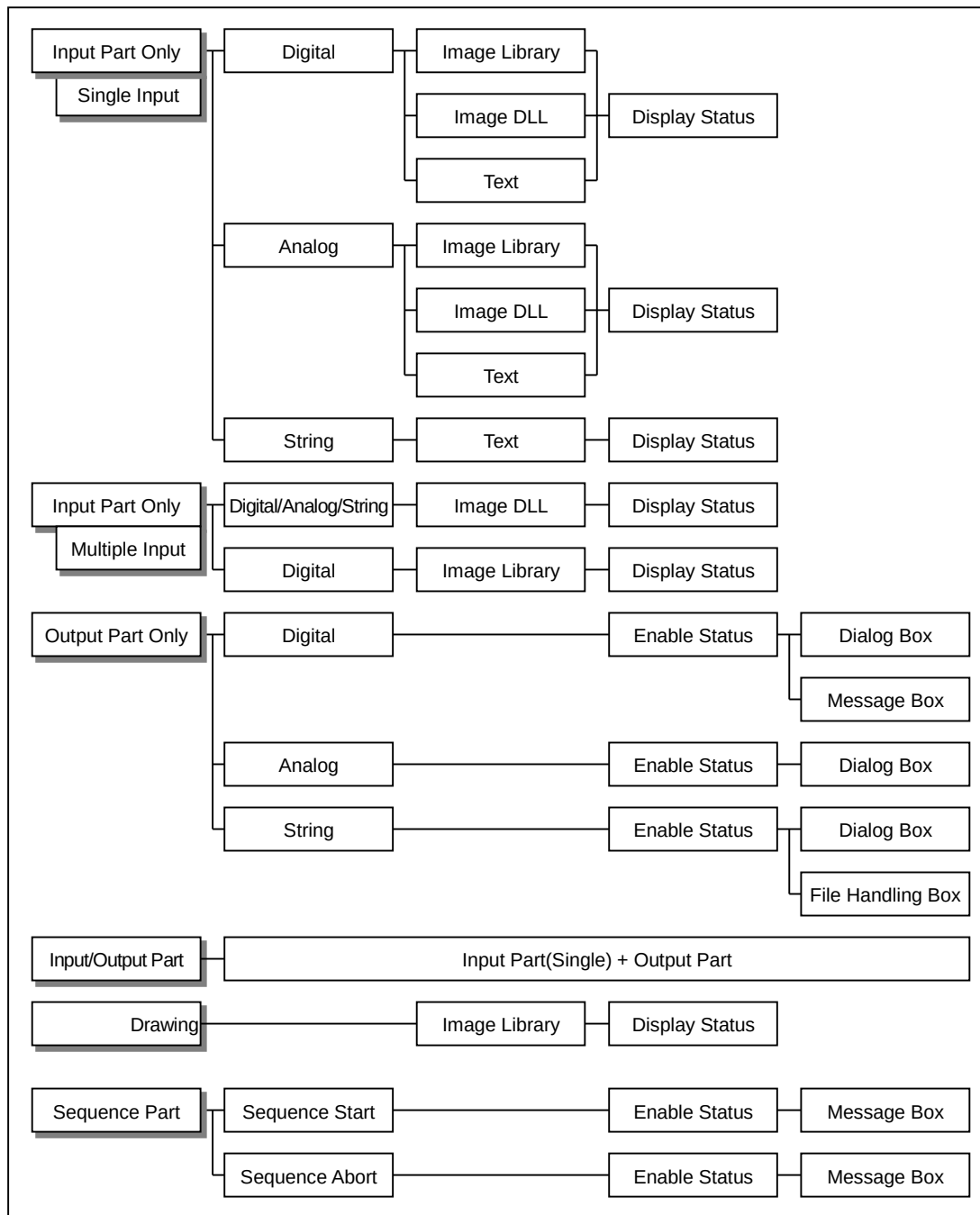
- GDF File 에 표현할 IO 의 이름에 공통적으로 같은 부분이 있을 때 이 부분을 따로 관리하고자 할 때 사용된다.

ex) **#REPLACER "PM2."**

위의 ex)를 보면 뒤에 있는 “PM2.” 의 String 내용은 IO 이름을 정의한 곳에 #의 문자가 존재할 경우 그것이 “PM2.”으로 대치 된다. 위의 예)처럼 정의 하고 IO 이름을 정의 하는 곳에 #Valve 란 이름으로 IO 를 선언 하였다면 실제 IO 의 이름은 PM2.Valve 가 되어진다.

■ Object Interface Part

- Object Interface Part 는 실제로 표현하고자 하는 내용을 설정하는 부분으로 Digital , Analog , String , Sequence 의 내용을 Input 과 Output 으로 나누어 Input 에서는 Image, Image DLL, Text 중에서 표현하며 Output 에서는 어느 영역이 Output 영역이며 표현을 어떻게 할지 정의하게 된다.
- Object Interface Part 는 아래와 같은 Structure 를 가지게 된다.
- 선언할 수 있는 Object 의 개수는 최대 300 개이다.



■ Object Tag Structure

- Object 를 나타내기 위해 GDF File 에 정의하기 위해 Tag(공백으로 구분)형식의 구조를 가진다. Object 는 한 줄에서 선언하여야 하며 총 11 개의 Tag 가 그 Object 의 특성의 결정 짓게 된다(Input 의 경우는 8 개의 Tag 가 사용된다).

```
ex) Gas1_Flow  AI  TEXT_D_F18(8)  ""  125  149  70  22
      Gas1_Set  AO  TEXT_S_F18(8)  ""  125  120  70  22 IO_POPUP() ""  FALSE
      MAIN_SEQUENCE  SEQ ImgExeBtn_2("Process")  ""  923  76  -  -
                        SEQ_RUN("PROCESS")  "MAIN_CONTROL==0"
      MAIN_SEQUENCE  SEQ  ImgAbortBtn_2("ABORT")  ""  1037  76  -  -
                        SEQ_ABORT_ALL("ABORT")  "MAIN_CONTROL==1"
      RECIPE_FILE  SIO  TEXT_S_F18(20)  ""  916  131  227  24
                        RCP_OPEN("Recipe\Process")  "MAIN_CONTROL==0"
```

No	Tag	의미	기타
1	Object Name	IO 이름	
2	Object Type	DI/DO/DIO, AI/AO/AIO, SI/SO/SIO, SEQ	
3	Input Instance	Screen display Input Instance Name	GOI.Table 에 등록된 것
4	""	IO or Button Visable 조건	
5	125	IO or Button 위치 X 좌표	
6	149/120	IO or Button 위치 Y 좌표	
7	70	Button X size	없을때는 button 크기
8	22	Button Y size	없을때는 button 크기
9	Output Instance	Screen display Output Instance Name	GOI.Table 에 등록된 것
10	""	IO or Button Click enable 조건	
11	FALSE	중첩되었을 경우 하위에 event 발생 조건	FALSE or TRUE

- Input/Output Instance 은 GOI.Table 에 등록 하여 사용가능 하다.

아래 Instance 에 대한 표를 참조

Instance	ex	설명	
TEXT	<pre>TEXT TEXT_NAME(StrLen) { #FONT FNT_10 #ALIGN Center_Middle #COLOR_TEXT #Black #COLOR_BACK #Silver #COLOR_LT #Black #COLOR_RB #White }</pre>	TEXT_NAME : Instance 이름, StrLen:Test Display 길이	
		#FONT : Font.Table 에 등록된 Pointer	
		#ALIGN : TEXT Align 방법	
		#COLOR_TEXT : Text Color	
		#COLOR_BACK : Background Color	
		#COLOR_LT :BoxLine Left Top Color	
		#COLOR_RB :BoxLine Right Bottom Color	
	TEXT BOX 관련 Instance 이다.		
STATIC	<pre>STATIC STATIC_NAME(StrLen) { #TEXT TEXT_NAME }</pre>	STATIC_NAME : Instance 이름, StrLen:display 길이	
		#TEXT : GOI.Table 에 등록된 TEXT Instance	
	고정된 문자열을 표현시 사용되는 STATIC TEXT BOX 이다		
IMAGE	<pre>IMAGE IMG_NAME(ImgID) { #IMAGE_LIB ImgLibrary #MODE NORMAL }</pre>	IMG_NAME : Instance 이름, ImgID: 등록된 Img 번호	
		#IMAGE_LIB : Img 가 등록된 DII	
		#MODE ;NORMAL=Img 를 그대로 display TRANS = Black & White 된 부분을 투명하 게 display	
		TRANS 인경우 Img 등록예 IMG_0101_0000000001 IMG_0101_0000000001_X	
Animation 을 위한 Image 을 display 하기 위해 사용되는 Instance 이다			
IMGSHIFT	<pre>IMGSHIFT IMG_NAME(ImgID) { #IMAGE_LIB ImgLibrary #INTERVAL 100 #MODE NORMAL }</pre>	IMG_NAME : Instance 이름, ImgID: 등록된 Img 번호	
		#IMAGE_LIB : Img 가 등록된 DII	
		#INTERVAL : Img reflash Interval 이다	
		#MODE ;NORMAL=Img 를 그대로 display TRANS = Black & White 된 부분을 투명하 게 display	
	TRANS 인경우 Img 등록예 IMG_0101_0000000001 IMG_0101_0000000001_X		
IMAGE 와 동일한 기능이나, 자동으로 Img 를 다음것으로 변경하여 display 한다.			
IMGBTN	<pre>IMGBTN IMGBTN_NAME(Text) { #IMAGE_LIB ImgLibrary #IMAGE_ID 102 #FONT FNT_20 #COLOR_TEXT_ENABLE #Black #COLOR_TEXT_DISABLE #Gray }</pre>	IMGBTN_NAME : Instance 이름, Text : Button 에 표시할 버튼 이름	
		#IMAGE_LIB : Img 가 등록된 DII	
		#IMAGE_ID : 등록된 Img 번호	
		#FONT : Font.Table 에 등록되어 있는 것	
		#COLOR_TEXT_ENABLE : Enable 상태시 Text Color(#Black 은 Color.Table 에 등록되어 있는 것)	
		#COLOR_TEXT_DISABLE : Disable 상태시 Text Color(#Gray 은 Color.Table 에 등록되어 있는 것)	
		(0:Enable, 1:Press, 2:Disable)	
	Button display 시 사용되는 Instance 이다.		
MSGBOX	<pre>MSGBOX MSG_RUN(AddMsg) { #TITLE "SEQUENCE" #MESSAGE "Do You Want Start Program??" #BUTTON "YES CANCEL" #RETURN 0 }</pre>	MSG_RUN : Message Box Instance 이름 AddMsg : 전달받은 Parameter 내용	
		#TITLE : Message Box Title	
		#MESSAGE : Message Box 에 표시될 내용	
		#BUTTON : Message Box 에 표시될 버튼	
	<pre>MSGBOX MSG_ABORT(AddMsg) { #TITLE "SEQUENCE" #MESSAGE "Do You Want Abort Program??" #BUTTON "YES CANCEL" #RETURN 0 }</pre>	#Return : default 0 이다	
			//DEFAULT
	SEQUENCE 실행시 Message Box 를 display 하기위해 정의한다		

SEQUENCE	SEQUENCE SEQ_RUN(Param) { #MSGBOX MSG_RUN(Param) #MODE RUN }	SEQ_RUN : Sequence Instance 이름 Param : 전달받은 Parameter
	SEQUENCE SEQ_EVENT(Param) { #MSGBOX MSG_RUN(Param) #MODE EVENT } SEQUENCE SEQ_ABORT(Param) { #MSGBOX MSG_ABORT(Param) #MODE ABORT } SEQUENCE SEQ_ABORT_ALL(Param) { #MSGBOX MSG_ABORT(Param) #MODE ABORT_ALL } }	#MSGBOX : GOI.Table 에 등록된 MSGBOX call #MODE : RUN=Excute, EVENT=Event, ABORT=Abort, ABORT_ALL= All Abort
	Sequence 를 Run/Event/Abort 하기위한 Instance 이다.	
FILEDIALOG	FILEDIALOG RCP_OPEN(Path) { #MODE OPEN }	RCP_OPEN : File 선택을 위한 Instance 이름 Path : Open 할 경로
		#MODE : default OPEN
	File Open 을 위한 Instance 이다	
POPUPCLOSE	POPUPCLOSE POPUP_CLOSE { }	POPUP_CLOSE : Popup Close 를 위한 Instance 이다
ECCTRL	ECCTRL P_CONTROL { }	P_CONTROL : Parameter Screen 의 Load/Adapt/Save/Cancel 버튼 올릴 때 사용함

Parameter Interface Part

- Parameter Interface Part 는 IO 의 내용을 저장하거나 저장되어 있는 내용들을 IO 의 값으로 적용하고자 할 때 사용된다. 하나의 Page 당 각각의 Parameter Group 을 구성할 수 있으며 Setup Part 와 IO Template Part 로 구성되어 있다. Parameter 는 실제 IO 와 같은 방법으로 화면상에서 값을 줄 수 있고 보여지게 되는데 실제 IO 와 다른 점은 화면상에 보여지는 내용이 실제 IO 의 내용이 아니라 가상으로 보관하고 있는 내용이라는 것이다. 가상으로 보여지는 내용은 File 에 저장할 수도 있고 실제 IO 에 그 내용을 적용할 수도 있다.

IO Template Part

- Output Object 를 정의한 것과 같은 방법으로 정의하면 된다. 다른 점은 Tag 2 에서 Digital 의 경우는 DO 대신 P_DO 를 Analog 의 경우는 AO 대신 P_AO 를 String 의 경우는 SO 대신 P_SO 로 정의 하여 주면 된다.

Setup Part

- IO Template 로 구성된 내용을 위한 다양한 Operation 을 나타낸다.
- #PARAMETER
 - IO Template 에 대한 내용을 저장하고 읽어올 File(file name)을 정의한다.

예)

#PARAMETER "" "file name" 3

- Tag 3

File name = File Path & File Name

- Tage

1= AUTO_ADAPT_WHEN_LOA,

2= AUTO_ADAPT_WHEN_SAVE

3= AUTO_ADAPT_WHEN_LOAD or AUTO_ADAPT_WHEN_SAVE

- LOAD/ADAPT/SAVE/CANCEL 버튼

예)

- - ImgExeBtn_2("LOAD") "" 1047 290 - - P_CONTROL("LOAD") ""

- - ImgExeBtn_2("ADAPT") "" 1047 360 - - P_CONTROL("ADAPT") ""

- - ImgExeBtn_2("SAVE") "" 1047 430 - - P_CONTROL("SAVE") ""

- - ImgExeBtn_2("CANCEL") "" 1047 500 - - P_CONTROL("CANCEL") ""

-

Chapter 11

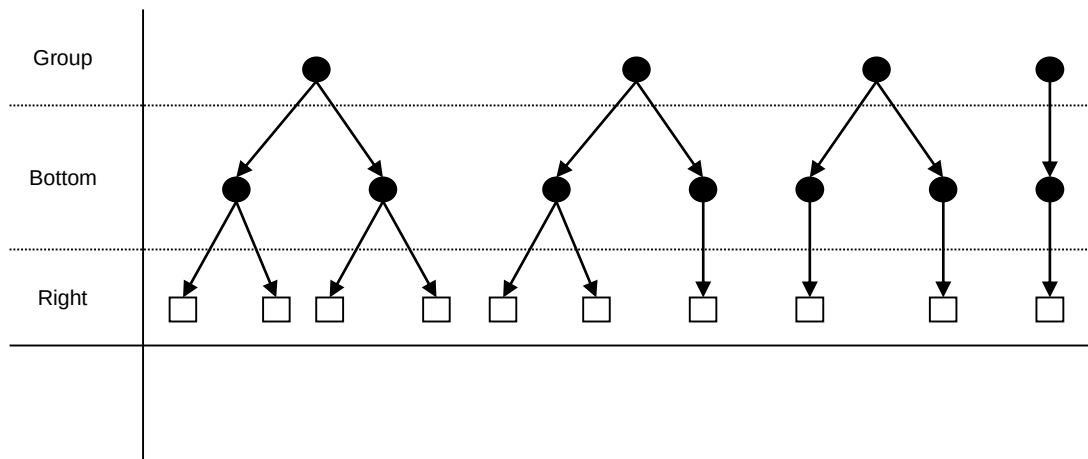
Screen Navigation

In this chapter...

- ◆ Screen Navigation 의 정의
- ◆ User Group(Security)
- ◆ GUI Navigation(Scenario) Define
- ◆ System GUI IO
- ◆ GUI Navigation Define 예제

Screen Navigation 의 정의

- Screen Navigation 은 User 에게 보여줄 GUI Screen 의 Menu 체계를 작성하여 Bottom Bar -> Right Bar 로 연계되는 Tree 구조로 설계되어 있다.
- Screen Navigation 은 Tree 구조의 Bottom(BottomBar)->Right(RightBar) 순서에 의해 결정된다. Group 은 Module Navigation 과 같이 동일한 Bottom Bar 또는 Right Bar 위치에 다른 버튼을 올려 사용하고자 할 때 정의한다. 일반적으로 하나의 Group 으로 사용하는 경우가 대부분 이다.



- Fest Engine 에서 지원하고 있는 기본 Button Image 는 아래 PDF 파일 참고 할 것

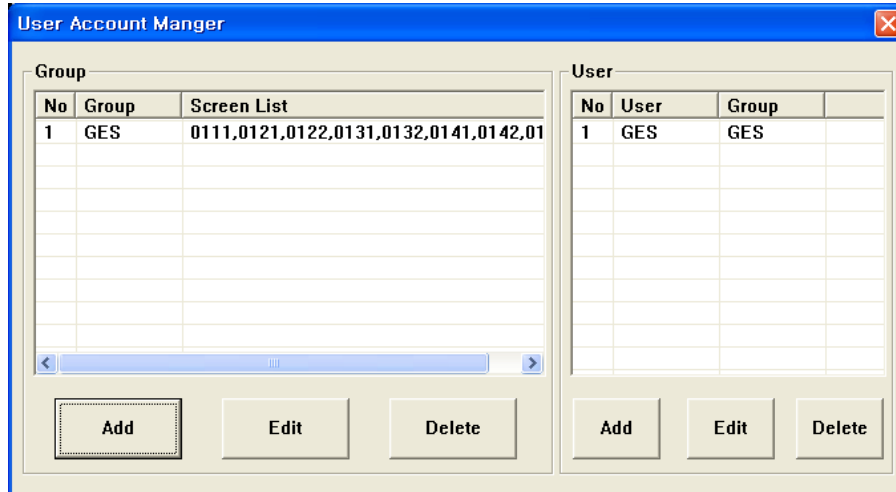
Top Image = t:/Hmi/Image/FEST_IMG_TOP.pdf

Bottom Image = t:/Hmi/Image/FEST_IMG_BOTTOM.pdf

Right Image = t:/Hmi/Image/FEST_IMG_RIGHT.pdf

■ User Group(Security)

- GUI Navigation Screen 에 대한 접근 가능한 권한을 LogIn User Group 별로 설정 가능하다.
- FEST Control Window 의 User Account Manager 에서 Group 및 User 를 수정/삭제/등록 가능하다.



■ GUI Navigation(Scenario) Define

- GUI Navigation(Scenario)에 대한 정의는 Project/Gui/LayOut/Screen_Bottom.cfg 와 Screen_Right.cfg 라는 File 에 정의한다. 이 File 에서는 Navi.ScreenID IO 를 이용하여 Bottom, Right 에 따른 어떤 Screen 를 Open 할 것인지에 대해 정의한다.

- Navigation Scenario Format.

ex) Bottom

```
Navi.ScreenID DIO fNaviBtn(2,0x120) "" 487 7 0 0 IO_SET(0x120) "" FALSE
```

위에서 fNaviBtn(2,0x120) / IO_SET(0x120) 이부분이 Navigation 과 관련되 부분이다

2 : Button Image 번호

1 : Group ID

2 : Bottom Bar ID

0 : Right Bar ID(0 일때는 기존에 열어본 Right Bar 가 Open)

(숫자 정의시는 정의된 Right Bar 가 Open)

ex) Right

```
Navi.ScreenID DIO fNaviBtn(21,0x123) "" 487 7 0 0 IO_SET(0x123) "" FALSE
```

위에서 fNaviBtn(21,0x123) / IO_SET(0x123) 이부분이 Navigation 과 관련되 부분이다

21 : Button Image 번호

1 : Group ID

2 : Bottom Bar ID

3 : Right Bar ID

여기서 0x123 이 Open 되어지는 Screen Page ID 이다

- Bottom Bar ID : Button ID of Bottom Bar(from 1 to F)
- Right Bar ID : Button ID of Right Bar(from 1 to F)
- Open ID : GUI Sequence 에서 정의한 Sequence 가 여러 개의 Screen 를 가지고 있는 경우 그 중 열고자 하는 Screen 의 Page ID 이다. .

System Gui IO

- GUI IO 를 관장하는 Driver 는 NAVIGATION 으로 FEST(Main Engine) 내부의 driver 이다.
- GUI IO 는 String IO 와 Digital IO 두 종류가 존재한다.

String IO - Navi.UserName/ Navi.GroupName

Digital IO - Navi.Mode / Navi.ScreenID

GUI String IO Part

- Navi.UserName 과 Navi.GroupName 은 사용자가 Login 할 때 다이얼로그 박스가 나타나는 데 이 다이얼로그 박스를 통해 사용자의 정보가 입력되고 GUI 에서 Login 창에 나타난다.

Login	
OEM	Group Name
GES	User Name

GUI Digital IO Part

- Navi.Mode : Navigation Button 에 대한 Group select 시 사용된다
- Navi.ScreenID : Group/Bottom/Right 에 대한 정의시 사용된다.
- Bottom Bar 는 프로그램을 실행시키면 아래부분에 항상 나타나는 Bar 이다.



- Right Bar 는 프로그램을 실행시키면 오른쪽 부분에 항상 나타나는 Bar 이다



■ GUI Navigation Define 예제

■ Example Data

[Screen_Bottom.cfg]

```
//Main
X, Y : Postion
Navi.ScreenID DIO fNaviBtn(2,0x110) "1" 7 7 0 0 IO_SET(0x110) "" FALSE

//Recipe
Navi.ScreenID DIO fNaviBtn(3,0x120) "1" 107 7 0 0 IO_SET(0x120) "" FALSE

//PMMaint
Navi.ScreenID DIO fNaviBtn(4,0x130) "1" 207 7 0 0 IO_SET(0x130) "" FALSE
```

[Screen_Right.cfg]

```
//Main-System
X, Y : Postion
Navi.ScreenID DIO fNaviBtn(2,0x111) "1" 7 7 0 0 IO_SET(0x111) "" FALSE

//Main-Transfer
Navi.ScreenID DIO fNaviBtn(2,0x112) "1" 107 7 0 0 IO_SET(0x112) "" FALSE

//Recipe-Root
Navi.ScreenID DIO fNaviBtn(3,0x121) "1" 7 7 0 0 IO_SET(0x121) "" FALSE

//Recipe-Route
Navi.ScreenID DIO fNaviBtn(3,0x122) "1" 107 7 0 0 IO_SET(0x122) "" FALSE

//Recipe-Process
Navi.ScreenID DIO fNaviBtn(3,0x123) "1" 207 7 0 0 IO_SET(0x123) "" FALSE

//PMMaint-Temp
Navi.ScreenID DIO fNaviBtn(4,0x131) "1" 7 7 0 0 IO_SET(0x131) "" FALSE

//PMMaint-Motor
Navi.ScreenID DIO fNaviBtn(4,0x132) "1" 107 7 0 0 IO_SET(0x132) "" FALSE

//PMMaint-Power
Navi.ScreenID DIO fNaviBtn(4,0x133) "1" 207 7 0 0 IO_SET(0x133) "" FALSE
```

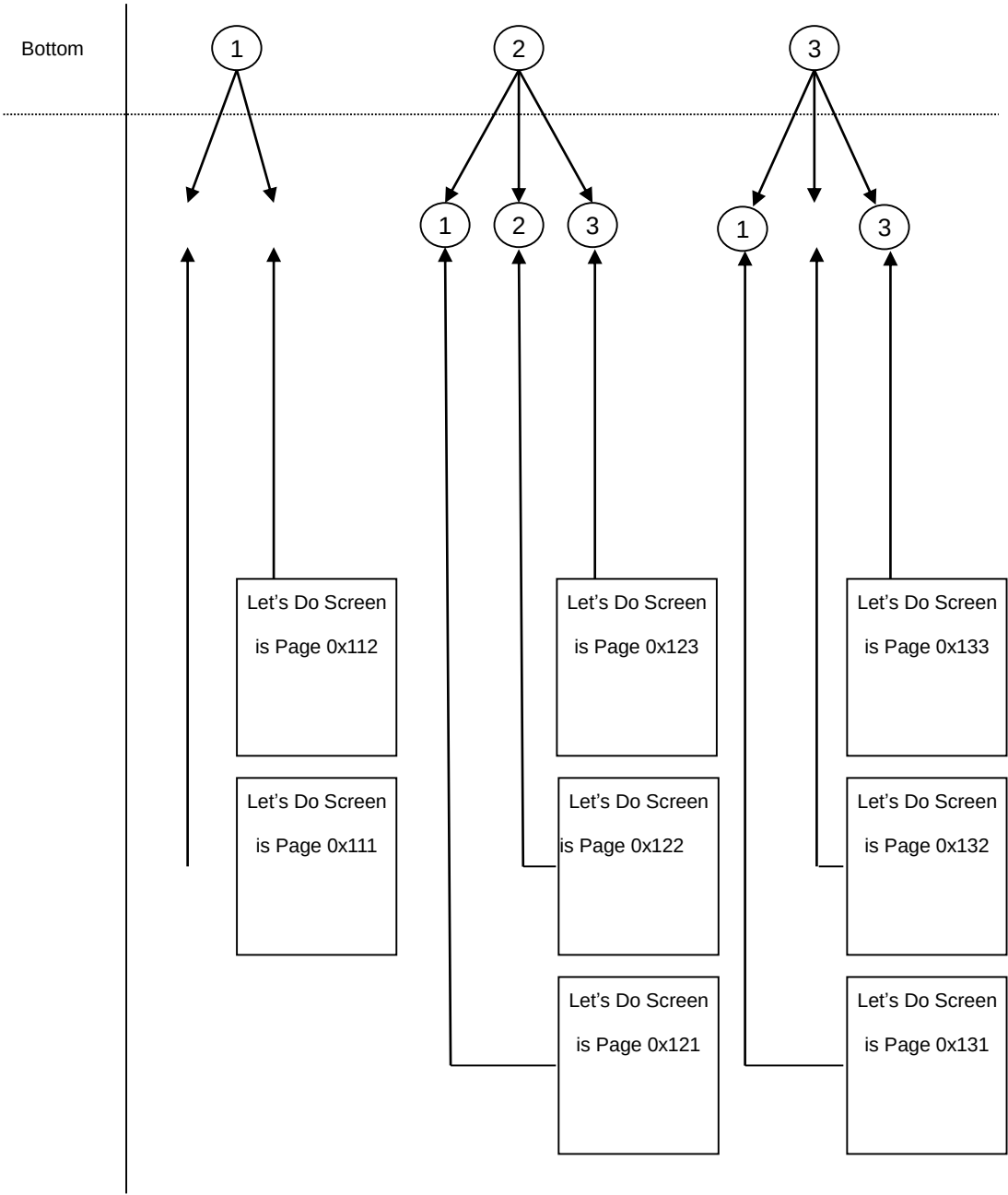
■ Make Bottom/Right Index

Group index 은 1 인 경우

Bottom Bar	Right Bar	Screen Info	Comment
1	1	Main-System	
	2	Main-Transfer	

2	1	Recipe-Root	
	2	Recipe-Router	
	3	Recipe-Process	
3	1	PMMaint-Temp	
	2	PMMaint-Motor	
	3	PMMaint-Power	

Node Flow Chart



Advance

Chapter 12: RS232 Driver

Chapter 13: TCP/IP Driver

Chapter 12

RS232 Device Driver

What's In this chapter...

- ♦ RS232 Device Driver 관련 File
- ♦ Setup Part 의 역할
- ♦ Setup Part - TComSerial() / Initialize() / Connect(), CleanUp() / Disconnect()
- ♦ Write Part 의 역할
- ♦ Write Part - Send()
- ♦ Read Part 의 역할
- ♦ Read Part - ClearBuffer()
- ♦ Read Part - Recv() (1st)
- ♦ Read Part - Recv() (2nd)
- ♦ Log Part 의 역할
- ♦ Log Part - StartLog() / StopLog()

RS232 Device Driver 관련 File

- RS232 Device 의 IO Handling Driver 제작시 Interface 를 위해 RS232 Interface Library 를 제공한다.
- 관련 Library file : TDrvIF.lib, TSeqIF.lib, TComSerial.lib
- 관련 Header file : TDrvInterface.h, TSeqIF.h, TComSerial.h
- TComSerial.h(Header File)는 아래와 같이 Interface Driver 작성을 위한 Class 을 제공한다.

HEADER File

```
#ifndef __T_COM_SERIAL_H_
#define __T_COM_SERIAL_H_
//-----
#include <TComBase.h>
#include <TIOData.h>
#include <TThread.h>
#include <TUtilSync.h>
#pragma comment( lib , T_LIB(TComSerial.lib) )
//-----
class DLL_INTERFACE TComSerial : public TComBase {
public:
    enum { eNotAvailable , eInitialized , eConnected };
    typedef struct _tagComInf {
        unsigned char nPort;
        int nBaudRate;
        unsigned char nDataBit;
        unsigned char nStopBit;
        unsigned char nParity;
        unsigned char FlowControl;
        unsigned long dwOldMask;
    } COMINF;
};
```

```

public:
    TComSerial( int PortNum , int BaudRate , int DataBit , int StopBit , int ParityBit ,
                bool DTRDSR=false , bool RTSCTS=false , bool XONXOFF=false );
    TComSerial( TComHandler *pcComHandler , int PortNum , int BaudRate , int DataBit , int StopBit ,
                int ParityBit , bool DTRDSR=false , bool RTSCTS=false , bool XONXOFF=false );
    virtual ~TComSerial();

public:
    bool Initialize();
    bool CleanUp();
    bool Connect();
    bool Disconnect();
    bool Send( const char* szSendData , int nNumOfBytesToSend );
    bool Recv( char* szRecvBuf , const char* szDelimiter , int nNumOfByteToRecv ,
                int* nNumOfBytesTransferred , int TimeOut );
    bool Recv( char* szRecvBuf , int nNumOfByteToRecv , int *nNumOfBytesTransferred ,
                int TimeOut );
    void ClearBuffer();

public:
    long GetConnectionState() const;
    long GetLastError() const;
    void StartLog( const char* szLogFile , bool fAscii );
    void StopLog();

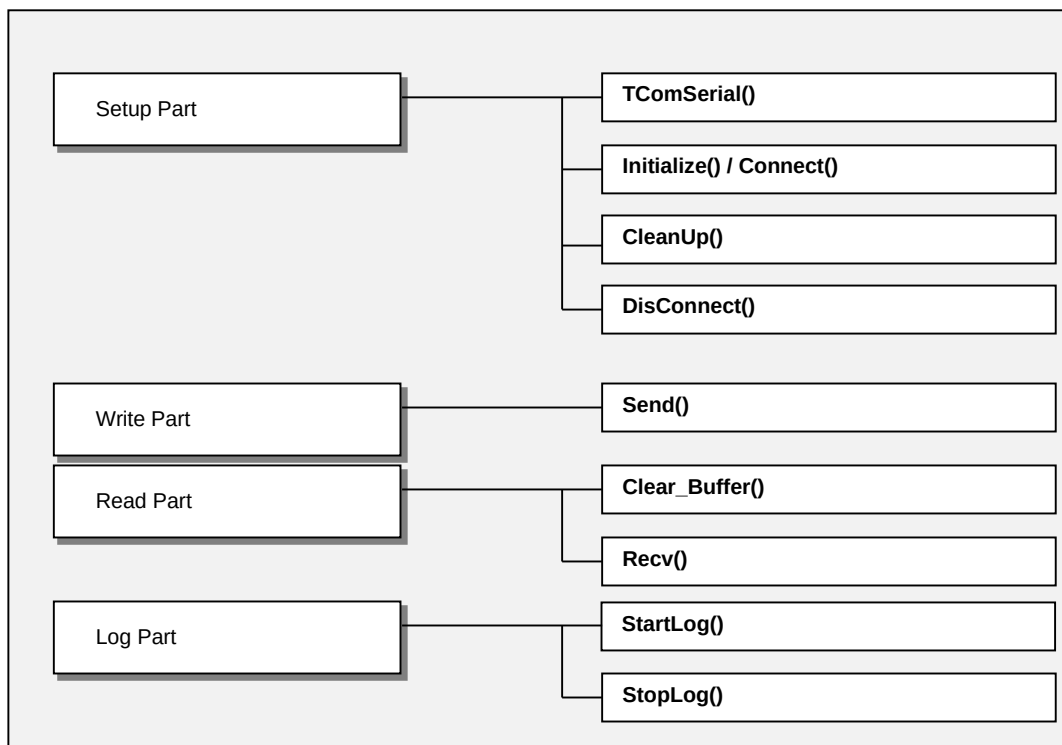
public:
    bool SetEventString( const char* szEventStr );
    bool WaitEventString( unsigned long nWaitTime=INFINITE );

protected:
    void SetConnectionState( long nConnectionState );
    unsigned ThreadHandlerStartPoint( TThread* pcThreadIdentity , void* pParam );

private:
    bool BuildDCB();
    size_t ReadCommBlock();
};

```

- Class 의 Method 중 통신관련하여 Setup, Write, Read, Log Part 로 나뉜다.



■ Setup Part 의 역할

- 장비에 연결된 RS232 Device 와의 통신 Port 를 열고 닫는 부분으로
TComSerial() / Initial() / Connect() / CleanUp() / Disconnect() 5 개의 Function 을 제공한다.
- TComSerial()/Initialize()/Connect() Function 은 IO Handling Driver 의 OnInitDevice() 부분에 삽입
되어지며, CleanUp()/Disconnect() Function 은 IO Handling Driver 의 OnKillDevice() 부분에 삽입
되어진다.

■ Setup Part - TComSerial() / Initialize() / Connect()

- Prototype : TComSerial(int PORT ,
int BAUD ,
int DATA ,
int STOP ,
int PARITY ,
bool DTRDSR ,
bool RTSCTS ,
bool XONXOFF);
: BOOL Inialize();
: BOOL Connect();
- Function 의 역할 : Parameter 인자에 맞추어 RS232 Port 를 Open 한다.
- Parameter 의 의미

Parameter	형식	설 명
PORT	Integer	- 열고자 하는 Port 번호 - 제어판의 포트에 설정되어 있는 COM Port 의 Port 번호만 가능하다. (단, 가상 Serial Port 프로그램을 사용하는 경우 해당 Program 의 Port 설정에 따른다.)
BAUD	Integer	- 열고자 하는 Port 의 속도 - 256000, 128000, 115200, 57600, 56000, 38400, 19200, 14400, 9600, 4800, 2400, 1200, 600, 300 , 110 의 속도 중에서만 사용하여야 한다.
DATA	Integer	- 열고자 하는 Port 의 Data Bit - 5 , 6 , 7 , 8 중에서만 사용하여야 한다.
STOP	Integer	- 열고자 하는 Port 의 Stop Bit - 1 , 2 , 3 중에서만 사용하여야 하며 1 일 때는 1Bit, 2 일 때는 2Bit, 3 일 때는 1.5Bit 를 의미한다.
PARITY	Integer	- 열고자 하는 Port 의 Parity Bit - 0 , 1 , 2 , 3 , 4 중에서만 사용하여야 하며 0 일 때는 No Parity, 1 일 때는 Odd Parity, 2 일 때는 Even Parity, 3 일 때는 Mark Parity, 4 일 때는 Space Parity 를 의미한다.

DTRDSR	Integer	- 열고자 하는 Port 가 DTR/DSR 를 사용할 지 결정 - 0 일 때는 DTR/DSR 을 사용하지 않음을 의미하고, 0 이외의 값일 때는 DTR/DSR 을 사용함을 의미한다.
RTS/CTS	Integer	- 열고자 하는 Port 가 RTS/CTS 를 사용할 지 결정 - 0 일 때는 RTS/CTS 를 사용하지 않음을 의미하고, 1 일 때는 RTS/CTS 를 사용함을 의미하고, 그 이외의 값일 때는 RS485 Style 의 RTS/CTS 를 사용함을 의미한다.
XONXOFF	Integer	- 열고자 하는 Port 가 XON/XOFF 를 사용할 지 결정 - 0 일 때는 XON/XOFF 를 사용하지 않음을 의미하고, 0 이외의 값일 때는 XON/XOFF 를 사용함을 의미한다.

- Return – Initialize(), Connect()

Port 가 정상적으로 열렸을 때는 TRUE 열리지 않았을 때는 FALSE

■ Setup Part - CleanUp() / Disconnect()

- Prototype : bool CleanUp(); bool Disconnect();
- 역 할 : 열려 있는 Port 를 닫는다.
- Parameter : 없음
- Return : 없음

■ Write Part 의 역할

- RS232 Port 에 Data 를 Writing 하기를 원할 때 사용된다.
- Send() Function 을 제공한다.

■ Write Part - Send()

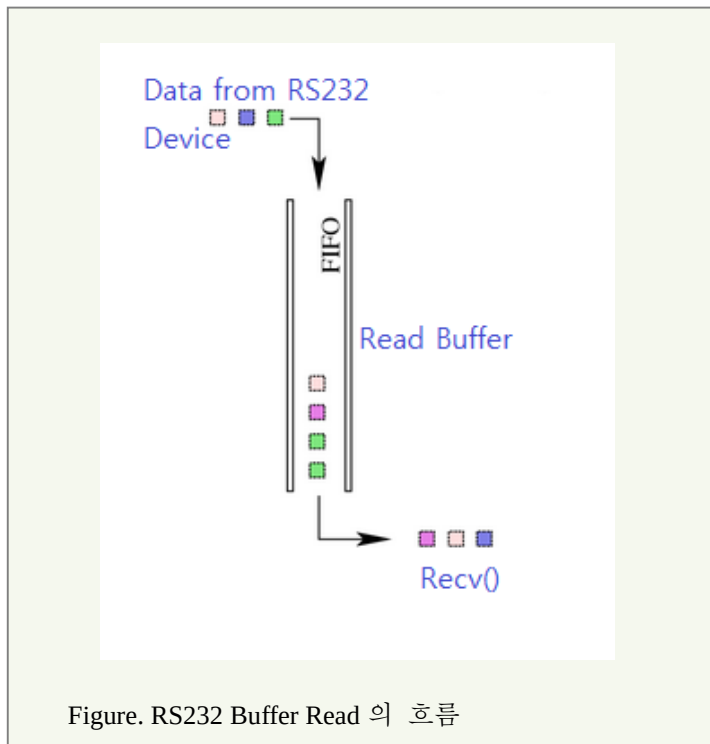
- Prototype : BOOL Send(char *Buffer , int Count);
- 역 할 : Parameter 에 정한 개수만큼 RS232 Port 에 Writing 한다.
- Parameter 의 의미

Parameter	형식	설 명
Buffer	char *	- Writing 하고자 하는 Data 의 Pointer
Count	Integer	- Writing 하고자 하는 Data 의 개수

- Return : Port 에 정상적으로 Writing 되었을 때는 TRUE Writing 되지 않았을 때는 FALSE

■ Read Part 의 역할

- RS232 Port 에 도착한 Data 를 Reading 할 때 사용된다.
- ClearBuffer() / Recv() 2 개의 Function 을 제공한다.
- RS232 Port 에 도착한 Data 는 Read Buffer 에 쌓이게 되는데 Recv() Function 을 이용하여 Buffer 의 Data 를 읽어서 활용하고, 읽어진 Data 는 Read_Buffer 에서 없어진다.
Read_Buffer 는 FIFO(First In First Out)의 구조를 갖는다.



■ Read Part - ClearBuffer()

- Prototype : void ClearBuffer();
- 역 할 : Port(Read Buffer)에 도착해 있으나 읽지 않은 Data 를 Read Buffer 로부터 지운다.
- Parameter : 없음
- Return : 없음

■ Read Part - Recv() (1st)

• Prototype : `BOOL Recv(char *Buffer ,
char *TermStr ,
int MaxCount ,
int *ReadCount,
int Timeout);`

- 역 할 : Read Buffer 의 Character 를 정해진 방식대로 읽는다. Buffer 에 저장된 Data 를 종료 인식 문자(TermStr)와 일치되는 String 을 만날 때 까지 MaxCount 만큼 읽어 들인다. 실제 읽힌 Data 의 개수는 *ReadCount 로 확인 가능하다.

FALSE Retrun 조건

- MaxCount 도달까지 종료 인식 문자(TermStr)와 일치하는 String 을 찾지 못하거나
- TimeOut(milisec)이내에 TermStr 와 일치하는 String 을 만나지 못했을 경우

TRUE Retrun 조건

- MaxCount 를 넘지 않고 Data 에서 TimeOut 시간 이내에 TermStr 과 일치되는 String 을 찾은 경우

- Parameter 의 의미

Parameter	형식	설 명
Buffer	char *	- Reading 되어진 Character Data 의 Pointer
TermStr	char *	- Character Data 의 Pointer 로서 Null 로 끝나는 String 이며 이 String 과 일치하는 String 을 만날 때 까지 Data 를 읽는다.
MaxCount	Integer	- 읽을 수 있는 최대 Character Data 의 수
ReadCount	Integer *	- 실제 읽혀진 Data 의 수를 나타내는 Pointer
TimeOut	Integer	- Data 읽기 제한 시간(Milisec 단위)

- Return : 정상적으로 읽혔을 때는 TRUE, 읽히지 않았을 때는 FALSE

■ Read Part - Recv() (2nd) - overloading

- Prototype : `BOOL Recv(char *Buffer ,
int ReqCount ,
int *ReadCount ,
int TimeOut);`
- 역 할: Read Buffer 의 Character 를 정해진 방식대로 읽는다.
Buffer 에 저장된 Data 를 ReqCount 의 개수 만큼 읽어들인다.
실제 읽힌 Data 의 개수는 *ReadCount 로 확인 가능하다.

FALSE Retrun 조건

- TimeOut(milisec)이내에 ReqCount 개수만큼 Data 를 읽지 못한 경우

TRUE Retrun 조건

- TimeOut 시간 이내에 ReqCount 의 개수 만큼 Data 를 읽은 경우

• Parameter

Parameter	형식	설 명
Buffer	char *	- Reading 되어진 Character Data 의 Pointer
ReqCount	Integer	- 읽기를 원하는 Character Data 의 수
ReadCount	Integer *	- 실제 읽혀진 Data 의 수를 나타내는 Pointer
TimeOut	Integer	- Data 읽기 제한 시간(Milisec 단위)

• Return

정상적으로 읽혔을 때는 TRUE 읽히지 않았을 때는 FALSE

Note

Recv() 함수는 함수 오버로딩으로 작성되었다.
Recv() 1st 함수는 TermStr 을 기준으로 읽어들이고
Recv() 2nd 함수는 Request Count 를 기준으로 읽어들이는 차이점을 가진다.
Read Buffer 의 Size 는 2048 Byte 의 제한이 있다.

■ Log Part 의 역할

- RS232 통신 중에 주고 받는 Data 의 내용을 저장하고자 할 때 사용한다.

■ Log Part - StartLog()

- Prototype : void StartLog(char *FileName, BOOL Bmode);
- 역 할 : Logging File 이름과 저장 형식을 지정하며 , Logging 을 시작한다.
- Parameter

Parameter	형식	설 명
Bmode	BOOL *	- False : Binary Type - True : Ascii Type
FileName	char *	- 저장할 File Name 을 지정

- Return : 없음

■ Log Part - StopLog()

- Prototype : void StopLog();
- 역 할 : Logging 의 진행을 멈춘다.
- Parameter : 없음
- Return : 없음

Chapter 13

Winsock Device Driver

What's In this chapter...

- ♦ Winsock Device Driver 관련 File
- ♦ Setup Part 의 역할
- ♦ Setup Part – TP2PSocket() / Initialize() / Connect(), CleanUp() / Disconnect()
- ♦ Write Part 의 역할
- ♦ Write Part - Send()
- ♦ Read Part 의 역할
- ♦ Read Part - ClearBuffer()
- ♦ Read Part - Recv() (1st)
- ♦ Read Part - Recv() (2nd) – overloading
- ♦ Status Part 의 역할
- ♦ Status Part - GetConnectionState(), GetLastError()
- ♦ Log Part 의 역할
- ♦ Log Part - StartLog() / StopLog()

Winsock Device Driver 관련 File

- Winsock 통신 방식 Device 의 IO Handling Driver 제작시 Interface 를 위해 Winsock Interface Library 를 제공한다.
- 관련 Library file : TDrvIF.lib, TSeqIF.lib, TP2PSock.lib
- 관련 Header file : TDrvInterface.h, TSeqIF.h, TP2PSocket.h
- TP2PSocket.h(Header File)는 아래와 같이 Interface Driver 작성을 위한 Class 을 제공한다.

HEADER File

```
#ifndef __T_P2P_SOCKET_H_
#define __T_P2P_SOCKET_H_
class DLL_INTERFACE TP2PSocket : public TComBase {
protected:
    bool m_bActiveEntity;
    SOCKET m_hSocket , m_hTargetSocket;
    struct sockaddr_in m_nServerAddr , m_nTargetAddr;
    int m_nTargetAddrLen;
    char m_nStreamType;
    char* m_pszRemoteAddr;
    short m_nServerPort;
```

```

public:
    TP2PSocket( const char* szRemoteAddr , short nServerPort , bool bStreamModeTCP=true ,
                bool bActiveMode=true/*client*/ );
    TP2PSocket( const wchar_t* szRemoteAddr , short nServerPort , bool bStreamModeTCP=true ,
                bool bActiveMode=true/*client*/ );
    TP2PSocket( TComHandler *pcComHandler , const char* szRemoteAddr , short nServerPort ,
                bool bStreamModeTCP=true , bool bActiveMode=true/*client*/ );
    TP2PSocket( TComHandler *pcComHandler , const wchar_t* szRemoteAddr , short nServerPort ,
                bool bStreamModeTCP=true , bool bActiveMode=true/*client*/ );
    virtual ~TP2PSocket();

public:
    bool Initialize();
    bool CleanUp();
    bool Connect();
    bool Disconnect();
    bool Send( const char* szSendData , int nNumOfBytesToSend );
    bool Recv( char* szRecvBuf , const char* szDelimiter , int nNumOfByteToRecv ,
               int* nNumOfBytesTransferred , int TimeOut );
    bool Recv( char* szRecvBuf , int nNumOfByteToRecv , int *nNumOfBytesTransferred ,
               int TimeOut );
    void ClearBuffer();

public:
    long GetConnectionState() const;
    long GetLastError() const;
    void StartLog( const char* szLogFile , bool fAscii );
    void StopLog();

protected:
    void SetConnectionState( long nConnectionState );
    unsigned ThreadHandlerStartPoint( TThread* pcThreadIdentity , void* pParam );

public:
    void SetConnectionRetry( unsigned long nRetryTime=0 );
    void SetLinkTest( unsigned long nInterval=0 );

protected:
    unsigned NetRecvHandler( void *pParam );
    unsigned PhysicalLinkCheck( void *pParam );

};
#endif

```


• Parameter 의 의미

Parameter	형식	설 명
Host_Addres	char	- 열고자 하는 Computer 의 TCP/IP Address (ex : 192.1.127.101)
Port_No	Integer	- 열고자 하는 Port 번호
ModeTCP	BOOL	- UDP or TCP 결정 - TRUE 일 때는 TCP 로 FALSE 일 때는 UDP 로 사용
Client	BOOL	- Server or Client 결정 - TRUE 일 때는 Client 로 FALSE 일 때는 Server 로 사용

• Return – Initialize(), Connect()

Port 가 정상적으로 열렸을 때는 TRUE 열리지 않았을 때는 FALSE

Setup Part - CleanUp() / Disconnect()

- Prototype : bool CleanUp(); bool Disconnect();
- 역 할 : 열려 있는 Port 를 닫는다.
- Parameter : 없음
- Return : 없음

Write Part 의 역할

- Network Port 에 Data 를 Writing 하기를 원할 때 사용된다.
- Send() Function 을 제공한다.

Write Part - Send()

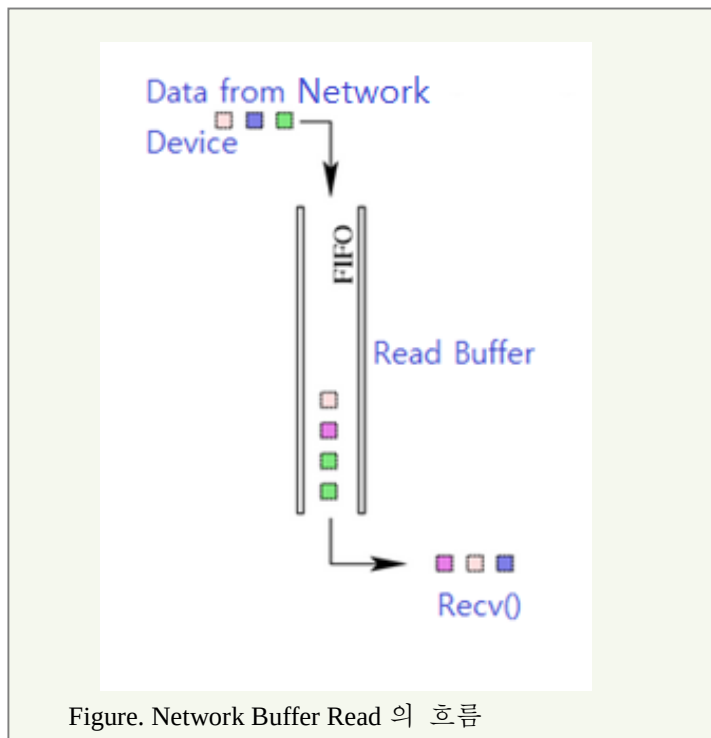
- Prototype : BOOL Send(char *Buffer , int Count);
- 역 할 : Parameter 에 정한 개수만큼 Network Port 에 Writing 한다.
- Parameter 의 의미

Parameter	형식	설 명
Buffer	char *	- Writing 하고자 하는 Data 의 Pointer
Count	Integer	- Writing 하고자 하는 Data 의 개수

- Return : Port 에 정상적으로 Writing 되었을 때는 TRUE
Writing 되지 않았을 때는 FALSE

Read Part 의 역할

- Network Port 에 도착한 Data 를 Reading 할 때 사용된다.
- ClearBuffer() / Recv() 2 개의 Function 을 제공한다.
- Network Port 에 도착한 Data 는 Read Buffer 에 쌓이게 되는데 Recv() Function 을 이용하여 Buffer 의 Data 를 읽어서 활용하고, 읽어진 Data 는 Read_Buffer 에서 없어진다.
Read_Buffer 는 FIFO(First In First Out)의 구조를 갖는다.



Read Part - ClearBuffer()

- Prototype : void ClearBuffer();
- 역할 : Port(Read Buffer)에 도착해 있으나 읽지 않은 Data 를 Read Buffer 로부터 지운다.
- Parameter : 없음
- Return : 없음

■ Read Part - Recv() (1st)

• Prototype : `BOOL Recv(char *Buffer ,
char *TermStr ,
int MaxCount ,
int *ReadCount,
int Timeout);`

- 역 할 : Read Buffer 의 Character 를 정해진 방식대로 읽는다. Buffer 에 저장된 Data 를 종료 인식 문자(TermStr)와 일치되는 String 을 만날 때 까지 MaxCount 만큼 읽어 들인다. 실제 읽힌 Data 의 개수는 *ReadCount 로 확인 가능하다.

FALSE Retrun 조건

- MaxCount 도달까지 종료 인식 문자(TermStr)와 일치하는 String 을 찾지 못하거나
- TimeOut(milisec)이내에 TermStr 와 일치하는 String 을 만나지 못했을 경우

TRUE Retrun 조건

- MaxCount 를 넘지 않고 Data 에서 TimeOut 시간 이내에 TermStr 과 일치되는 String 을 찾은 경우

- Parameter 의 의미

Parameter	형식	설 명
Buffer	char *	- Reading 되어진 Character Data 의 Pointer
TermStr	char *	- Character Data 의 Pointer 로서 Null 로 끝나는 String 이며 이 String 과 일치하는 String 을 만날 때 까지 Data 를 읽는다.
MaxCount	Integer	- 읽을 수 있는 최대 Character Data 의 수
ReadCount	Integer *	- 실제 읽혀진 Data 의 수를 나타내는 Pointer
TimeOut	Integer	- Data 읽기 제한 시간(Milisec 단위)

- Return : 정상적으로 읽혔을 때는 TRUE, 읽히지 않았을 때는 FALSE

■ Read Part - Recv() (2nd) - overloading

- Prototype : `BOOL Recv(char *Buffer ,
int ReqCount ,
int *ReadCount ,
int TimeOut);`
- 역 할: Read Buffer 의 Character 를 정해진 방식대로 읽는다.
Buffer 에 저장된 Data 를 ReqCount 의 개수 만큼 읽어들인다.
실제 읽힌 Data 의 개수는 *ReadCount 로 확인 가능하다.

FALSE Retrun 조건

- TimeOut(milisec)이내에 ReqCount 개수만큼 Data 를 읽지 못한 경우

TRUE Retrun 조건

- TimeOut 시간 이내에 ReqCount 의 개수 만큼 Data 를 읽은 경우

• Parameter

Parameter	형식	설 명
Buffer	char *	- Reading 되어진 Character Data 의 Pointer
ReqCount	Integer	- 읽기를 원하는 Character Data 의 수
ReadCount	Integer *	- 실제 읽혀진 Data 의 수를 나타내는 Pointer
TimeOut	Integer	- Data 읽기 제한 시간(Milisec 단위)

• Return

정상적으로 읽혔을 때는 TRUE 읽히지 않았을 때는 FALSE

Note

Recv() 함수는 함수 오버로딩으로 작성되었다.
Recv() 1st 함수는 TermStr 을 기준으로 읽어들이고
Recv() 2nd 함수는 Request Count 를 기준으로 읽어들이는 차이점을 가진다.
Read Buffer 의 Size 는 2048 Byte 의 제한이 있다.

■ Staus Part 의 역할

- Network 의 연결상태 및 연결에 사용된 Parameter 를 보여준다.
- GetConnectionState 의 Function 을 제공한다.

■ Staus Part - GetConnectionState()

- Prototype : long GetConnectionState();
- 역 할 : Network 의 연결 상태를 알 수 있다.
- Parameter : 없음
- Return : Port 가 정상적으로 연결되었을 때는 TRUE 연결되어 있지 않았을 때는 FALSE

■ Staus Part - GetLastError()

- Prototype : long GetLastError();
- 역 할 : Port 에 발생한 마지막 error 를 알 수 있다.
- Parameter : 없음
- Return : 마지막 error

■ Log Part 의 역할

- TCP/IP 통신 중의 주고 받는 Data 의 내용을 저장하고자 할 때 사용한다.

■ Log Part - StartLog()

- Prototype : void StartLog(char *FileName, BOOL Bmode);
- 역 할 : Logging File 이름과 저장 형식을 지정하며 , Logging 을 시작한다.
- Parameter

Parameter	형식	설 명
Bmode	BOOL *	- False : Binary Type - True : Ascii Type
FileName	char *	- 저장할 File Name 을 지정

- Return : 없음

■ Log Part - StopLog()

- Prototype : void StopLog();
- 역 할 : Logging 의 진행을 멈춘다.
- Parameter : 없음
- Return : 없음

Utility

Chapter 14: FEST Control Window

Chapter 15: Alarm Manager

Chapter 16: Chart Manager

Chapter 17: Job Log Manager

Chapter 18: ETC Log Viewer

Chapter 19: Online Log Viewer

Chapter 14

FEST Control Window

What's In this chapter...

- ♦ **FEST Control Window**
- ♦ **IO - IO view**
- ♦ **Sequence - Control View / Make Template**
- ♦ **Alarm - Alarm Test / Alarm View**
- ♦ **Account - User Account Manager**

FEST Control Window

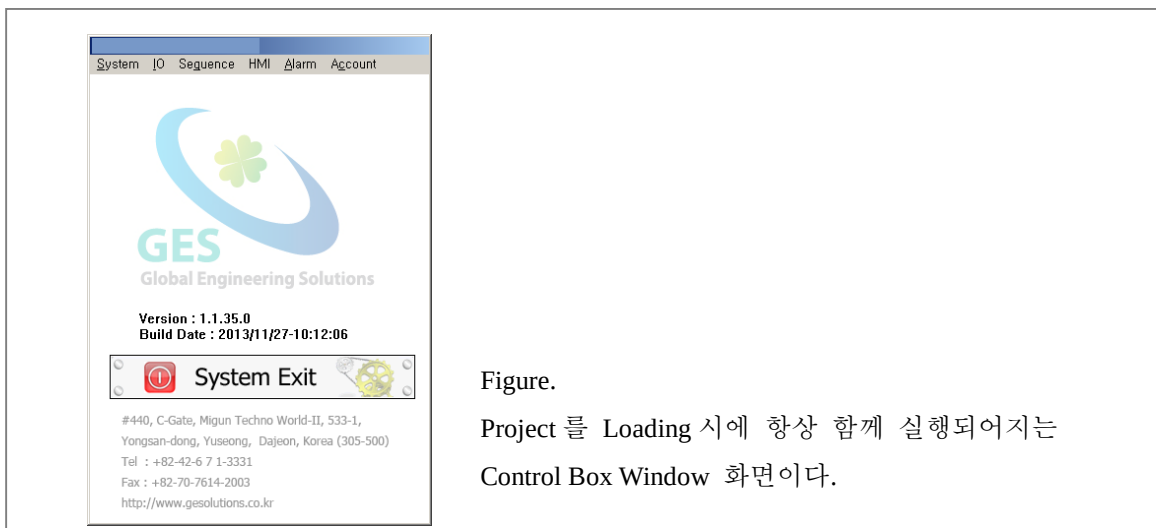


Figure.

Project 를 Loading 시에 항상 함께 실행되어지는 Control Box Window 화면이다.

Name		Contents	Etc
FEST_CTC_DEMO		StartUp 에서 설정한 System Name 이 표시된다	
System	Console Clear	Dos Consol message 를 clear	
	Console Hide	Dos Consol message Window 를 숨긴다.	
	Exit	System Exit	
IO	IO View	Project 에 등록된 모든 IO 를 조회 및 Setting 가능하다. (System IO 포함)	
Sequence	Control View	사용된 모든 Sequence 의 Status/Command/Parameter/Argument 등을 확인 가능하며, Sequence Kill/Load/Run/Event/Abort 등의 Action 도 가능하다.	
	Make Template	User 가 등록된 IO, Alarm, Sequence 들을 검색하여 Sequence Template 를 작성할 수 있다.	
HMI	Hide All / Show All	GUI Screen 을 Hide / Show toggle 한다.	
	Select Monitor	Dual Monitor 환경인 경우 Monitor 1 / 2 로 GUI 화면을 이동시킬 수 있다.	
Alarm	Alarm Test	Alarm Search/Post/Clear 등을 테스트 가능하다.	
	Alarm View	등록된 Alarm 을 조회 가능한 화면이 display 된다.	
Account	User Account Manager	User 및 Group 을 등록/수정/삭제 가능한 화면이 display 된다	
System Exit...		System 을 종료한다.	

IO -> IO view

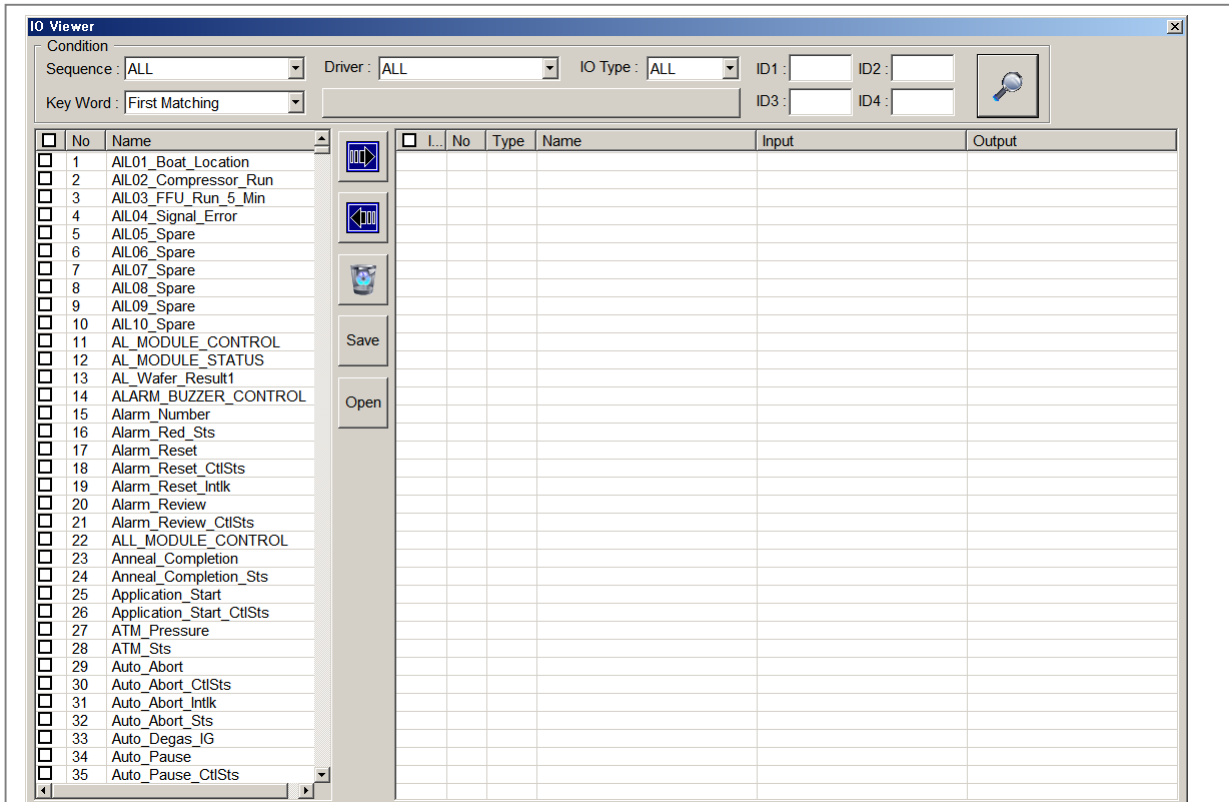


Figure .

FEST Control Window 의 Menu 중 [IO --> IO View] 버튼을 Click 하여 실행시킨 화면

- IO Viewer 는 현재 load 된 IO 의 상태를 확인하거나 직접 Control 할 수 있는 Interface 를 제공한다.
- 검색 조건으로 Sequence/Driver/IO Type/Key Word/IO Name 을 and 조건으로 검색 가능하다.
- 조작하고자 하는 IO 의 Output 항목을 click 하면 다음과 같은 Setting Dialog Box 가 나타난다.

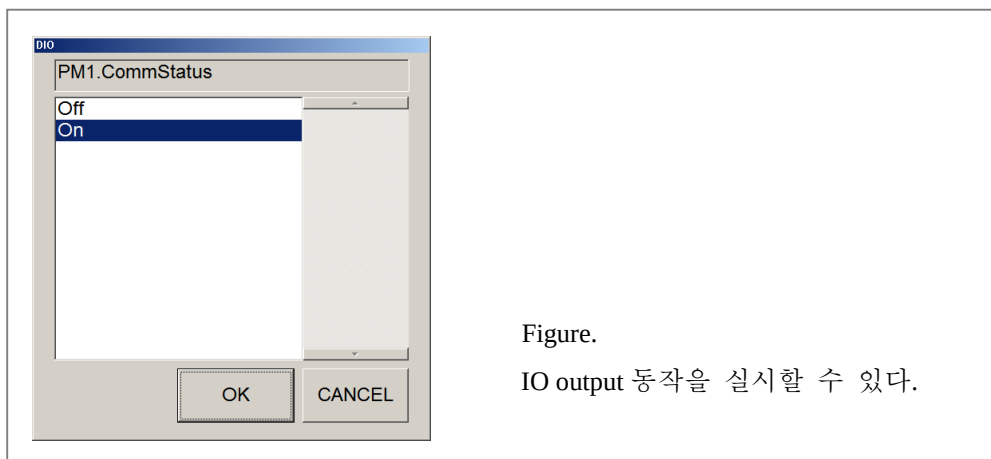
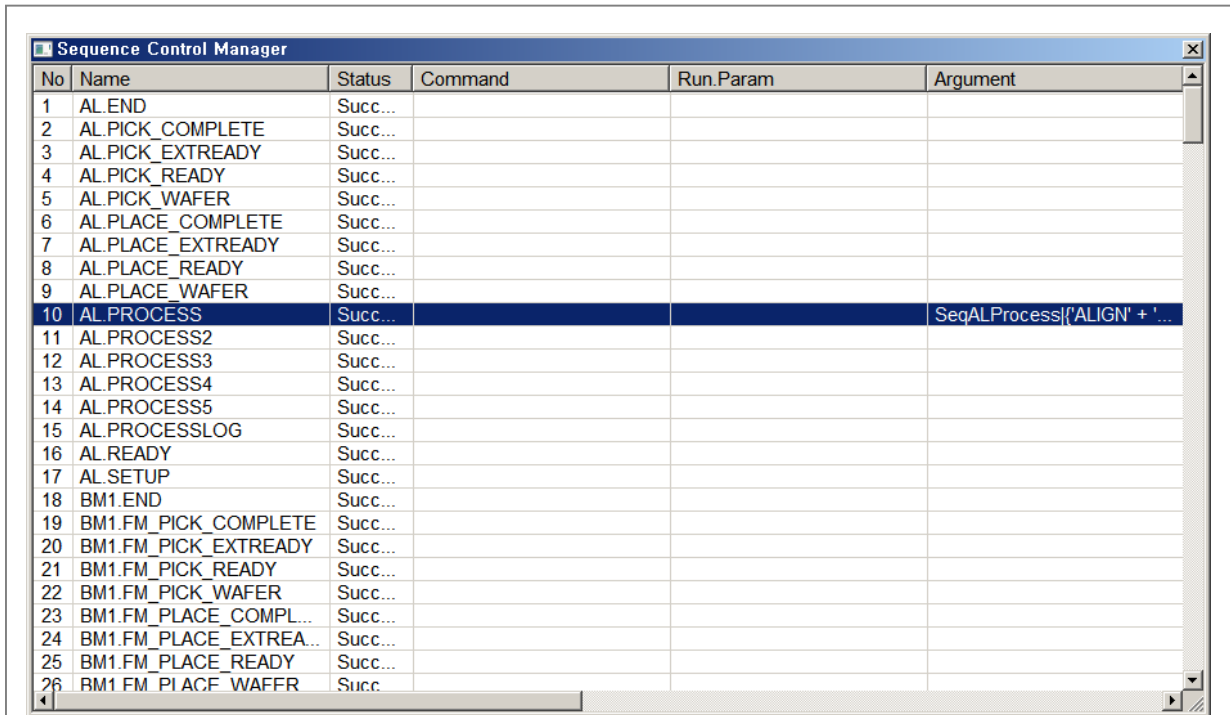


Figure.

IO output 동작을 실시할 수 있다.

Sequence -> Control View



No	Name	Status	Command	Run.Param	Argument
1	AL.END	Succ...			
2	AL.PICK_COMPLETE	Succ...			
3	AL.PICK_EXTREADY	Succ...			
4	AL.PICK_READY	Succ...			
5	AL.PICK_WAFER	Succ...			
6	AL.PLACE_COMPLETE	Succ...			
7	AL.PLACE_EXTREADY	Succ...			
8	AL.PLACE_READY	Succ...			
9	AL.PLACE_WAFER	Succ...			
10	AL.PROCESS	Succ...			SeqALProcess['ALIGN' + '...
11	AL.PROCESS2	Succ...			
12	AL.PROCESS3	Succ...			
13	AL.PROCESS4	Succ...			
14	AL.PROCESS5	Succ...			
15	AL.PROCESSLOG	Succ...			
16	AL.READY	Succ...			
17	AL.SETUP	Succ...			
18	BM1.END	Succ...			
19	BM1.FM_PICK_COMPLETE	Succ...			
20	BM1.FM_PICK_EXTREADY	Succ...			
21	BM1.FM_PICK_READY	Succ...			
22	BM1.FM_PICK_WAFER	Succ...			
23	BM1.FM_PLACE_COMPL...	Succ...			
24	BM1.FM_PLACE_EXTREA...	Succ...			
25	BM1.FM_PLACE_READY	Succ...			
26	BM1.FM_PLACE_WAFER	Succ...			

Figure.

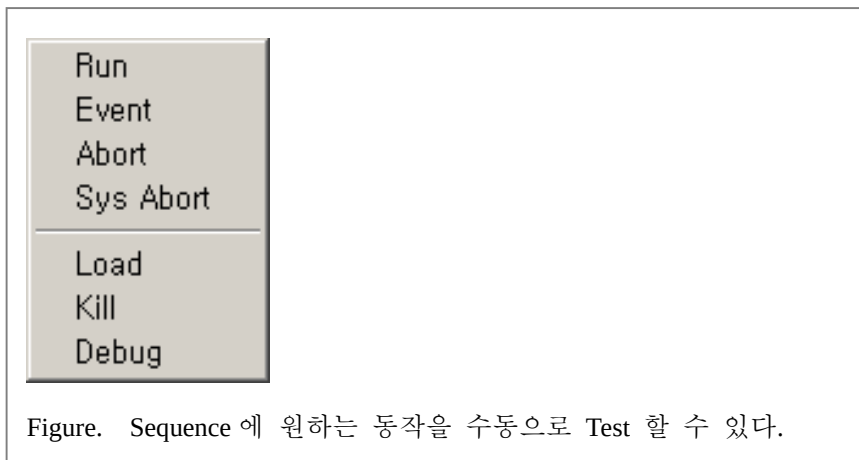
FEST Control Window 의 Menu 중 [Sequence --> Control View] 버튼을 Click 하여 실행시킨 화면

- Sequence Control View load 된 Sequence 목록을 확인 또는 Control 할 수 있는 Interface 를 제공한다.

Column Name	내 용	Etc
No	Sequence 일련번호	
Name	Sequence Name.	
Status	Sequence 의 Status 를 나타내는데, Success, Unknown, Abort, Running, Unknown 이 display 된다. - Success : Sequence 의 실행결과가 Success 이다. - Fail : Sequence 의 실행결과가 Fail 이다. - Aborted : Sequence 의 실행결과가 Aborted 이다. - Running : Sequence 가 현재 실행중이다. - Unknown : Sequence 가 Loading 실패하거나 Kill 된 경우	
Command	Sequence Loading 시에 Option Parameter 정보이다.	
Run.Param	마지막으로 Sequence 에 전달된 Parameter 이다.	
Argument	초기 실행 시 Sequence 에 전달된 Argument 이다.	
EventData	마지막으로 Sequence 에 전달된 Event Data 이다.	
EventData	마지막으로 Sequence 에 전달된 Event Parameter 이다.	
Path	Sequence 의 실제 경로이다. (%Virtual%은 선언시 SUCCESS 한 Sequence 를 의미)	
Version	실행 File 에 사용된 Interface Library Version Data	
Build Date	Sequence 의 최종 Build date	

Build Time	Sequence 의 최종 Build Time	
------------	--------------------------	--

- Control 해보고자 하는 Sequence 이름위에서 Mouse double click 시 아래와 같은 Control 명령 이 표시된다.



Name	내 용	etc
Run	Sequence 을 실행시킨다. Click 하면 아래 그림(Figure 32-3.)과 같은 Dialog Box 가 나타나는데 Sequence 에 Parameter 를 전달할 수 있다. (SEQ_STATUS SeqMain()으로 전달된다.)	Figure 32-3.
Event	Sequence 에 Event 를 전달한다. . Click 하면 아래 그림(Figure 32-3.)과 같은 Dialog Box 가 나타나는데 Sequence 에 Parameter 를 전달할 수 있다. (void OnEventRecv()으로 전달된다.)	Figure 32-3.
Abort	선택되어진 해당 Sequence 에 Abort signal 을 전달한다.	
Abort	모든 Sequence 에 Abort signal 을 전달한다.	
Load	Sequence 가 Unknow 상태(Kill 되었거나 Loading Fail 인 경우) 일 때 재 Load 시도를 한다. (Project Loading 시의 Sequence 초기 Loading 과 같은 동작)	
Kill	Sequence 를 Engine 에서 연결을 끊고 종료 시킨다.	
Debug	Debug Mode 로 Sequence 를 Loading 한다.	

Figure .

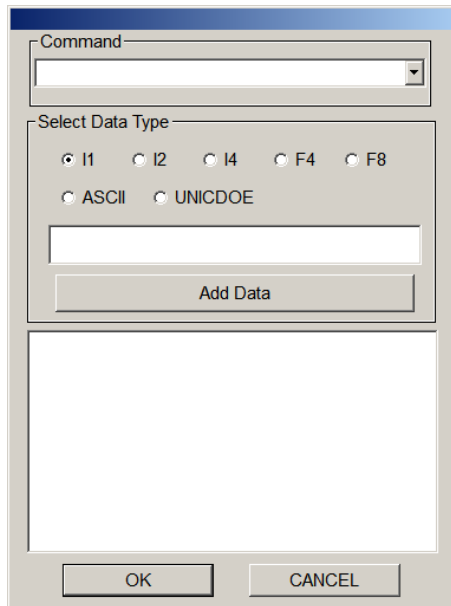
Sequence 에 Run 또는 Event 동작 수행시 표시되는 Dialog Box

- Command

- Sequence 에 전달된 Run Message, 10 개까지 이력이 보존된다.

- Select Data Type

- Data Type 을 선택, 값을 입력 후 Add Data 를 한 후 OK 를 Click 하면 실행된다.



Sequence -> Make Template

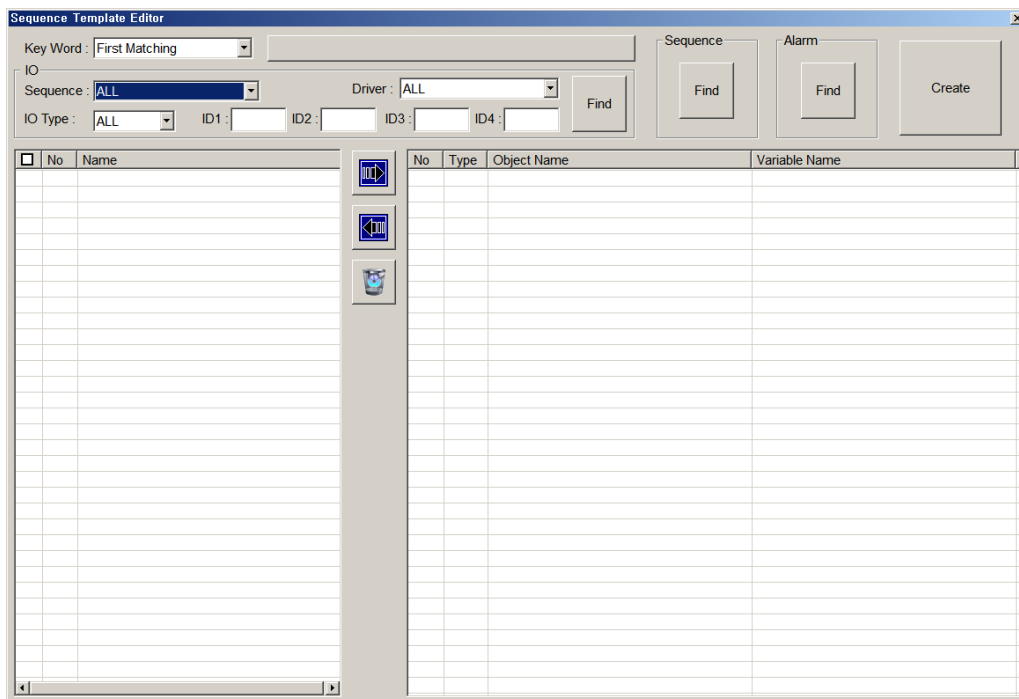


Figure.

Sequence 를 User 가 선택한 항목을 바탕으로 생성해주는 Template Maker 화면이다.

- User 편의를 위하여 Sequence Template Editor 를 제공하고 있다.

- 작성 방법은

Sequence 에 등록하고자 하는

IO 검색 후 , Sequence 검색 후 , Alarm 검색 후  를 Click 하여 오른쪽 list 에 목록을 추가 후에 Create Button 으로 Sequence File 을 생성할 수 있다.

Note

작성되는 파일은 `?.C` 또는 `?.Cpp` 파일로 저장할 수 있으며

이것을 작성하고자 하는 **Sequence**의 **WorkSpace**에 복사하여 사용하면 된다.

이 기능은 **Sequence**를 작성하는 데 도움을 하는 기능이지만, 이미 사용할 **Object** 목록을 알고 있다면 기존의 다른 **Sequece**를 **Rename**하여 작성하는 것이 편하다.

Alarm -> Alarm Test / Alarm View

• Alarm Test

System 에 정의 되어 있는 Alarm 을 검색하여 Alarm Post & Clear 할 수 있다.

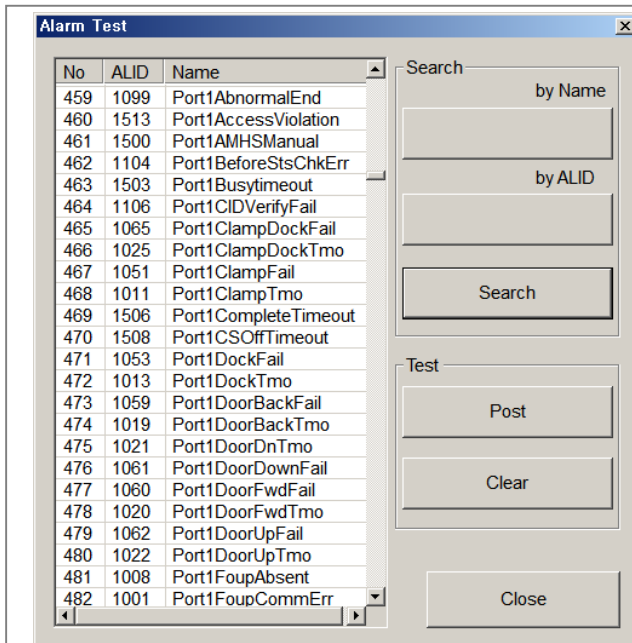


Figure . Alarm Test Dialog

• Alarm View

System 에 정의 되어 있는 Alarm 을 문자열 keyword 로 검색할 수 있다.

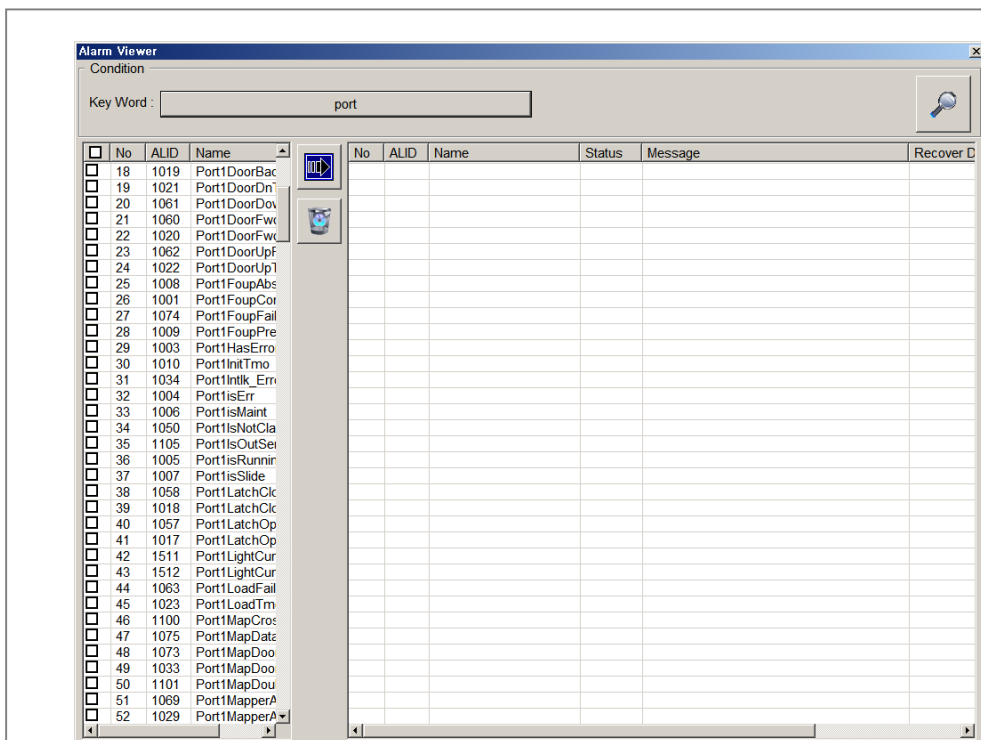


Figure . Alarm View Dialog

Account -> User Account Manager

- Group & User 을 등록/수정/삭제 가능하다.

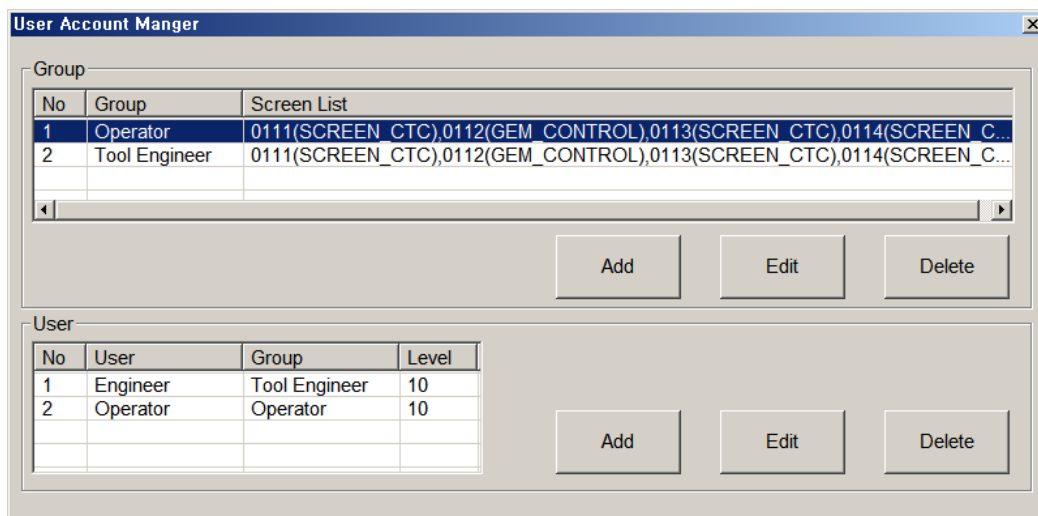


Figure . User Account Manager Dialog

Note

FEST 에서는 Group 및 User 를 추가할 때에 Windows 의 Registry 에 등록한다.

Chapter 15

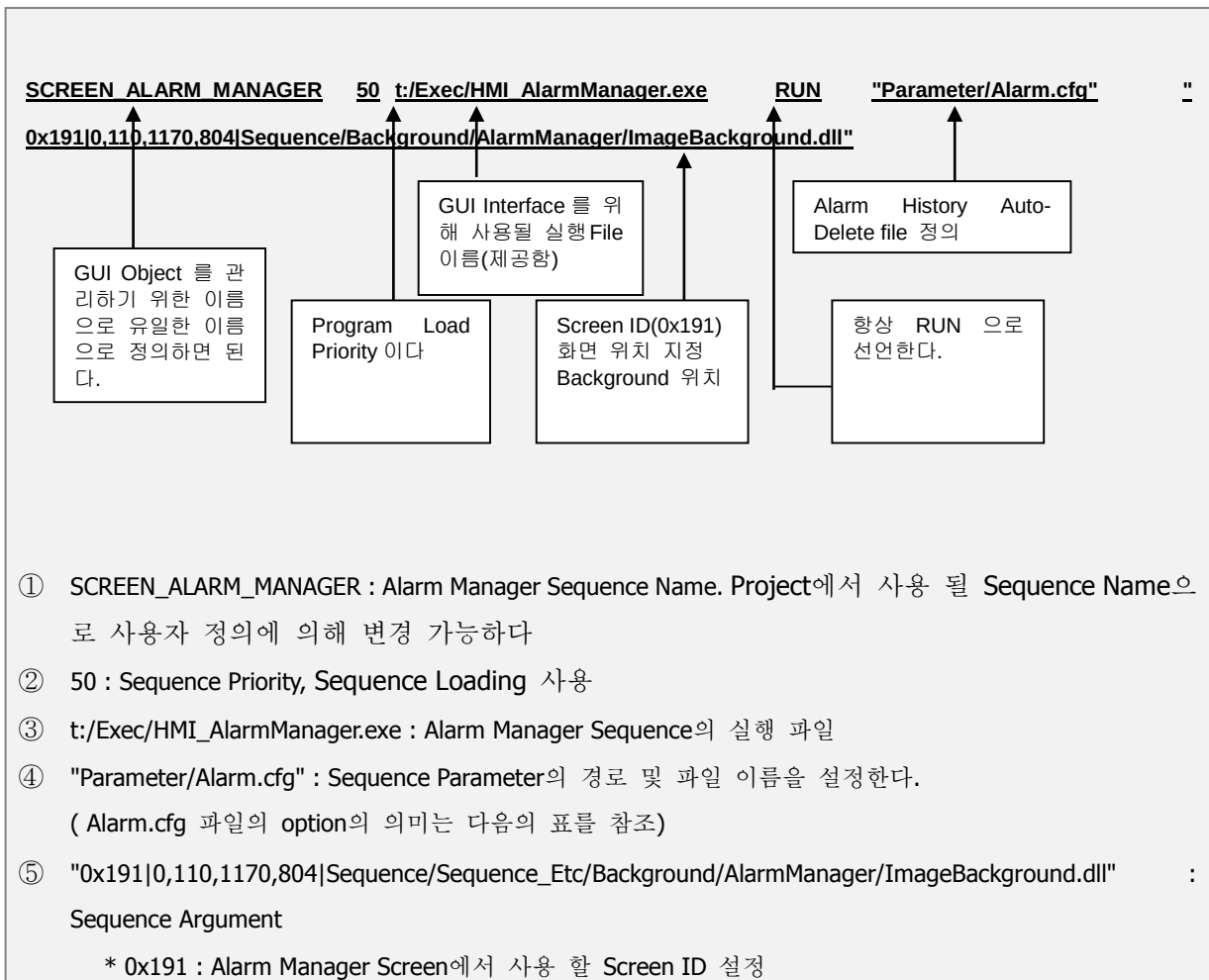
Alarm Manager

What's In this chapter...

- ◆ Alarm Manager 정의
- ◆ Operation

Alarm Manger 정의

- Alarm 발생시 발생한 Alarm 의 정보, Alarm Recovery Action 을 하기 위해 사용되는 GUI 용 Sequence 이다.
- S_GUI.list (User 정의에 따라 다를 수 있음) 파일등에 아래와 같이 Alarm Manager 를 등록한다.



• Alarm.cfg 내의 Item 의 의미

Item	Data	Description
AUTO_DELETE_MONTY	1~12	단위는 Month 이며, History File 자동 Delete 기능을 수행한다.
ALM_LIST_VIEW_STYLE	0 (Happened)	Alarm 발생 순서대로 List View 에 Display 된다.
	1 (Newest)	최근에 발생한 Alarm 이 List View 상단에 Display 된다.
ALM_ALLRECOVERY_USE	0 (Not Use)	All Recovery / Button 을 사용 할 수 없다
	1 (Only Clear)	CLEAR Recovery Item 만 가지는 Alarm 만 Auto Recovery 가 이루어 진다.
	2 (One Item)	1 개의 Recovery Item 만 가지는 Alarm 만 해당 Recovery Item 으로 Auto Recovery 한다.
	3 (All Recovery)	Auto Recovery Priority 에 설정된 Recovery Item 순서대 (1~5)로 Auto Recovery 한다. 아래 ALM_RECOVERY_PRIORITY1 ~ PRIORITY5 참조.
ALM_RECOVERY_PRIORITY1 ~ ALM_RECOVERY_PRIORITY5	0 (Not Use /Skip)	Skip / 사용 안함.
	1 (ABORT)	Recovery List 에 ALM_ABORT 가 있으면 ABORT 로 Auto Recovery, 없으면 Next Priority 를 사용한다.
	2 (RETRY)	Recovery List 에 ALM_RETRY 가 있으면 RETRY 로 Auto Recovery, 없으면 Next Priority 를 사용한다.
	3 (IGNORE)	Recovery List 에 ALM_IGNORE 가 있으면 IGNORE 로 Auto Recovery, 없으면 Next Priority 를 사용한다.
	4 (ERROR)	Recovery List 에 ALM_ERROR 가 있으면 ERROR 로 Auto Recovery, 없으면 Next Priority 를 사용한다.

Note

Alarm Manager 에서는 Alarm.cfg file 을 Loading 시에 읽어들여 Alarm Manager 의 동작시에 참조하는 option 으로 활용한다.

또한 Alarm Manager Screen 에서 상기에 해당하는 Option 을 조작하여 저장할 수 있기 때문에 직접 Alarm.cfg 파일을 수정할 필요는 없다.

Operation

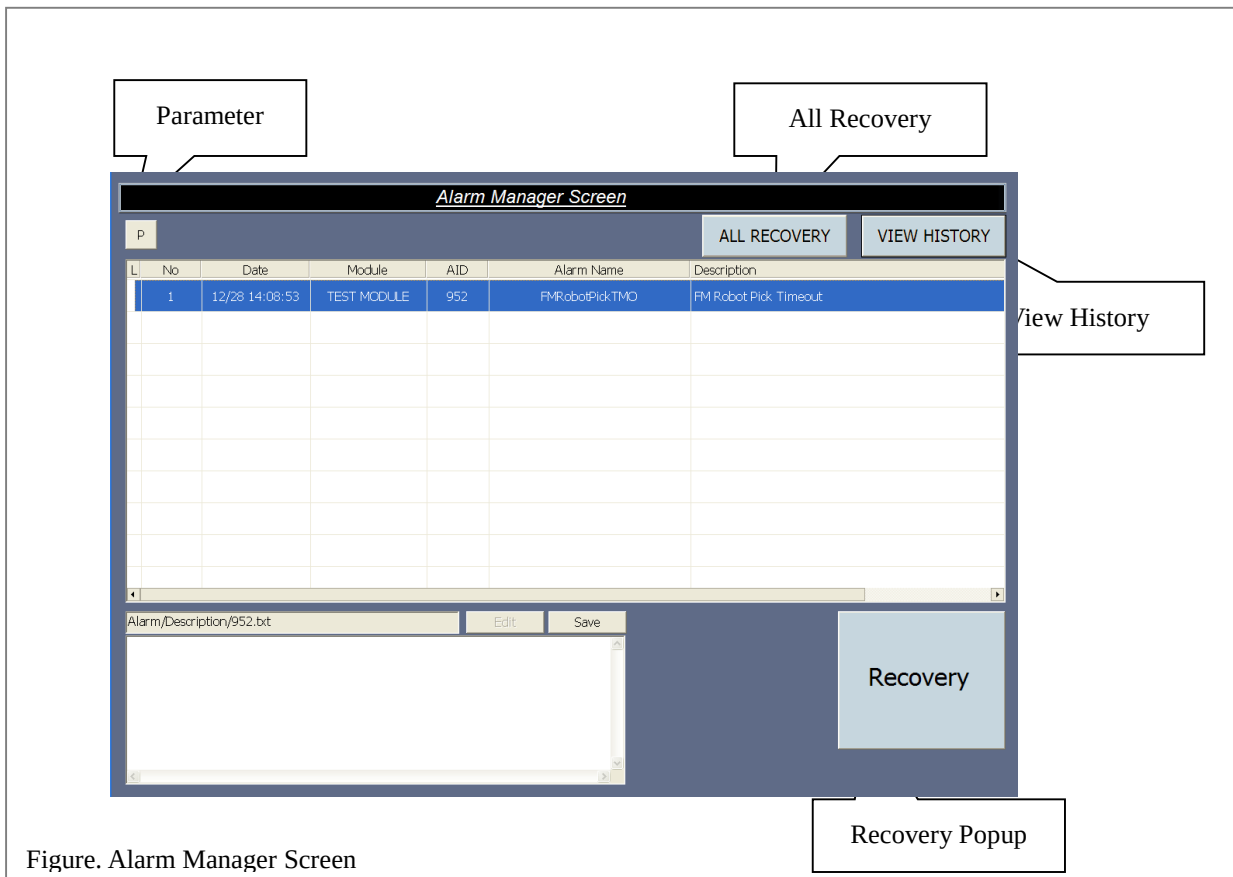


Figure. Alarm Manager Screen

• Screen Column 의 의미

Column Name	설 명
No	발생한 Alarm 의 일련번호가 매겨진다.
Date	Alarm 이 발생한 날짜와 시간을 나열한다.
Module	Alarm 이 발생한 Sequence Name 이 표시된다.
AID	발생한 Alarm 의 ID 번호가 표시된다.
Alarm Name	발생한 Alarm 의 Name 이 표시된다.
Description	Alarm 의 세부 내용이 표시된다.

• Parameter

설정된 Parameter 확인 및 변경이 가능하다.

The 'Parameter Screen' dialog box contains the following settings:

- Hisotry File Delete Period(Month): 6
- Alarm List View Style: Newest
- Auto Recovery Mode: All Recovery
- Auto Recovery Priority(All Recovery):

Priority 1	ABORT
Priority 2	RETRY
Priority 3	IGNORE
Priority 4	-
Priority 5	-

Buttons: SAVE, CANCEL

Auto Recovery Item

Figure. Parameter Screen

The 'Auto Delete' dialog box prompts the user to 'Put Auto Delete Period?'. It features a numeric keypad with buttons for digits 0-9, CLR, BackSpace, 12, and Exp. The current value is 6. Buttons for OK and CANCEL are at the bottom.

Figure . Auto Delete

- Sequence Parameter File의 AUTO_DELETE_MONTY 항목에 의해 설정된 값이다.
- 1~12개월까지 변경 가능하다.

The 'Alarm View' dialog box prompts the user to 'Select Alarm View Style'. It shows a list with 'Hpppened' and 'Newest' (selected). Buttons for OK and CANCEL are at the bottom.

Figure. Alarm View

- Parameter File의 ALM_LIST_VIEW_STYLE 항목에 의해 설정된 값이다
- Alarm 발생 시 List View에 발생 된 Alarm 순서대로 또는 최신 순서대로 볼 것인지 설정할 수 있다.

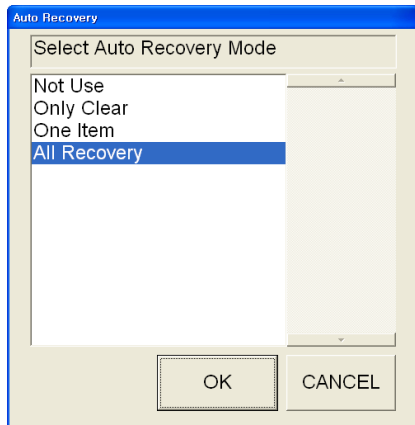


Figure . Auto Recovery

- Parameter File의 ALM_ALLRECOVERY_USE 항목에 의해 설정된 값이다
- Auto Recovery 항목에 대한 설정이 가능하다

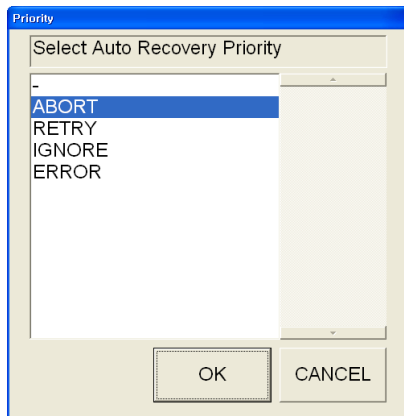


Figure . Priority

- Auto Recovery Mode가 All Recovery 일 때 사용된다.
- Priority 1 부터 순차적으로 확인하여 해당하는 Alarm Recovery가 설정 되어 있으면, 그 결과로 Alarm을 자동 Recovery한다.
- ABORT/RETRY/IGNORE/ERROR 설정이 가능하다.
(-는 사용 안함/Skip을 의미한다)

• All RecoveryParameter

Alarm Parameter Screen에 설정된 정보로 모든 발생된 모든 Alarm을 Auto Recovery를 수행한다.

• View History

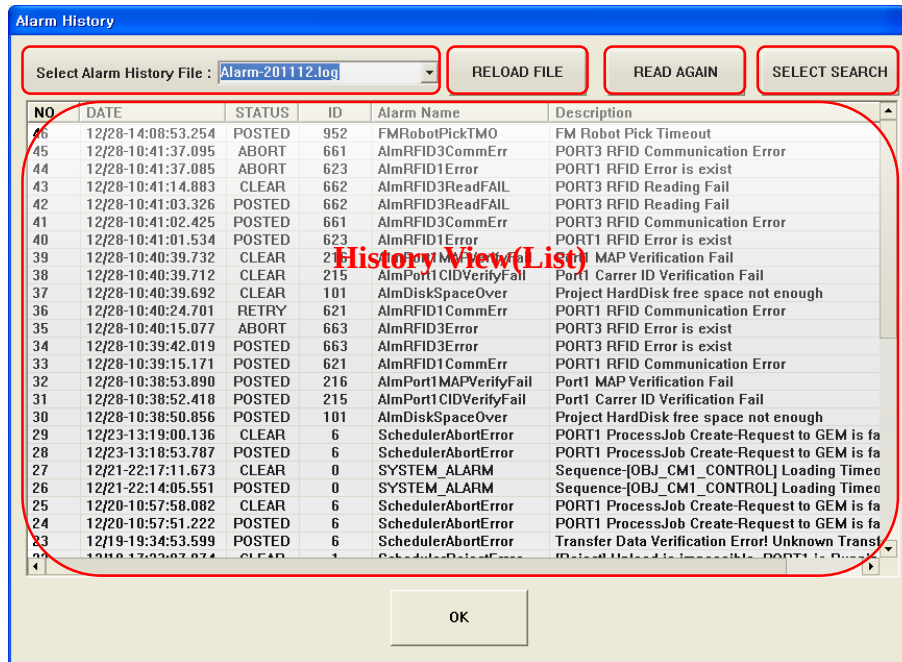


Figure . View History

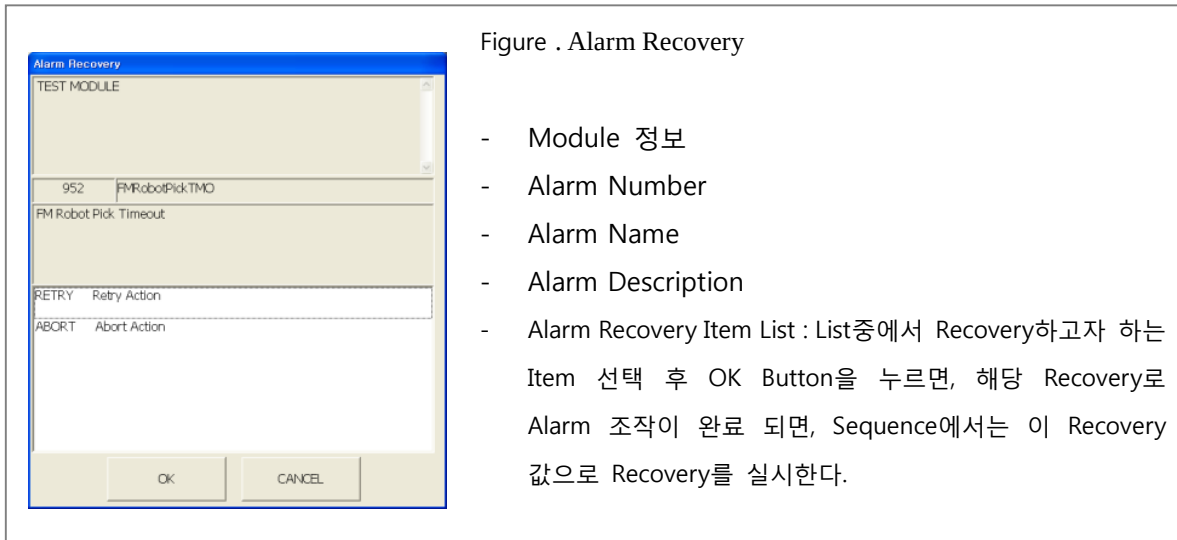
Name	설 명
Select Alarm History File	- File Path : Project의 History Folder내에 존재한다. - File Name : Alarm 발생 시 자동 생성되면, Alarm-년월.log 이름을 가지면, 월별로 History File이 생성된다. 기본적으로 현재(최근) History File이 Display된다.
RELOAD FILE	- 새로운 File 생성시 RELOAD FILE Button Click시 Combo Box에 List가 Update(추가) 된다.
READ AGAIN	- READ AGAIN Button Click 시 선택된 File 을 Read 하여 History View List 를 Update 한다.
SELECT SEARCH	- 현재 Alarm History View에서 Date / Alarm No. / Alarm Name / Alarm Type 항목으로 Search 할 수 있다. - 복수 선택 / Search가 가능하다.
History List	- 누적된 Alarm 이력 또는 검색되어진 Alarm 이력이 표시된다.

● Alarm List View

현재 발생 되어져 있는 Alarm List가 Display된다.

Alarm List View의 Alarm 선택 후에 RECOVERY Button에 의해 User Recovery를 실시한다.

Alarm Recovery



● Alarm Detail Description

Alarm List View의 발생된 Alarm 선택 시 선택된 Alarm의 Detail Information이 보여진다.

- Detail Information File 추가 방법

- 1) DESCRIPTION "Alarm#Description#Motor#1001.txt" 과 같이 적당한 이름의 File 을 준비하여 Description 을 작성해둔다.
- 2) Alarm 정의시에 1 줄 추가한다.

Example)

```
MotorError          1001      "Motor has a Error" {
    RETRY            "Retry Action"
    ABORT             "Abort Action"
    DESCRIPTION "Alarm#Description#1001.txt"
}
```

즉, DESCRIPTION 이란 Key Word 를 사용하고, Description File 을 링크한다.

- Description 파일이 정의 되지 않은 경우에는 Alarm Number 이름이 표시되면, Screen에서 직접 입력 후 저장도 가능하다. (이때 파일명은 발생한 Alarm번호.txt로 저장된다.)

Chapter 16

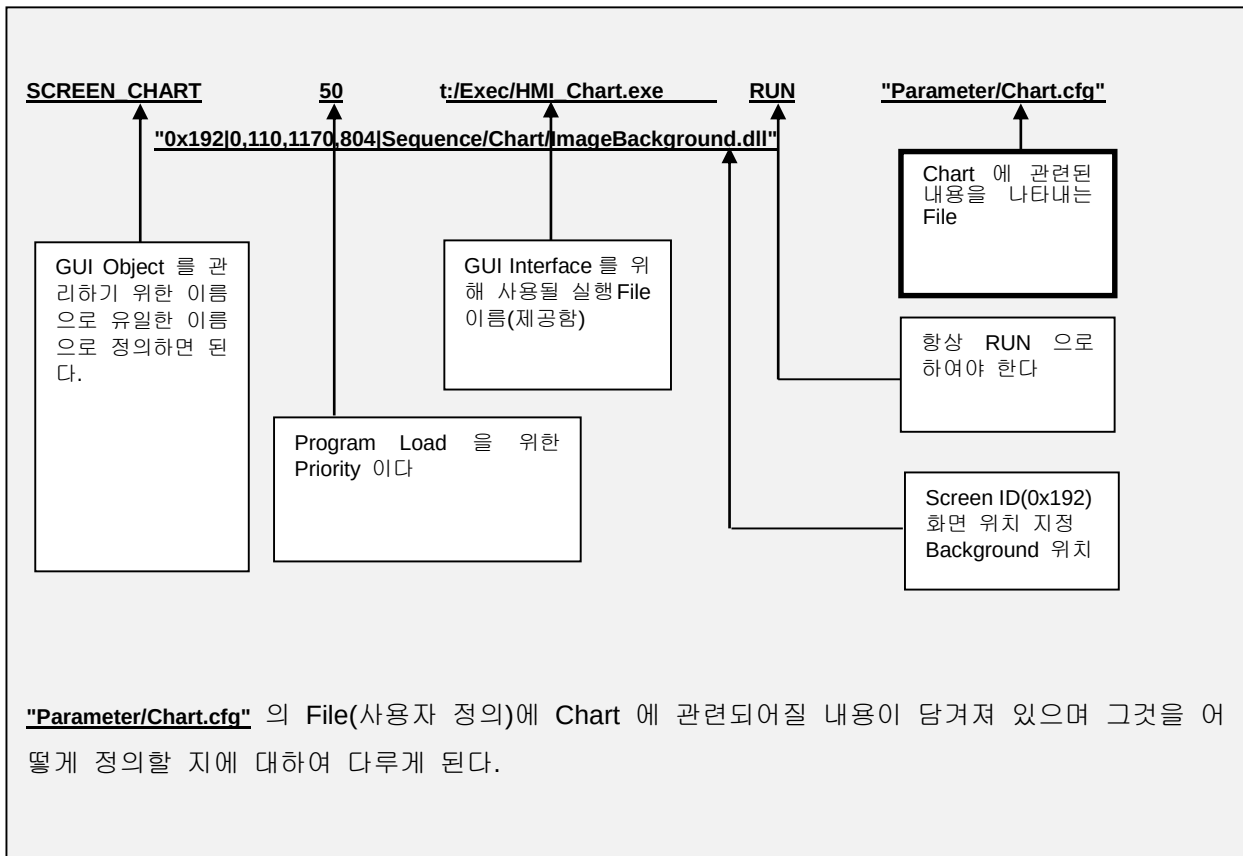
Chart Manager

What's In this chapter...

- ◆ Chart Manager 정의
- ◆ Operation

Chart Manger 정의

- User 가 지정한 IO 의 내용을 Chart 화 하기위해 사용되는 GUI 용 Sequence 이다
- S_GUI.list (User 정의에 따라 다를 수 있음) 파일등에 아래와 같이 Chart Manager 를 등록한다.



• Chart.cfg File 에 Item 의 의미

Item	Data	Description
\$DATALOG_CONTROL	CHART_LOG_CONTROL	Chart Start/Stop 시 사용될 IO.
\$DATALOG_TIME	1000	단위는 milliseconds. (Minimum 500msec ~ Maximum 5000msec)
\$DATALOG_MODE	TIMER/COUNTER	File Log 시 각 Line Header 에 Time/Counter 를 Log 할 수 있다.(Default Counter)
\$DATALOG_DIRECTORY	DATALOG/ETC/CHART	Log File 이 저장될 위치
\$IO	IO Name	Chart 에 사용 할 IO 를 정의한다. (IO 를 정의하는 방법은 아래에 설명한다.)

• IO 등록 방법 (Chart.cfg)

Analog IO만 가능하다

Example)

\$IO PM1.Heat_Temp TRUE Solid 3 (0,255,255) PM1_TEMP

Item	Data	Description
Fixed	\$IO	Define Tag.
IO Name	PM1.Heat_Temp	Chart 에 사용 할 IO 를 선언
Draw	TRUE/FALSE	Chart Screen Loading Screen 에 현재 상태(값)의 Draw 여부 결정/Screen Loading 완료 후에는 개별적 선택이 가능하다.
Line Type	Solid	Draw Line Type 을 설정
	Dash	
	Dot	
	DashDot	
	DashDotDot	
	SmallDot	
Line Width	3	Line 의 Draw 굵기를 설정
Line Color	(0,255,255)	R/G/B 에 의해 Draw Line 의 Color 설정
Display	PM1_TEMP	PM1.Heat_Temp 를 PM1_TEMP 의 title 로 표시한다. (선언 시 공백을 허용하지 않음)

Operation

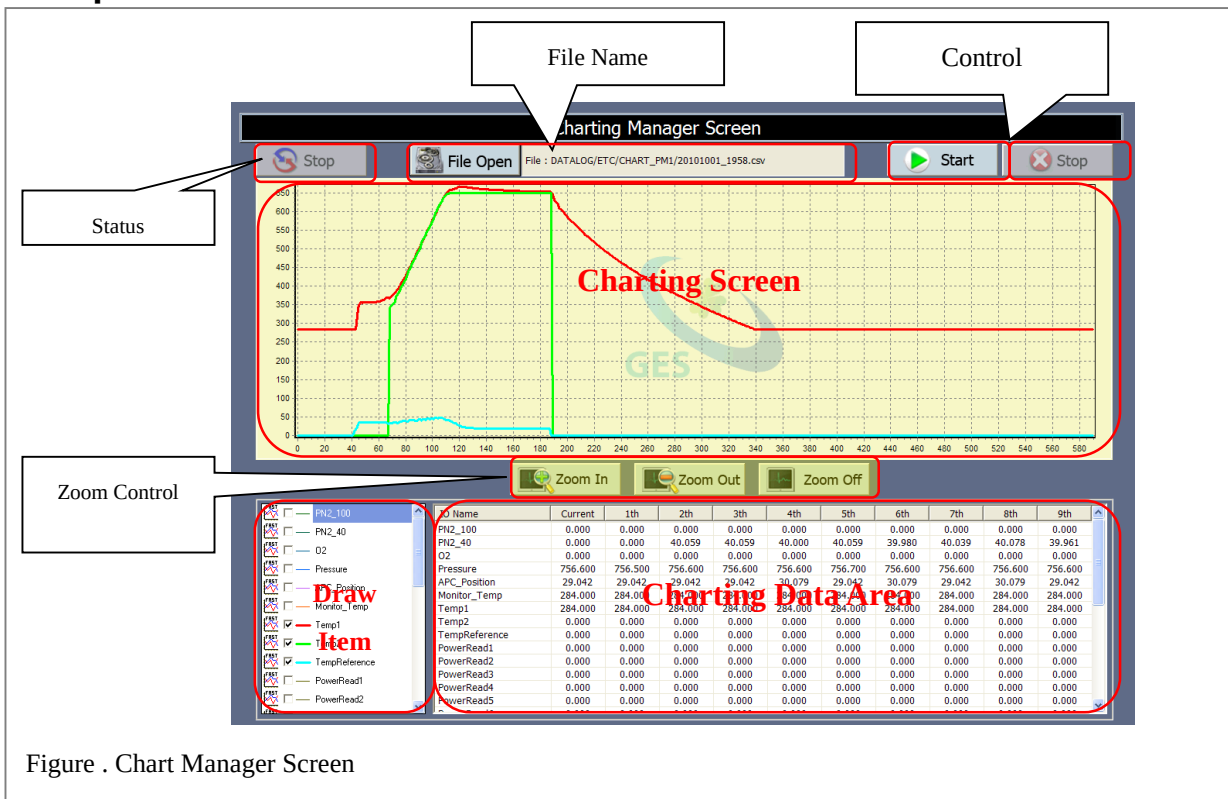


Figure . Chart Manager Screen

• 각 Item 설명

Item	설 명
Status	현재 Charting 진행 여부 표시
File Open	Charting을 진행하고 있지 않을 경우, Log File을 읽어, Charting Screen에 표시 가능하다.
File Name	File Open에 의해 열린 File / Charting에 의해 현재 Log되어지고 있는 File의 Name을 Display한다.
Start / Stop Control	- Start : Charting을 시작한다 - Stop : Charting을 멈춘다
Charting Screen	Draw Item들 중에서 선택된 Items에 대한 Graph(Draw) 표시 부분
Zoom Control	Charting Screen 의 확대/축소/복귀 작업 버튼 - Zoom In : 한번 Click시 마다 현재 Screen 대비 120% 확대 한다. - Zoom Out : 한번 Click시 마다 현재 Screen 대비 80% 축소 한다. - Zoom Off : 100% 화면 비율로 복귀한다.
Draw Item	Screen 에 Draw 하고자 하는 Item 을 선택/해제 할 수 있다.
Charting Data Area	가장 최근부터 10 개까지의 상태를 값으로 볼 수 있다.

Note

Zoom In 에 의해 화면이 확대된 경우나 특정 부분을 화면의 중앙으로 옮기고자 할 경우에는 마우스의 오른쪽 버튼을 Charting Screen 에 대고 누른 후(Mouse Right Button Down), 원하는 위치로 Drag 하듯 끌면 원하는 위치로 이동 가능하다.

Chapter 17

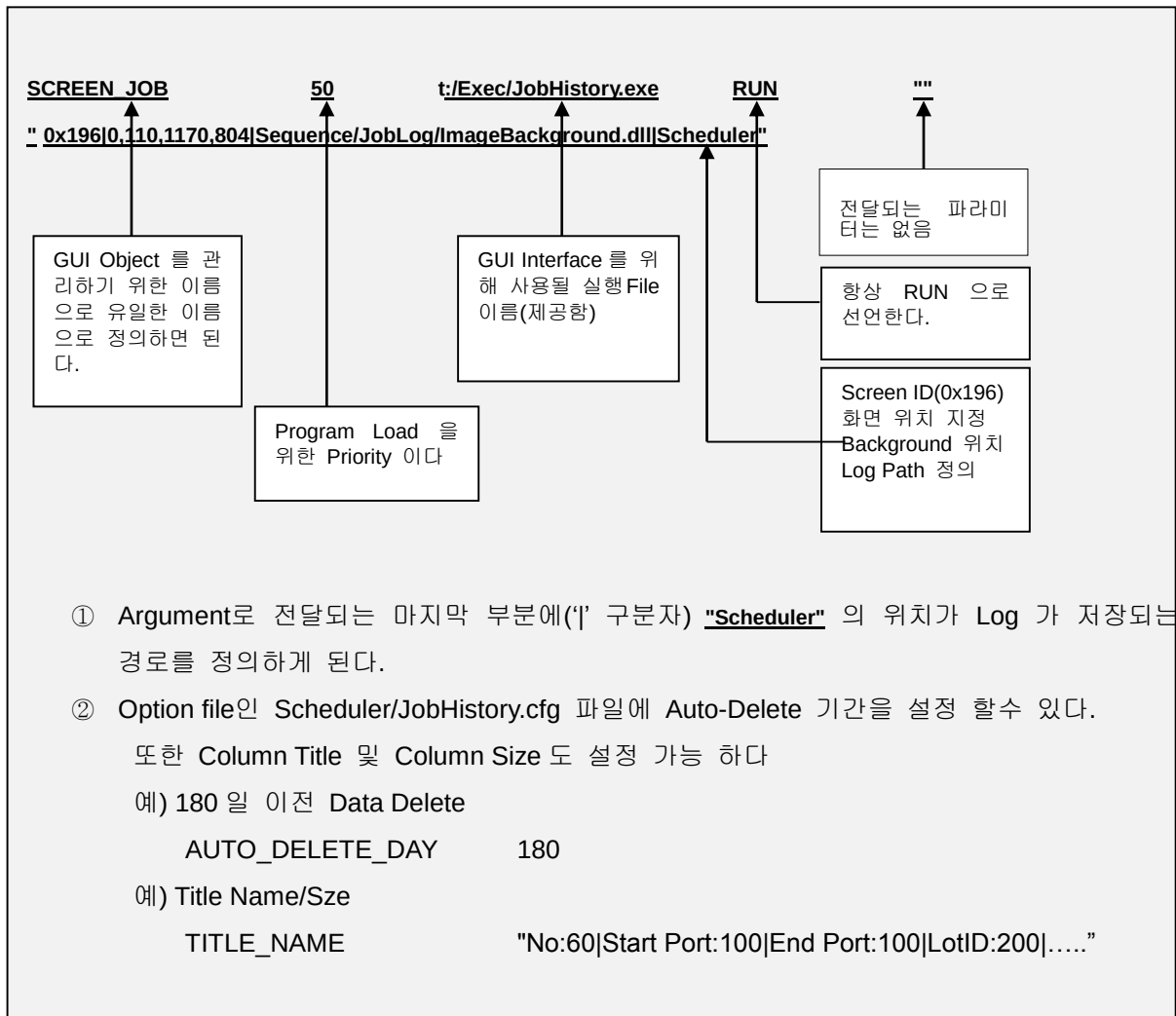
Job Log Manager

What's In this chapter...

- ◆ Job Log Manager 정의
- ◆ Operation

Job Log Manger 정의

- Auto Run 으로 현재까지 작업한 Job 의 모든 History 를 확인 할수 있는 화면이다.
- S_GUI.list(User 정의에 따라 다를 수 있음)파일에 아래와 같이 Job Log Manager 를 등록한다.



Operation

No	Start Port	End Port	LotID	CstID	PPID	Start Slot	End Slot
717	PORT2	PORT2	dummy	dummy	ITO Alloy_PM2	1	19
716	PORT1	PORT1	BPG226FQ0B00900	J6LP0701242	ITO Alloy_PM2	1	19
715	PORT2	PORT2	test	osm	ITO Alloy_PM1_PM2	1	19
714	PORT1	PORT1	TR6	3	ITO Alloy_PM1_PM2	1	19
713	PORT2	PORT2	BPG226FQ0B01300	J6LP0701190	ITO Alloy_PM2	1	19
712	PORT1	PORT1	BPG256FM0B02690	hsk	ITO Alloy_PM2	1	19
711	PORT1	PORT1	a	a	ITO Alloy_PM1	1	19
710	PORT1	PORT1	DPA036FS0B00800	hsk	ITO Alloy_PM2	1	19
709	PORT1	PORT1	a	a	ITO Alloy_PM2	1	19
708	PORT1	PORT1	tr	psh	ITO Alloy_PM1	1	19
707	PORT2	PORT2	v5tr	psh	WF300B7ZZH	1	19
706	PORT1	PORT1	1	1	ITO Alloy_PM1	1	19
705	PORT2	PORT2	2	2	P20-120_PM1	1	19
704	PORT2	PORT2	1	1	ITO Alloy_PM1	1	19
703	PORT1	PORT1	1	1	P120_PM1	1	19
702	PORT1	PORT1	DPAZY6FQ0B00200	J6LP1000046	ITO Alloy_PM2	1	19
701	PORT1	PORT1	BPG256FM0B02300	J6LP1000045	ITO Alloy_PM2	1	19
700	PORT1	PORT1	a	a	ITO Alloy_PM2	1	19
699	PORT1	PORT1	DPA056FQ0B00300	hsk	ITO Alloy_PM2	1	19
698	PORT1	PORT1	a	a	ITO Alloy_PM1	1	19
697	PORT1	PORT1	a	a	ITO Alloy_PM2	1	19
696	PORT1	PORT1	tr	ppp	ITO Alloy_PM1	1	19
695	PORT1	PORT1	tr	psh	ITO Alloy_PM1	1	19
694	PORT1	PORT1	v6tr	psh	EF490B7ZZH	1	19
693	PORT1	PORT1	osm	osmzr	P20-120_PM1	1	19
692	PORT2	PORT2	DPAZY6FS0B00100	J6LP1000053	ITO Alloy_PM1_PM2	1	19
691	PORT1	PORT1	BPGZY6FQB00500	J6LP1000050	ITO Alloy_PM1_PM2	1	19
690	PORT2	PORT2	BPAZY6FQ0B00200	J6LP1000052	ITO Alloy_PM1_PM2	1	19
689	PORT1	PORT1	BPGZY6FM0B01600	J6LP1000051	ITO Alloy_PM1_PM2	1	19
688	PORT2	PORT2	BPG226FQ0B01100	J6LP0701275	ITO Alloy_PM2	1	19
687	PORT1	PORT1	v5tr	psh	WF300B7ZZN	1	19
686	PORT1	PORT1	tr	psh	ITO Alloy_PM1	1	19
685	PORT1	PORT1	v6tr	psh	EF490B7ZZH	1	19
684	PORT1	PORT1	tr	psh	ITO Alloy_PM1	1	19

Delete Search View Data Up Down PgUp PgDn Left(<) Right(>)

Figure. Jog Log Manager Screen

• 기능 설명 (Control Area)

Item	설 명	비 고
Delete	선택되어진 Job Log 항목을 삭제한다.	
Search	Job Log 가 대량으로 기록된 경우 원하는 Job Log 를 찾기 어려워진다. 이 때, 조건을 입력하여 원하는 Job Log 를 검색할 수 있다.	Figure 36-1 참조
View Data	해당 Job 에 기록된 Log 를 다음 항목별로 볼 수 있다. - Lot Transfer History - Wafer Process History - Wafer Transfer History - Process Data - Process Recipe	Figure 36-2 참조
Up / Down PgUp / PgDn	Log 항목이 많을 때 Up / Down 이동 할 수 있다.	
Left / Right	왼쪽 / 오른쪽으로 이동할 수 있다.	

Chapter 18

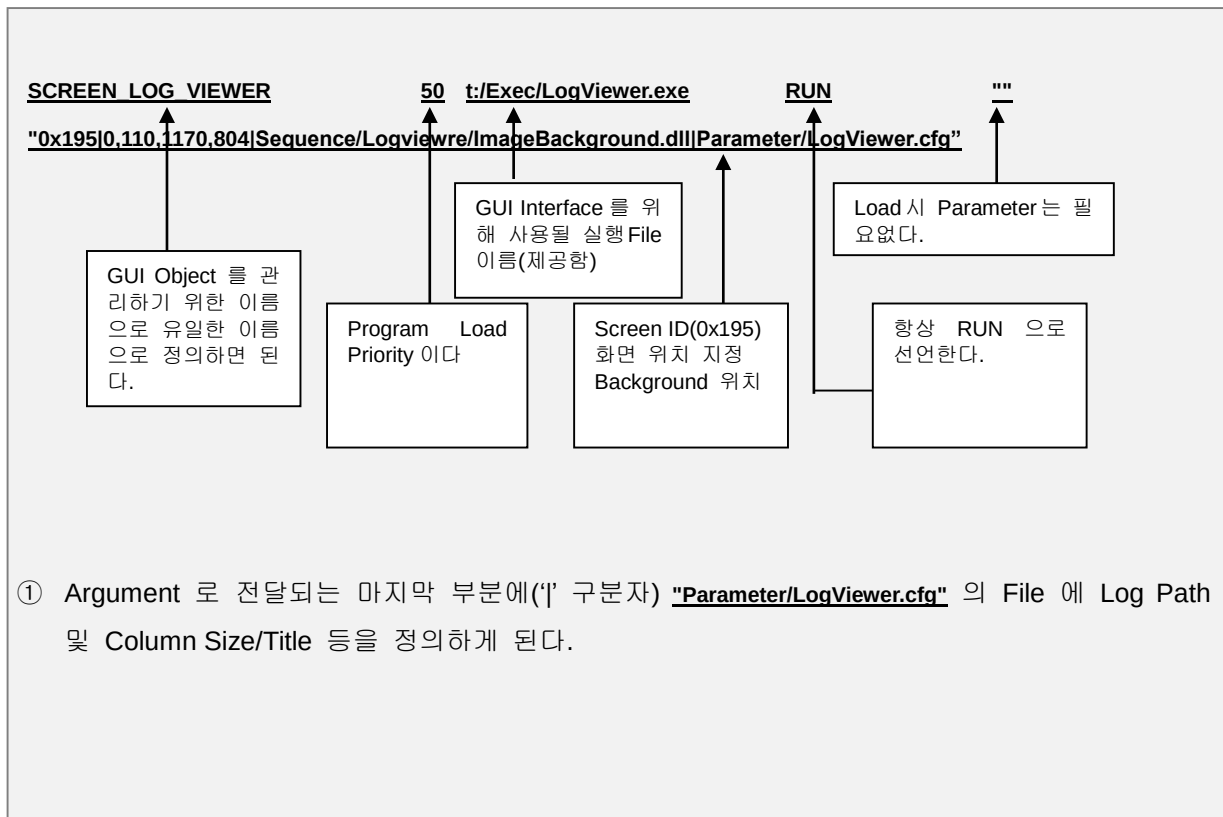
ETC Log Viewer

What's In this chapter...

- ♦ ETC Log Viewer 정의
- ♦ ETC Log Viewer 환경 File
- ♦ Operation

ETC Log Viewer 정의

- Equipment 에 저장된 여러가지 Data 를 확인 가능한 GUI 용 Sequence 이다.
- S_GUI.list (User 정의에 따라 다를 수 있음) 파일등에 아래와 같이 ETC Log Viewer 를 등록한다.



ETC Log Viewer 환경 File

- Parameter/Logviewer.cfg 파일 내에 다음과 같이 정의한다.

예)

```
"OPERATOR LOG" "History" "OP_" LISTVIEW|R "/" " " "Date""Description:700"
```

- ① GUI Display Title : "OPERATOR LOG"
- ② Log 경로 : "History"(History 폴더 정의)
- ③ File 이름 : "OP_" ("OP_"로 시작되는 모든 파일)
- ④ Column Display Type :
 - LISTVIEW : Default Left Side 정렬
 - "" |R : Right Side 정렬
 - "" |C : Center Side 정렬
- ⑤ 주석 처리 : "/"("/"로 되어 있는 부분은 보여주지 않음)
- ⑥ Log file separation char : Column display 구분자
- ⑦ Column display Title : "Date""Description"
- ⑧ Column display Size : "Description:700" (Description Size 를 700 으로 정의)

Note

ETC Log Viewer 는 User 가 정의한 환경 File 에 의존하여 Directory, File List 생성된다.
각종 통신 및 User 가 편의를 위해 추가한 Log 등을 상기 환경 File 에 Script 로 정의하여
Log Viewer Screen 에서 Log File 을 참조할 수 있도록 설계되어 있다.

Operation

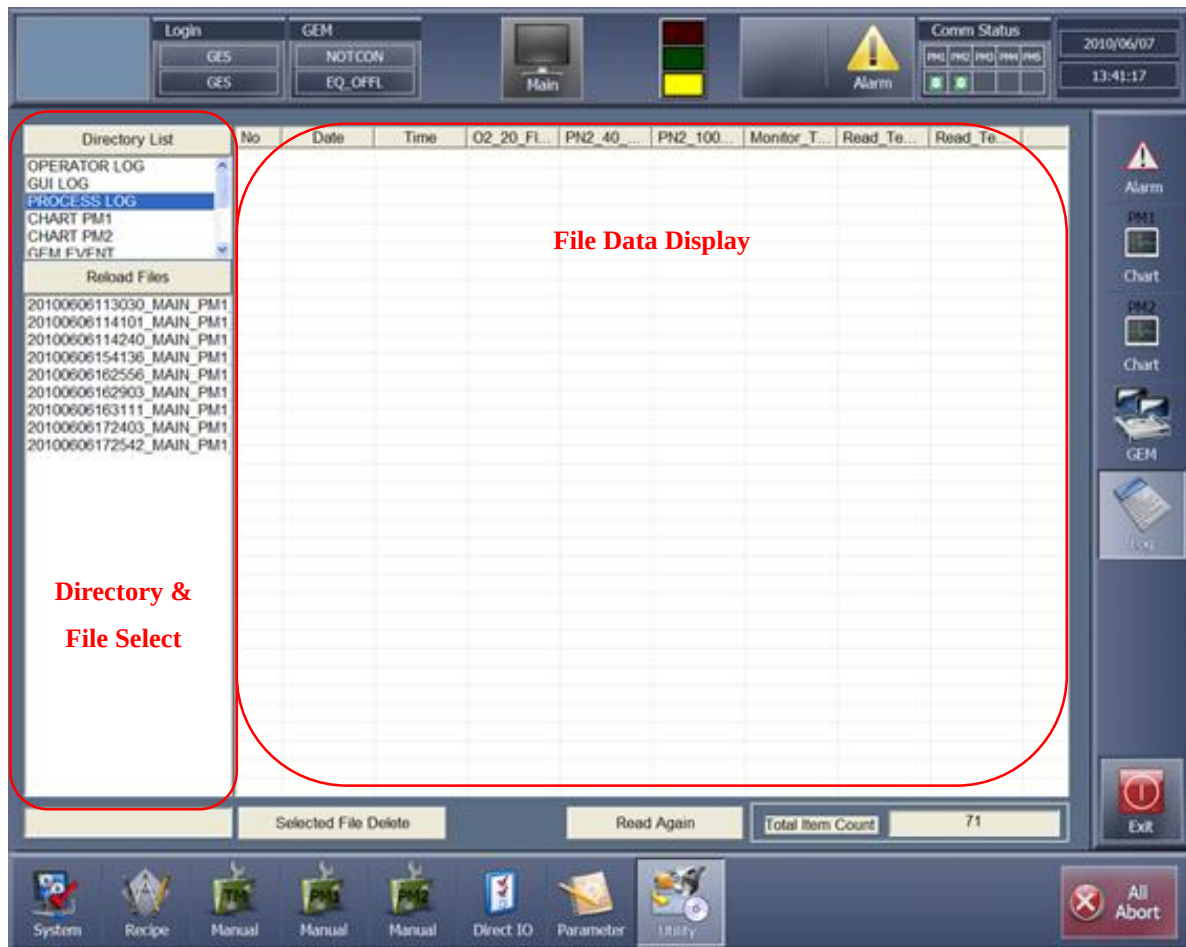


Figure . ETC Log Viewer

- 좌측에 Directory 및 File 을 선택하면, 우측에 File Data 가 보여 진다.
- Selected File Delete Button 을 이용하여 user 가 선택한 Log File 을 삭제할 수 있다.

Chapter 19

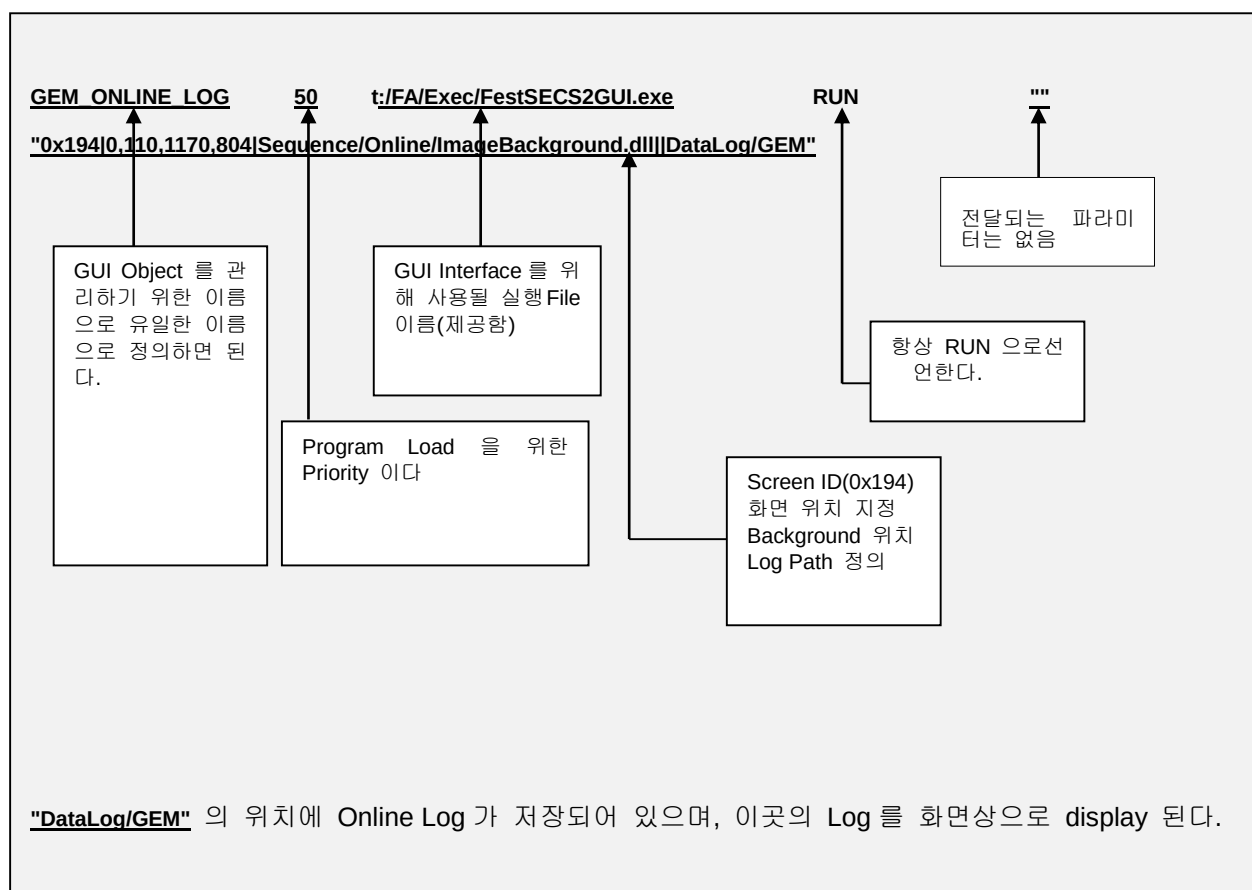
Online Log Viewer

What's In this chapter...

- ◆ Online Log Viewer 정의
- ◆ Operation (Binary Mode)
- ◆ Operation (SECS(Ascii) Mode)
- ◆ Search

Online Log Viewer 정의

- Host 와 Equipment 사이의 주고 받은 Data 를 확인 가능한 Screen 이다.
- S_GUI.list (User 정의에 따라 다를 수 있음) 파일등에 아래와 같이 Alarm Manager 를 등록한다.



Note

Online Log Viewer 는 User SW 에 FEST GEM(FA)가 탑재되어 Host 와 통신이 가능한 상태로 개발되어졌을 때에만 유효하다.

Host 와 통신하지 않는 StandAlone 장비의 SW 에서는 필요로 하지 않는다.

Operation (Binary Mode)

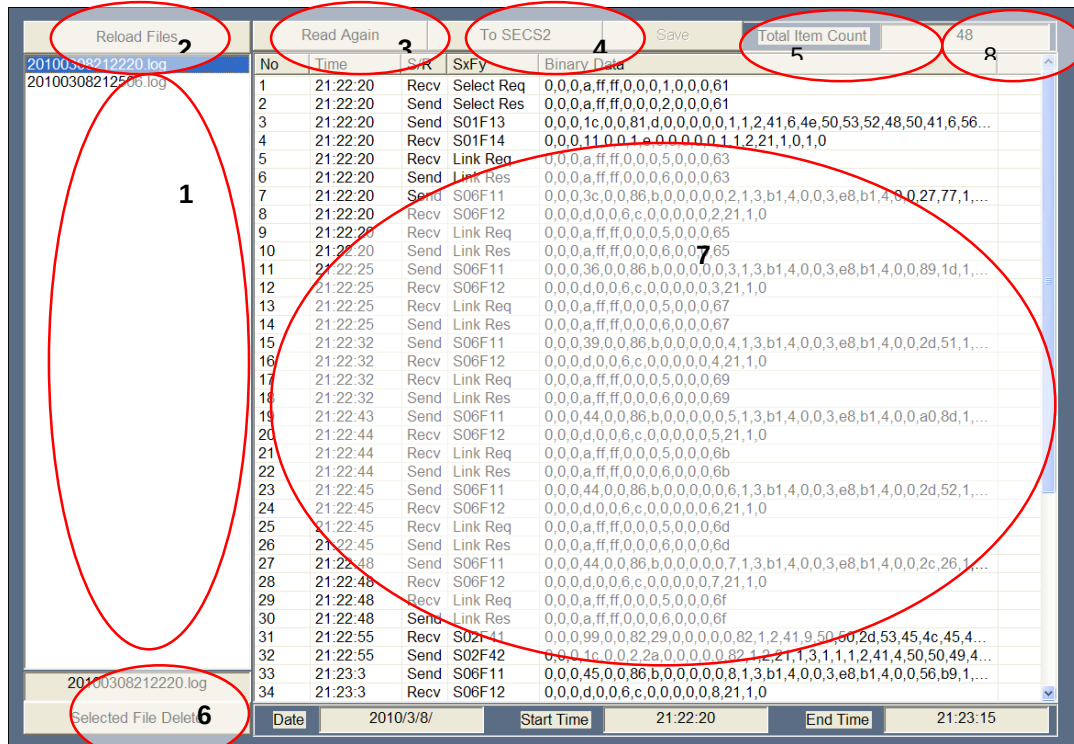


Figure .

Online Log Viewer (Binary Mode)

• 기능 설명

No	내 용	Etc
1	File의 List를 보여준다.	
2	File의 List를 다시 읽어온다.	
3	Select된 현재 File의 내용을 다시 읽어온다.	
4	Binary Type / Ascii(SECS-2) Mode간 toggle한다.	
5	현재 선택된 File이 가지고 있는 Item의 개수를 나타낸다.	
6	현재 선택된 File을 삭제한다.	
7	현재 선택된 File의 내용을 보여준다. 크게 5가지의 Column을 가지고 있으며 첫번째는 순번이고 2번째는 발생 일시,시간이고 세번째는 현재 Item이 보낸 Data(Send)인지 받은 Data(Recv)인지를 나타내며 네번째는 Item의 SECS-2 의 어떤 Type인지를 나타내며 다섯번째는 Binary Data를 Hexadecimal 기준으로 나열한 내용이다.	
8	검색 기능을 제공한다. 자세한 것은 Figure 31-2. 참조	Figure 31-2.

Operation (SECS2(Ascii) Mode)

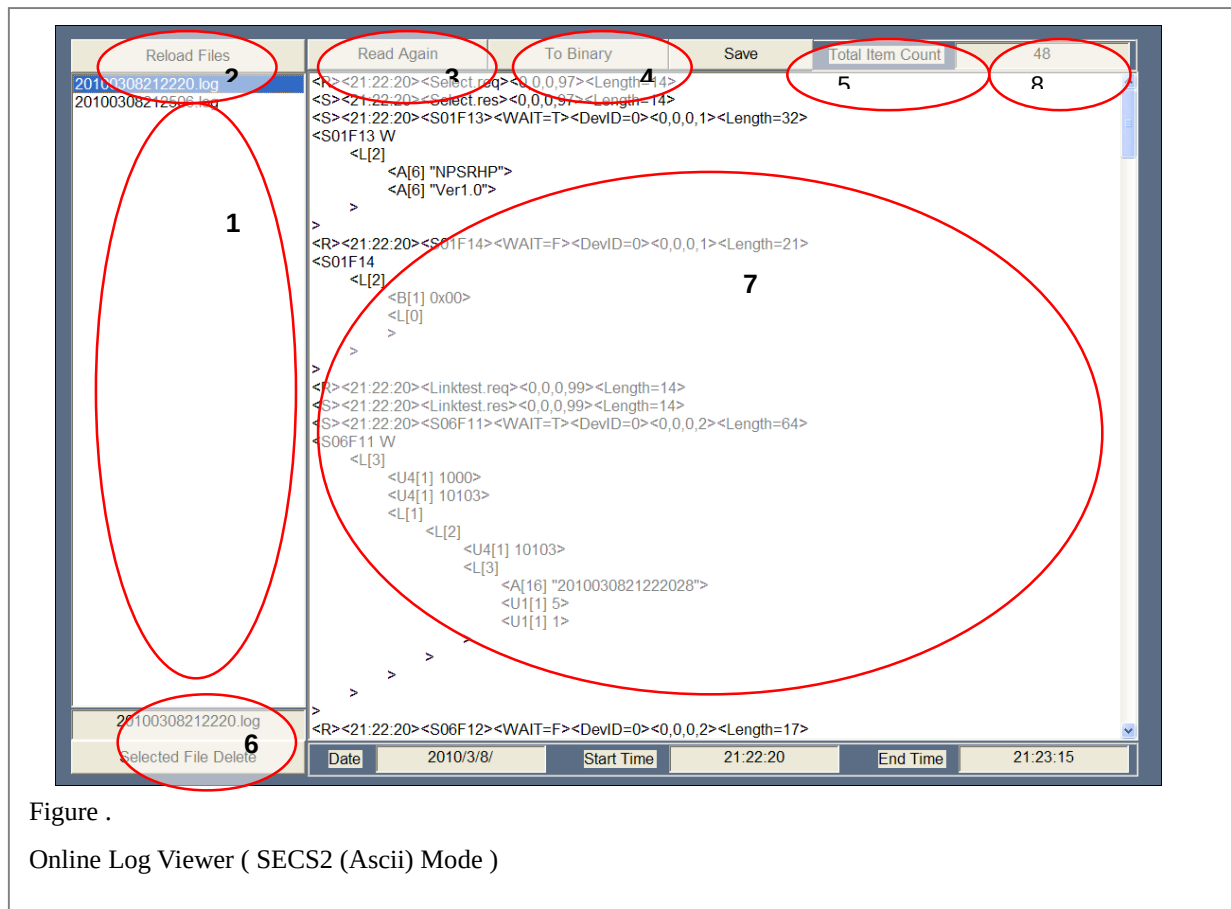


Figure .

Online Log Viewer (SECS2 (Ascii) Mode)

• 기능 설명

No	내 용	Etc
1	File의 List를 보여준다.	
2	File의 List를 다시 읽어온다.	
3	Select된 현재 File의 내용을 다시 읽어온다.	
4	Binary Type / Ascii(SECS-2) Mode간 toggle한다.	
5	현재 선택된 File이 가지고 있는 Item의 개수를 나타낸다.	
6	현재 선택된 File을 삭제한다.	
7	현재 선택된 File의 내용을 Ascii(SECS-2) Type으로 보여준다. ">" 로 시작되는 Data는 Receive Data "<" 로 시작되는 Data는 Send Data	
8	검색 기능을 제공한다. 자세한 것은 Figure 31-2. 참조	Figure 31-2.

Search

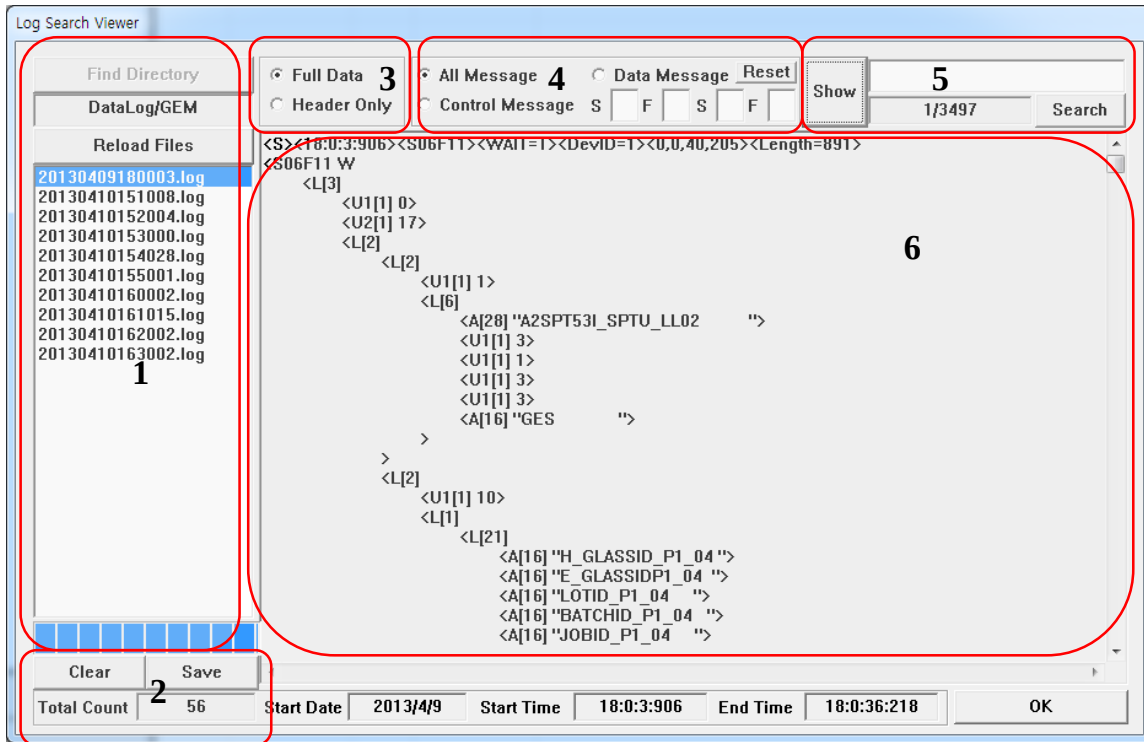


Figure . Online Log Viewer 의 Search Dialog

• 기능 설명

No	Item	설명
1	List	Online Log List 중에서 검색하고자 하는 대상 File 을 선택한다. 한 개 File 또는 복수 File 에서 검색도 가능하다.
2	Clear	검색되어진 결과를 Clear 한다
	Save	검색되어진 결과를 별도 File 에 저장한다.
3	Full Data	검색되어진 모든 결과를 보여준다.
	Header Only	검색되어진 결과중 Header message 만 보여준다.
4	All Message	검색시에 모든 message 중에서 검색한다.
	Control Message	검색시에 Control message 중에서 검색한다.
	Data Message	검색시에 Data message 중에서 검색한다.
	SxFxSxFx	검색하고자 하는 Stream message number 를 입력한다.
	Reset	SxFxSxFx 에 입력한 message 를 clear 한다.
5	Show	1~4 에서 선택되어진 결과에 맞추어 화면을 갱신한다.
	Search	Search button 상단에 특정문자를 입력 후 Search 하면 해당 문자가 포함된 Message 만 보여준다.
6	Search Result	1~5 에서 선택과 검색에 의해 Filtering 된 Data 를 보여준다.

ETC

Chapter 20: System 예약 IO

Chapter 21: 제한사항

Chapter 22: Scheduler Interface DLL

Chapter 20

System 예약 IO

What's In this chapter...

- ◆ System 예약 IO 란?
- ◆ T:\IO\System.io File

■ System 예약 IO 란?

- Engine 내부의 내용을 IO 를 통하여 참조할 수 있다.
- T:\IO\System.io File 에 정의되어 있으며 사용자가 확장하여 사용할 수 없다.
- System IO 는 SYSTEM 이라는 이름의 Driver 를 선언해 주어야 한다.

(ex)

```
System.AlarmStatus  DI SYSTEM eALMSTS - %d 1 1 0 0 Poll
System.Year         DI SYSTEM      -    - %03d 0 0 0 1 Poll
```

Note

System IO 도 내부 선언되는 Driver 의 통로를 통해 동작된다.

즉, 실제 Device Driver 는 아니지만 내부 상태 참조시에도 Driver 형태로 정의되어야 한다.

T:\IO\System.io File

- GES 사에서 현재 정의한 System IO 는 다음과 같다.
- 추후 기능상의 변경으로 추가 개선될 수 있다.

IO Name	Type	ID1	ID2	ID3	ID4	Contents
System.Year	DI	0	0	0	1	System 년/월/일/시/분/초/밀리초
System.Month	DI	0	0	0	2	
System.Day	DI	0	0	0	3	
System.Hour	DI	0	0	0	4	
System.Min	DI	0	0	0	5	
System.Sec	DI	0	0	0	6	
System.MilSec	DI	0	0	0	7	
System.Date	SI	0	1	0	0	System 날짜 id3 : 0=yyyy/mm/dd , 1=dd/mm/yyyy
System.Time	SI	0	2	0	0	System 시간
System.Interlock	DO	1	0	0	0	System 인터락
System.AlarmStatus	DI	1	1	0	0	System Alarm 발생상태 <i>Enum : 0=ABSENT, 1=PRESENT</i>
System.AlarmCount	DI	1	2	0	0	System 에 현재 발생되어 있는 Alarm 개수
System.AlarmBuzzer	DI	1	3	0	0	신규 Alarm 발생 여부 Indicate New Alarm Happened , Need to Reset(DISABLE) by Application <i>Enum : 0= DISABLE , 1= ENABLE</i>
System.SimulMode	DI	2	1	0	0	Project 가 Simulation Mode 로 기동되었는지 여부 <i>Enum : 0= REAL , 1= SIMULATION</i>
System.ExitLevel	DI	3	1	0	0	[10/21/2013] : Add User Level

Chapter 21

제한사항

What's In this chapter...

- ◆ 제한사항

■ 제한사항

- Main Engine 에서는 Memory 관리를 위하여 아래와 같은 제한사항을 바탕으로 설계되었다.
- 충분한 여유를 두었으나 아래 제한사항을 초과하여 User SW 를 제작하는 경우 Project Loading 이 불가할 수 있다.

Item	Restriction	Unit
Maximum Driver Name	32	byte
Maximum IO Count	1,000,000	ea
Maximum IO Name	64	byte
Maximum String IO Value	512	byte
Maximum Enum Data	128	Byte
Maximum Enum String	48	Byte
Maximum Digital IO Value	0x7FFFFFFF	hex
Minimum Digial IO Value	0xFFFFFFFF	hex
Maximum Lookup String	48	byte
Maximum Sequence Name	64	byte
Maximum Alarm Name	48	byte

Chapter 22

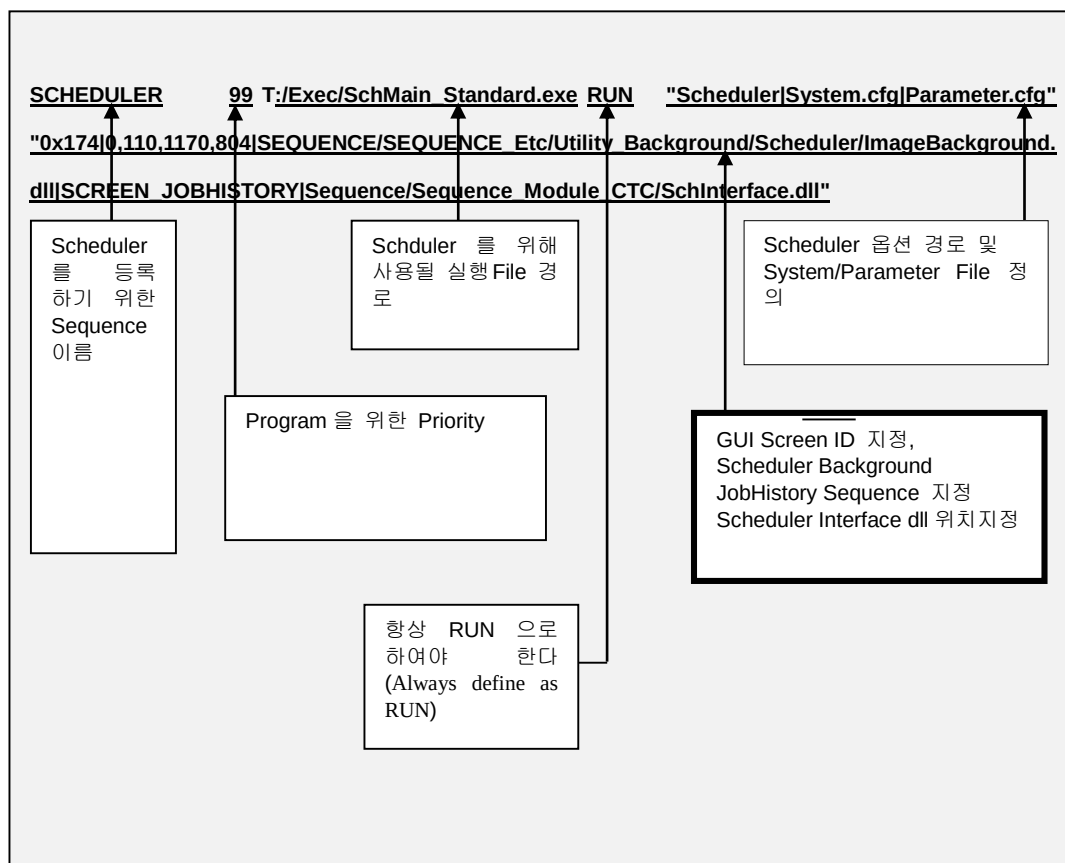
Scheduler Interface Dll

What's In this chapter...

- ◆ Scheduler Interface Dll 정의
- ◆ User Interface DLL Function
- ◆ Scheduler Diagram

Scheduler Interface Dll 정의

- Scheduler 에서 이 Interface 을 사용하기 위한 방법은 아래와 같이 선언하면 된다.



- Interface DLL에서는 Scheduler가 Job을 생성하기 이전에 User가 Root/Route/Process Recipe 등을 변경/조작하고자 할 때 매우 유용하다. Job Start 전 뿐만 아니라 Transfer 중에도 참조할 수 있는 Interface API를 제공하므로 User는 해당 API에 적당한 작업을 통하여 보다 세밀한 제어를 할 수 있도록 하고 있다.

User Interface DLL Function

void UserSeqPrepareSetup(TSchControlJob* pControlJob, UserInterfaceSetupMaterialData* pMaterialData)

Return Type	Void	
Parameter	TSchControlJob*	SystemSetup()을 진행 할 Control Job Pointer
	UserInterfaceSetupMaterialData*	Control Job 에서 운용할 DB 정보
Call Time	각 Module Remote Sequence (PORTx.SETUP(). PMx.SETUP()...etc)이 Call 되기 직전에 Call 된다.	

bool UserSeqPrepareReady(TSchControlJob* pControlJob, UserInterfaceSetupMaterialData* pMaterialData)

Return Type	bool	
Parameter	TSchControlJob*	SystemReady()을 진행 할 Control Job Pointer
	UserInterfaceSetupMaterialData*	Control Job 에서 운용할 DB 정보
Call Time	각 Module Remote Sequence (PORTx.READY(). PMx.READY()...etc)이 Call 되기 직전에 Call 된다.	

void UserSeqPrepareEnd(TSchControlJob* pControlJob, int iPattern)

Return Type	Void	
Parameter	TSchControlJob*	SystemEnd()을 진행 할 Control Job Pointer
	UserInterfaceSetupMaterialData*	Control Job 에서 운용할 DB 정보
Call Time	각 Module Remote Sequence (PORTx.END(). PMx.END()...etc)이 Call 되기 직전에 Call 된다.	

int UserSeqProcessJobStart(TSchProcessJob* pProcessJob, int iQueuedPjCount, int iPjIndex)

Return Type	int	
Parameter	TSchProcessJob *	Executing Process Job Pointer
	int iQueuedPjCount	Process Job 이 속한 Control Job 의 모든 PJ Count.
	int iPjIndex	Process Job 의 Index (from 0)
Call Time	PROCEJOB 이 Executing 이 될 때 발생한다.	

int UserSeqProcessJobEnd(TSchProcessJob* pProcessJob, int iQueuedPjCount, int iPjIndex)

Return Type	int	
Parameter	TSchProcessJob *	End Process Job Pointer
	int iQueuedPjCount	Process Job 이 속한 Control Job 의 모든 PJ Count.
	int iPjIndex	Process Job 의 Index (from 0)
Call Time	PROCEJOB 이 End 될 때 발생한다.	

bool UserSeqPickCompleteFM (TSchWafer* pWafer, int module, int slot, int arm, int finger, bool bswitch)

Return Type	bool	False return 시 Scheduling will be ABROTED.
Parameter	pWafer *	Picked Wafer Pointer.
	int module, int slot	Picked Module & Slot.
	int arm, int finger	FM's Arm & Finger(Default = 1)
	bool bswitch	Switching 여부.
Call Time	PROCEJOB 이 End 될 때 발생한다.	

bool UserSeqPlaceCompleteFM(TSchWafer* pWafer, int module, int slot, int arm, int finger, bool bswitch)

Return Type	bool	False return 시 Scheduling will be ABROTED.
Parameter	pWafer *	Placed Wafer Pointer.
	int module, int slot	Placed Module & Slot.
	int arm, int finger	FM's Arm & Finger(Default = 1)
	bool bswitch	Switching 여부.
Call Time	PROCEJOB 이 End 될 때 발생한다.	

bool UserSeqPickCompleteTM (TSchWafer* pWafer, int module, int slot, int arm, int finger, bool bswitch)

Return Type	bool	False return 시 Scheduling will be ABROTED.
Parameter	pWafer *	Picked Wafer Pointer.
	int module, int slot	Picked Module & Slot.
	int arm, int finger	TM's Arm & Finger(Default = 1)
	bool bswitch	Switching 여부.
Call Time	PROCEJOB 이 End 될 때 발생한다.	

bool UserSeqPlaceCompleteTM(TSchWafer* pWafer, int module, int slot, int arm, int finger, bool bswitch)

Return Type	bool	False return 시 Scheduling will be ABROTED.
Parameter	pWafer *	Placed Wafer Pointer.
	int module, int slot	Placed Module & Slot.
	int arm, int finger	TM's Arm & Finger(Default = 1)
	bool bswitch	Switching 여부.
Call Time	PROCEJOB 이 End 될 때 발생한다.	

int UserSeqProcessStart(TSchControlJob* pControlJob, TSchProcessJob* pProcessJob, TSchWafer* pWafer, int module, int slot)

Return Type	int	● USER_PROCESS_START(0) : Processing Start ● USER_PROCESS_RESERVE(1):MOUDULE_PROCESS_START를 받을 때까지 Processing 대기. ● USER_PROCESS_NEXT(2):새로운 Wafer 투입시까지 대기
Parameter	TSchControlJob *	Placed Wafer 의 Control Job Pointer.
	TSchProcessJob*	Placed Wafer 의 Process Job Pointer.
	TSchWafer*	Placed Wafer Pointer.
	int module, int slot	Placed Module & Slot information.
Call Time	PROCEJOB 이 End 될 때 발생한다.	

bool UserSeqIsPossiblePickFM (TSchWafer* pWafer, int module, int slot)

Return Type	bool	False return 시 Pick 가능한 Wafer 중에서 제외된다.
Parameter	pWafer *	FM Robot 이 Pick 하고자 하는 Wafer Pointer.
	int module, int slot	FM Robot 이 Pick 하고자 하는 Module & Slot.
Call Time	Pick Wafer 를 찾는 알고리즘에서 최종 Pick 가능성 여부를 User 에게 Confirm 받을수 있다.	

bool UserSeqIsPossiblePlaceFM (TSchWafer* pWafer, int module, int slot, int arm, int finger)

Return Type	bool	False return 시 Place 가능한 Wafer 중에서 제외된다.
Parameter	pWafer *	FM Robot 이 Place 하고자 하는 Wafer Pointer.
	int module, int slot	FM Robot 이 Place 하고자 하는 Module & Slot.
	int arm, int finger	FM Robot 이 Place 하고자 하는 Arm/Finger Information (Finger Default = 1)
Call Time	Pick Wafer 를 찾는 알고리즘에서 최종 Pick 가능성 여부를 User 에게 Confirm 받고자 할 때 사용한다..	

bool UserSeqIsPossiblePickTM (TSchWafer* pWafer, int module, int slot)

Return Type	bool	False return 시 Pick 가능한 Wafer 중에서 제외된다.
Parameter	pWafer *	TM Robot 이 Pick 하고자 하는 Wafer Pointer.
	int module, int slot	TM Robot 이 Pick 하고자 하는 Module & Slot.
Call Time	Pick Wafer 를 찾는 알고리즘에서 최종 Pick 가능성 여부를 User 에게 Confirm 받고자 할 때 사용한다..	

bool UserSeqIsPossiblePlaceTM (TSchWafer* pWafer, int module, int slot, int arm, int finger)

Return Type	bool	False return 시 Place 가능한 Wafer 중에서 제외된다.
Parameter	pWafer *	TM Robot 이 Place 하고자 하는 Wafer Pointer.
	int module, int slot	TM Robot 이 Place 하고자 하는 Module & Slot.
	int arm, int finger	TM Robot 이 Place 하고자 하는 Arm/Finger Information (Finger Default = 1)
Call Time	Pick Wafer 를 찾는 알고리즘에서 최종 Pick 가능성 여부를 User 에게 Confirm 받고자 할 때 사용한다..	

BOOL UserSeqAppendProcessJobAtControlJob(TSchControlJob* pControlJob, UserInterfacePRJobData& PRJobData)

Return Type	BOOL	● FALSE : SKIP ● TRUE : Create the Process Job
Parameter	TSchControlJob*	Control Job Pointer
	UserInterfacePRJobData *	생성할 Process Job 정보
Call Time	User Interface function, UserSeqPrepareReady() Call 되기 이전에 사용된다.	

Scheduler Diagram

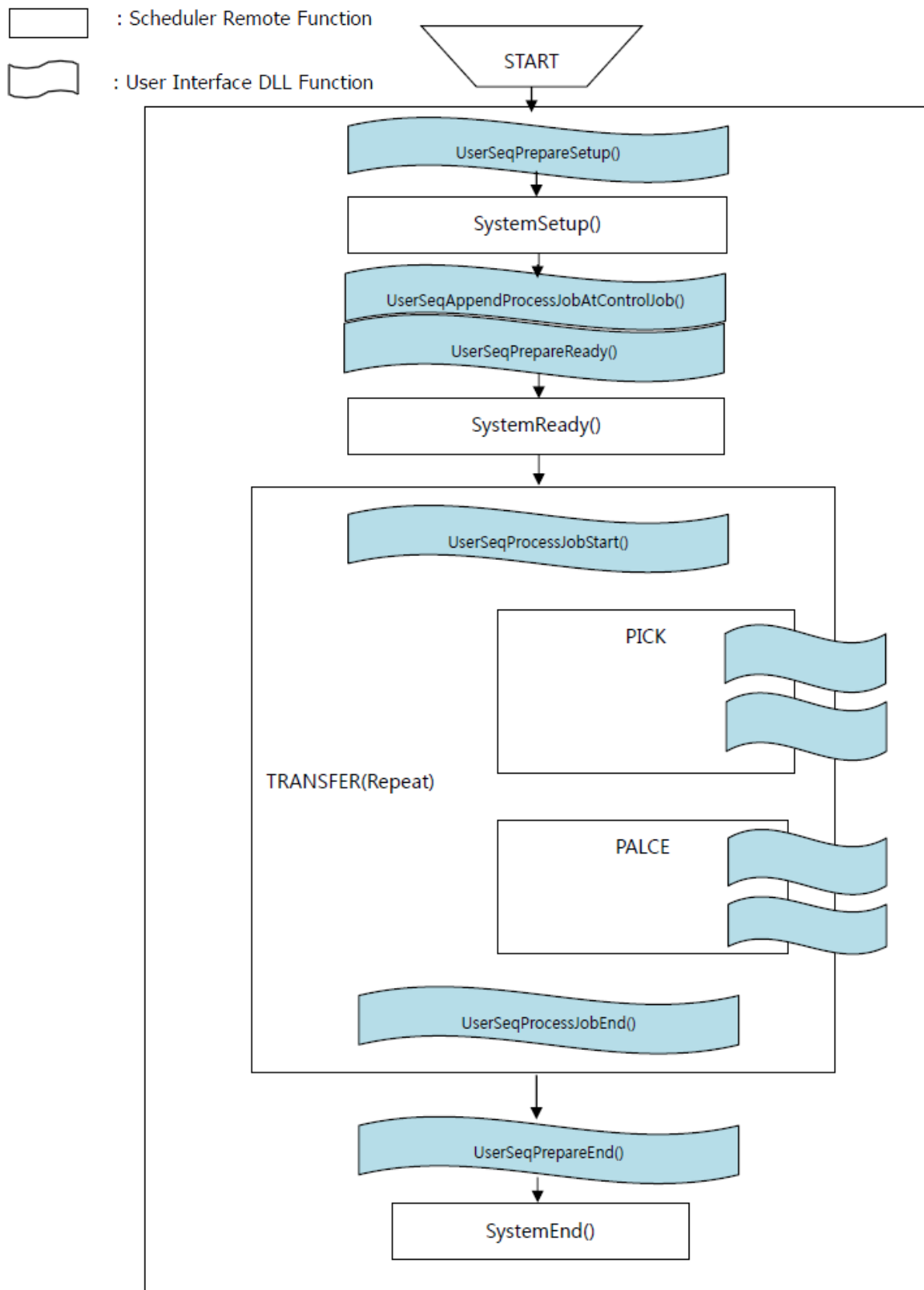


Figure. User Interface Dll Function call Timing in Scheduler Diagram



(주)게스

우 305-500

대전광역시 유성구 용산동 533-1 번지 미건테크노월드Ⅱ C 동 440 호

전화 : 042-671-3331

G.E.S

#440C-Gate Migun TechnoWorld Ⅱ, 533-1

Yongsan-dong, Yuseong-gu, Daejeon, Korea

Tel : +82-42-671-3331

URL : <http://www.gesolutions.co.kr/>